

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas Informáticos



Generación estructurada de archivos YAML a
partir de lenguaje natural mediante modelos
de lenguaje

TRABAJO DE FIN DE MÁSTER

Presentado para la obtención del título de Máster por:

Mario García Berenguer

Madrid, 2026



UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



Máster en Aprendizaje Automático y Datos Masivos

Generación estructurada de archivos YAML a partir de lenguaje natural mediante modelos de lenguaje

TRABAJO DE FIN DE MÁSTER

Presentado para la obtención del título de Máster por:

Mario García Berenguer

Bajo la supervisión de:
Dr. Javier Huertas Tato

Madrid, 2026

<Dedicatoria (opcional)>

Agradecimientos

<Página de agradecimientos (opcional)>

< Si el trabajo ha recibido financiación de alguna convocatoria competitiva, por favor, indíquelo aquí escribiendo: "Este trabajo ha sido parcialmente apoyada por...". De lo contrario, elimine este texto para dejar esta sección en blanco. >

Abstract

<Resumen en inglés: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Resumen

<Resumen en español: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Índice general

Agradecimientos	iii
Abstract	v
Resumen	vi
Índice de figuras	ix
Índice de tablas	xiii
Abreviaturas y acrónimos	xvii
1 Introduction	1
1.1 Motivación	1
1.2 Descripción	2
1.3 Objetivos	4
2 Estado del arte	7
2.1 Modelos de lenguaje autoregresivos y la tensión con las salidas jerárquicas . .	7
2.2 De texto libre a elementos técnicos estructurados	8
2.3 Fiabilidad de salidas estructuradas en LLMs	10
2.4 Control estructural: gramáticas, parsers y constrained decoding	11
2.5 Predicción estructurada y representación explícita de jerarquía	12
2.6 Adaptación supervisada eficiente: SFT, LoRA y QLoRA	13
2.7 Kubernetes como dominio de evaluación estructurada	14
2.8 Optimización por preferencias y alineamiento post-SFT	15
3 Análisis inicial	17
3.1 El conjunto de datos	17
3.1.1 Descripción general	17
3.1.2 Distribución de tipos de recurso	18
3.2 Representación jerárquica: la secuencia nivel-línea	20
3.2.1 El contrato de estructura	20
3.2.2 Estadísticas de profundidad y complejidad	20
3.2.3 Heterogeneidad estructural por tipo de recurso	22
3.2.4 Presencia de campos semánticos	23
3.3 Análisis de los prompts	24
3.3.1 Variantes y longitud	24
3.3.2 Similitud entre variantes	24
3.3.3 Requisitos capturados	25

3.3.4	Correlación entre longitud y complejidad	26
3.4	Observaciones con implicaciones de diseño	26
3.4.1	La pared del nivel 5: desbalanceo de profundidad	27
3.4.2	Sintaxis vs. estructura: la brecha del 86 %	27
3.4.3	Anti-patrones de seguridad presentes en el training data	28
4	Diseño Experimental y Métricas de Evaluación	29
4.1	Planteamiento experimental y comparabilidad	29
4.1.1	Formulaciones supervisadas	30
4.2	Formato de datos y representación habitual	30
4.2.1	Unidad básica: bloque estructural	30
4.2.2	Granularidad del dataset durante los experimentos	31
4.2.3	Pipeline de transformación: YAML a CSV/TSV	31
4.3	Protocolo de entrenamiento y evaluación	32
4.4	Métricas de evaluación: familias y utilidad	33
4.4.1	Métricas sintácticas y estructurales	34
4.4.2	Métricas de contenido y semántica de dominio	34
4.4.3	Validez de dominio Kubernetes: escala por niveles	35
4.4.4	Métricas de cumplimiento de prompt y agregación	37
4.5	Criterios de trazabilidad hacia resultados	37
5	Resultados I: Línea base y ajuste supervisado serializado	39
5.1	Línea base zero-shot	39
5.2	<code>serialized_sft</code> : ajuste con jerarquía en superficie	41
5.3	Análisis de errores del SFT serializado	46
6	Resultados II: Ajuste Fino con Predicción Explícita de Nivel	49
6.1	Motivación: los límites de la supervisión implícita	49
6.1.1	Análisis de la señal latente	50
6.2	Diseño de la arquitectura de dos cabezales	51
6.3	Resultados del experimento base: THM v1	53
6.3.1	Colapso hacia los niveles superficiales	55
6.4	Los modelos clasificadores ordinales	56
6.4.1	Variante inicial y tasa de aprendizaje de umbrales (lr25)	56
6.4.2	MLP de tres capas (mlp3)	58
6.5	Umbrales centrados: el primer acceso a los niveles profundos	58
6.6	Análisis de errores del THO centrado	59
6.6.1	Condicionamiento posicional del cabezal ordinal	62
6.6.2	Segundo experimento posicional: modulación FiLM/gating	65
7	Alineación Mediante Optimización de Preferencias Directa	69
7.1	Generación de candidatos para DPO	69
8	Impacto Social y Ética	75
8.1	Naturaleza y alcance del sistema	75
8.2	Encaje regulatorio y jurídico	76

8.3	Riesgos principales	77
8.4	Medidas de mitigación y responsabilidad	77
8.5	Resumen de impacto	78
9	Análisis Comparativo de Resultados	79
9.1	Criterios de comparación	79
9.2	De la línea base al SFT serializado	80
9.3	Jerarquía serializada frente a jerarquía explícita	81
9.4	Evolución de la rama two-head	82
9.5	Efecto de DPO sobre la referencia serializada	83
9.6	Evaluación final en test	83
9.7	Discusión transversal	84
10	Conclusiones y Trabajo Futuro	87
	Configuración experimental y reproducibilidad	95
.1	Entorno de hardware y software	95
.2	Modelo base y cuantización	96
.3	Configuración LoRA	96
.4	Hiperparámetros de entrenamiento por variante	98
.5	Configuración de inferencia y evaluación	103
.6	Reproducibilidad y trazabilidad	103
	Evaluación ética y matriz de riesgos	105
.7	Clasificación bajo el AI Act	105
.8	Factores RGPD	105
.9	Checklist ética AI HLEG	106
.10	Matriz de riesgos	106

Índice de figuras

3.1	Balance del dataset por split y control de fugas entre entrenamiento, validación y test.	18
3.2	Distribución de <i>kinds</i> en el corpus Kubernetes.	19
3.3	Distribución de versiones de API en los manifiestos del dataset.	19
3.4	Distribución de bloques por nivel de indentación en el conjunto canónico. . .	21
3.5	Distribuciones de complejidad estructural en el dataset canónico.	22
3.6	Matriz de confusión del <i>latent probe</i> para predicción de nivel de indentación.	23
3.7	Cobertura de campos semánticos relevantes en manifiestos Kubernetes del dataset.	24
3.8	Distribución de longitud de prompt por variante (<i>question</i> y <i>simplified</i>). . . .	25
3.9	Similitud semántica entre variantes de prompt por muestra.	26
3.10	Relación entre longitud de prompt y complejidad estructural del YAML. . .	27
4.1	Pipeline metodológico desde la representación canónica hasta la evaluación. .	32
5.1	Estado de las predicciones del baseline zero-shot en el split de validación. De las 70 muestras, 65 producen una salida estructurada parseable, pero solo 20 dan lugar a un YAML válido tras la reconstrucción determinista. El cuello de botella no es el formato de bloque sino la consistencia jerárquica dentro de él.	41
5.2	Curva de pérdida de entrenamiento del serialized_sft (LoRA, 3 épocas, 159 pasos globales, 424 muestras de entrenamiento efectivas (426 nominales, 2 omitidas por OOM)). La pérdida desciende de 1,32 en el primer paso a valores inferiores a 0,2 al final de la primera época. Las épocas segunda y tercera refinan los casos más complejos, con valores de pérdida entre 0,03 y 0,09. . .	43
5.3	Comparativa de métricas clave entre el baseline zero-shot y el serialized_sft en el split de validación. El salto de 31 % a 99 % en parseo YAML y de 28 % a 96 % en clave semántica muestra que el SFT resuelve de forma simultánea el problema estructural y el de cobertura semántica. La predicción de nivel exacto mejora de 11 % a 76 %, confirmando que la serialización implícita de jerarquía es suficiente para aprender indentaciones correctas.	44
5.4	Distribución de errores de dominio del serialized_sft en el split de validación. Los cuatro antipatrones de seguridad (rojo) acumulan 470 instancias; los errores de semántica de dominio (verde) se limitan a 7 instancias. El perfil refleja que el SFT ha resuelto el problema estructural pero ha heredado los patrones inseguros presentes en los datos de entrenamiento.	47

6.1	Recall por nivel del mejor aproximador lineal (<code>line_first_token</code>) sobre las representaciones del modelo base. Los niveles 0–4 muestran señal sólida; a partir del nivel 5 el recall cae por debajo del 32 %, y los niveles 7–8 son inestables por escasez de muestras.	51
6.2	Arquitectura del THM: el backbone compartido alimenta simultáneamente el cabezal de lenguaje y el cabezal de nivel. La política <code>record_prefix_state</code> lee el estado oculto justo antes del primer token de cada bloque para supervisar la predicción de nivel.	52
6.3	Curvas de pérdida durante el entrenamiento del THM v1: pérdida total, componente LM y componente de nivel ($\lambda = 1, 0, 3$ épocas, 159 pasos globales). . .	54
6.4	Comparativa visual de métricas clave entre el baseline zero-shot y THM v1 en el split de validación.	54
6.5	Distribución comparada de niveles: ground truth de validación (azul) y predicciones del THM v1 (rojo). El modelo emite cero predicciones para los niveles 5 a 8; la zona sombreada marca la región de colapso.	55
6.6	Comparativa visual de las cuatro variantes ordinal en cuatro métricas: parseo YAML, coincidencia exacta de nivel, F1 de contenido textual y recall off-by-one en niveles profundos (5–8).	57
6.7	Evolución de los ocho umbrales τ_k durante el entrenamiento de la variante <i>centered-gap05</i> (159 pasos, 3 épocas). A pesar del alto factor de tasa de aprendizaje ($\times 50$), los umbrales apenas se desplazan respecto a su inicialización.	59
6.8	Trade-off entre tasa de parseo YAML y recall exacto de niveles profundos (5–8) para las cinco variantes two-head probadas. Solo la variante <i>centered</i> activa predicciones profundas, pero a costa de colapsar la generación de contenido.	60
6.9	Comparación entre el THO centrado y el THOP con concatenación final. La compresión de niveles 5–8 a 0–4 debe leerse en sentido inverso: valores más altos indican más colapso de niveles profundos hacia niveles superficiales. . .	63
6.10	Diagnóstico por cuartiles del experimento THOP con concatenación final. La figura muestra, a la izquierda, el exceso de predicciones <code>level=0</code> ; a la derecha, la brecha entre la profundidad media de la referencia y la profundidad media predicha.	64
6.11	Progresión de métricas estructurales al introducir codificación posicional en el cabezal ordinal. El salto de THOP-FiLM es visible dentro de la rama ordinal-posicional, aunque sigue lejos del THM v1 en las métricas globales de validación.	66
7.1	Comparación de estrategias de generación de candidatos para DPO. Los pilotos se ejecutaron sobre subconjuntos distintos, por lo que deben leerse como una exploración metodológica y no como una comparación controlada exacta. La fila C6 hot final agrega las 426 unidades de prompt usadas en la generación principal.	70
7.2	Efecto de la temperatura en los candidatos generados con la configuración C6 hot . Las temperaturas bajas conservan mejor la calidad media; las temperaturas altas incrementan el riesgo de candidatos inválidos y reducen la adecuación al prompt.	71

7.3	Temperatura que produce el mejor y el peor candidato por prompt en la generación principal C6 hot. La distribución muestra que las temperaturas bajas dominan entre los mejores candidatos, mientras que las más altas aportan una parte importante de los peores casos.	72
7.4	Distribución del margen entre el mejor y el peor candidato por prompt. El umbral principal de 0,25 prioriza pares con una diferencia clara, reduciendo el riesgo de entrenar DPO con preferencias ambiguas.	72

Índice de tablas

3.1	Resumen cuantitativo del dataset canónico utilizado en este capítulo.	18
3.2	Distribución de bloques por nivel de indentación.	21
4.1	Comparativa de formulaciones supervisadas utilizadas en el estudio.	30
4.2	Serializaciones operativas usadas en el proyecto.	32
4.3	Familias de métricas y pregunta metodológica que responden.	33
4.4	Escala de validez de dominio Kubernetes: compuertas secuenciales.	36
5.1	Métricas del baseline zero-shot en el split de validación (70 muestras totales; 65 con salida estructurada parseable).	40
5.2	Comparativa de métricas entre el baseline zero-shot y <code>serialized_sft</code> en el split de validación. Δ indica la diferencia en puntos porcentuales (métricas de tasa) o en unidades absolutas (<code>level_mae</code>).	42
5.3	Tasa de superación de la compuerta de validez de dominio Kubernetes por nivel para el <code>serialized_sft</code> en el split de validación (70 muestras). Los niveles son acumulativos: superar el nivel k implica haber superado los niveles $0, \dots, k - 1$. La compuerta final corresponde al nivel 5.	44
5.4	Comparativa de métricas entre el run de 1 época (paso 53) y el modelo final de 3 épocas (paso 159) del <code>serialized_sft</code> en el split de validación (70 muestras). Δ indica la diferencia en puntos porcentuales o en unidades absolutas (<code>level_mae</code>).	45
5.5	Perfil de errores de dominio del <code>serialized_sft</code> en el split de validación (70 muestras). Los conteos son acumulados sobre todos los recursos generados; una muestra puede contener varios recursos y contribuir con múltiples instancias del mismo tipo de error.	46
6.1	Comparación de formatos de superficie entre <code>serialized_sft</code> y <code>two_head_sft</code> . En la variante de dos cabezales, el campo de nivel desaparece del texto generado y pasa a ser una etiqueta supervisada exclusiva del cabezal estructural. . . .	52
6.2	Comparativa de métricas compartidas en el split de validación: baseline zero-shot vs. THM v1 (70 muestras). La columna Δ indica THM v1 menos baseline; en <code>average_level_mae</code> , un valor negativo indica mejora porque la métrica es un error.	53

6.3	Comparativa de las cuatro variantes ordinales en el split de validación (70 muestras). La columna “Recall exacto 5–8” recoge el recall del cabezal sobre niveles profundos. La columna “Off-by-1 profundo” es el recall con tolerancia de ± 1 para niveles 5–8.	57
6.4	Distribución de niveles por cuartil en el análisis de errores del THO centrado. El patrón más relevante aparece en Q2: el YAML de referencia casi ya no está en <code>level=0</code> , pero el modelo conserva una proporción elevada de predicciones superficiales.	60
6.5	Resultado final del primer experimento con codificación posicional por concatenación final.	63
6.6	Resultado final del experimento THOP-FiLM con modulación posicional sobre el cuello de 512 dimensiones.	66
9.1	Resumen comparativo en validación de los modelos principales. Los valores de DPO corresponden a la run <code>beta=0.10</code> ; la variante DPO v2 se mantiene fuera de la tabla hasta completar su validación final.	80
9.2	Matriz de evaluación final en test. Las filas pendientes no deben usarse para selección de modelo, sino como confirmación de modelos previamente fijados en validación.	84
1	Especificaciones del entorno de hardware.	95
2	Versiones mínimas de las dependencias principales.	96
3	Arquitectura del modelo base Qwen2.5-7B-Instruct.	97
4	Configuración LoRA común a todos los experimentos de ajuste fino.	97
5	Parámetros de entrenamiento comunes a las variantes SFT supervisadas.	98
7	Inventario de experimentos respecto al contenido previo del anexo.	99
9	Parámetros específicos de <code>serialized_sft</code>	99
11	Parámetros específicos de <code>two_head_sft</code>	100
13	Serie de variantes ordinales previas a la incorporación explícita de posición.	100
15	Parámetros específicos de las variantes ordinales con información posicional.	101
17	Parámetros específicos de las ejecuciones DPO sobre <code>serialized_sft</code>	102
19	Configuración de inferencia por variante experimental.	103
20	Niveles de riesgo del AI Act y aplicabilidad al sistema propuesto.	105
21	Factores de riesgo RGPD relevantes para el sistema en su configuración actual.	106
22	Resumen de cumplimiento de los requisitos éticos del AI HLEG para el sistema propuesto.	107
23	Principales riesgos del sistema, valoración y medidas de mitigación.	108

Abreviaturas y acrónimos

UPM Universidad Politécnica de Madrid

Capítulo 1

Introduction

Esta plantilla sigue el formato de tesis de la *Universidad Politécnica de Madrid* (UPM), que se encuentra disponible[aquí](#).

En el siguiente capítulo se proporciona información acerca de las reglas para preparar el documento de la tesis, así como la explicación del uso de esta plantilla.

1.1 Motivación

La motivación de este trabajo surge frente a una necesidad natural de todo ingeniero, la automatización, especialmente de tareas repetitivas. Con el auge de los modelos de lenguaje de gran tamaño (LLMs), se ha abierto un campo totalmente inexplorado, en el que se posibilita la generación automática de texto a partir de instrucciones en lenguaje natural (prompting) [50, 27]. Sin embargo, aunque esta capacidad es impresionante, se vuelve mucho más compleja y menos fiable cuando la salida esperada no es texto libre, sino un resultado que requiere de cierta estructura formal, no solo sintáctica, sino también jerárquica y semántica. Este problema se manifiesta claramente, por ejemplo, en la generación de manifiestos YAML de Kubernetes, donde una salida que puede parecer razonable desde el punto de vista lingüístico, puede ser inválida, inconsistente o directamente inutilizable desde el punto de vista técnico. En este contexto, no basta con que el modelo “entienda” el prompt: debe además construir una representación del resultado que respete la estructura requerida por el dominio.

En este sentido, la motivación técnica central de este trabajo se enmarca en el estudio de la generación estructurada de YAML a partir de lenguaje natural. El interés del problema surge, en parte, de las limitaciones que presentan las soluciones generalistas actuales. Un LLM convencional, utilizado únicamente mediante prompting o incluso tras un ajuste supervisado básico, genera la salida de manera autoregresiva token a token, siguiendo el paradigma de los modelos Transformer modernos [39]. Es decir, su mecanismo natural consiste en producir una secuencia plana de tokens, uno detrás de otro. Este mecanismo no ofrece, por sí mismo, garantías sobre la estructura global de la salida, como la que esperarías conseguir en un manifiesto YAML, que debe respetar una jerarquía de claves y valores, así como dependencias internas entre distintos componentes. A primera vista, parecería que los LLMs no serían las

herramientas idóneas para este tipo de tareas, pero precisamente por eso resulta interesante estudiar cómo pueden adaptarse o complementarse para mejorar su capacidad de generar estructuras formales correctas.

A partir de esta tensión, el problema puede plantearse desde varias aproximaciones, cada una con ventajas e inconvenientes. Por un lado, podríamos tratar este desafío como un problema de estilización del texto. Es decir, el modelo aprendería a generar una salida que, aunque es texto plano, sigue un formato específico que permita más tarde reconstruir de alguna forma determinista la estructura deseada. Esta estrategia es relativamente sencilla de implementar, pero tiene la limitación de que el modelo no tiene una señal explícita sobre la jerarquía, lo que puede dificultar la generación de estructuras complejas o profundas. Por otro lado, podríamos tratar de incorporar una señal estructural explícita durante el proceso de generación, por ejemplo, mediante un cabezal adicional que prediga cierta información sobre la jerarquía o la estructura del documento. Esta estrategia es más compleja de implementar, pero podría facilitar que el modelo aprenda a organizar la información de manera más coherente y estructurada.

La elección de este tema también está motivada por el interés de trabajar sobre un caso de estudio técnicamente relevante y al mismo tiempo evaluable de forma formal. Kubernetes resulta especialmente apropiado porque permite combinar lenguaje natural, estructuras jerárquicas y reglas verificables en un dominio real, ampliamente utilizado y con una semántica técnica suficientemente rica [45, 48]. En este sentido, el proyecto no solo persigue mejorar la capacidad de un modelo para producir archivos YAML, sino también construir una metodología reproducible para estudiar cómo se comportan los LLMs cuando la tarea exige algo más que fluidez textual.

Por último, existe una motivación personal que también resulta importante en la elección del tema. Más allá del interés por los modelos de lenguaje y por las tareas de NLP, este trabajo nace de la intención de abordar un problema asociado habitualmente a sistemas de tipo “caja negra” desde una perspectiva más analítica y, en cierta medida, más matemática. En lugar de limitarse a observar el comportamiento externo del modelo, el objetivo es estudiar el problema mediante métricas objetivas, restricciones formales, análisis estructural del dataset y experimentos que permitan aislar con mayor claridad dónde aparecen los errores y por qué aparecen. Esta motivación conecta con la idea de trasladar herramientas de análisis más rigurosas a un ámbito en el que con frecuencia predominan aproximaciones muy empíricas o poco interpretables. De este modo, el proyecto no solo pretende aportar una solución práctica a una tarea de generación estructurada, sino también servir como marco para estudiar con mayor profundidad el comportamiento de los LLMs en un problema concreto, técnico y medible.

1.2 Descripción

Este Trabajo Fin de Máster se centra en el estudio de la generación estructurada de configuraciones técnicas a partir de instrucciones en lenguaje natural mediante modelos de lenguaje de gran tamaño. En particular, el proyecto aborda como caso de estudio la generación automática de manifiestos de Kubernetes, un dominio especialmente adecuado por combinar

una estructura jerárquica bien definida con restricciones sintácticas y semánticas que pueden validarse, al menos parcialmente, de forma automática.

A diferencia de las tareas de generación de texto libre, en este problema no basta con que el modelo produzca una salida lingüísticamente plausible. La salida debe respetar simultáneamente varios niveles de corrección: debe ser sintácticamente válida como YAML, mantener una estructura jerárquica coherente, utilizar claves y valores compatibles con el dominio y reflejar de forma fiel los requisitos expresados en el prompt. Además, en el caso concreto de Kubernetes, la calidad de la generación depende también de relaciones semánticas entre distintos componentes, como la correspondencia entre los selectores de un `Service` y las etiquetas de los pods que debe exponer, la presencia de campos básicos como `apiVersion`, `kind` o `metadata.name`, la definición correcta de contenedores e imágenes, o la coherencia entre volúmenes declarados y puntos de montaje usados por los contenedores.

El objetivo del proyecto es diseñar y evaluar un sistema capaz de transformar descripciones en lenguaje natural en manifiestos YAML estructurados, válidos y semánticamente consistentes dentro del dominio de Kubernetes. Para ello, el trabajo parte de la idea de que el problema debe abordarse no como una mera tarea de generación autoregresiva token a token, sino como una tarea en la que el modelo debe organizar el contenido del prompt dentro de una estructura formal. Viéndolo de esta forma, podemos entender este tipo de ficheros YAML como estructuras de árbol, donde cada entrada indentada ocupa una posición concreta dentro del documento final. Esta separación resulta especialmente relevante porque muchos de los errores observables en este tipo de tareas no provienen únicamente de una mala interpretación del prompt, sino también de la dificultad del modelo para serializar correctamente la información en un formato fuertemente restringido.

De forma esquemática, el problema que se estudia puede expresarse como una composición entre generación y reconstrucción estructural:

$$x \xrightarrow{f_\theta} B = \left((d_i, j_i, \ell_i, t_i) \right)_{i=1}^n \xrightarrow{\mathcal{P}} y.$$

En esta notación, x representa el prompt de entrada, B la secuencia de bloques predicha, d_i el índice de documento, j_i la posición de línea, ℓ_i el nivel jerárquico y t_i el texto de la línea. El operador $\mathcal{P}$ representa el parser determinista que reconstruye el manifiesto YAML final y . La expresión no pretende fijar una arquitectura concreta, sino hacer visible la idea metodológica que atraviesa el trabajo: el modelo no se evalúa solo por producir texto, sino por producir una representación intermedia que pueda convertirse de forma segura en una estructura YAML.

Metodológicamente, el proyecto se apoyará en un modelo base autoregresivo de tipo *decoder-only*, adaptado mediante técnicas de ajuste fino eficiente, como LoRA, sobre un conjunto de pares *prompt-salida* [31, 21, 11]. A partir de esa base, se compararán dos estrategias principales para estudiar la robustez estructural del sistema. La primera, denominada en el repositorio `serialized_sft`, mantendrá la jerarquía en la superficie textual de salida: el modelo generará una representación serializada en la que cada línea contiene tanto el contenido YAML como su nivel jerárquico, y un parser determinista se encargará de reconstruir el manifiesto final sin inventar claves ni reparar errores semánticos de forma oculta.

La segunda estrategia, denominada `two_head_sft`, parte de una intuición distinta. Si el YAML se interpreta como un árbol proyectado sobre una secuencia de líneas, el orden de los nodos queda dado por el propio orden de generación del modelo. Lo que no queda determinado de forma fiable es la profundidad de cada entrada. Por ello, esta rama propone separar la predicción del contenido de la predicción de la jerarquía: el modelo generará el texto de cada línea, mientras que un cabezal estructural explícito predecirá el valor `level`. La comparación entre ambas ramas permitirá estudiar si la jerarquía debe aprenderse como una parte más de la secuencia generada o si conviene modelarla como una señal estructural diferenciada.

La evaluación del sistema se diseñará específicamente para el problema de generación estructurada. Por ello, no se tomarán como referencia principal métricas textuales superficiales, una limitación ya observada en la evaluación de artefactos técnicos generados por LLMs [8, 45], ya que dos configuraciones pueden diferir léxicamente y, sin embargo, ser funcionalmente equivalentes, o parecer similares en texto y ser estructuralmente incorrectas. En su lugar, se definirán métricas en varios niveles: métricas sintácticas, como el porcentaje de salidas YAML parseables; métricas estructurales, como la reconstrucción segura desde bloques, la coincidencia de niveles `level` o la consistencia de la jerarquía; métricas de dominio Kubernetes, como la presencia de campos requeridos y la coherencia entre selectores, etiquetas, contenedores, puertos o volúmenes; y métricas de adecuación al prompt, destinadas a medir si la configuración generada refleja correctamente la intención del usuario. Esta evaluación se complementará con un análisis sistemático de errores para identificar patrones recurrentes y comprender mejor las limitaciones de cada estrategia.

1.3 Objetivos

El objetivo general de este Trabajo Fin de Máster es diseñar y evaluar un sistema basado en modelos de lenguaje capaz de generar manifiestos YAML de Kubernetes a partir de instrucciones en lenguaje natural, garantizando no solo la validez sintáctica de la salida, sino también su coherencia estructural y semántica respecto al prompt de entrada.

A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Analizar y caracterizar el dataset disponible, estudiando su tamaño, diversidad, complejidad estructural y posibles limitaciones para el entrenamiento del modelo.
- Diseñar un conjunto de métricas objetivas que permitan evaluar de forma rigurosa la calidad de las salidas generadas, considerando parseabilidad YAML, consistencia estructural, adecuación al prompt y validez de dominio Kubernetes.
- Definir una representación intermedia basada en bloques de línea, donde cada unidad contenga el texto de la línea y su nivel jerárquico `level`, de forma que la estructura pueda reconstruirse y evaluarse de manera explícita.
- Adaptar un modelo base de lenguaje al dominio de generación de configuraciones de Kubernetes mediante técnicas de ajuste fino eficientes.
- Comparar una estrategia de ajuste supervisado que serializa el nivel jerárquico como texto, `serialized_sft`, con una estrategia que incorpora un cabezal estructural explícito

para predecir dicho nivel, `two_head_sft`.

- Evaluar experimentalmente el sistema propuesto sobre un conjunto de prueba representativo, identificando fortalezas, limitaciones y patrones de error tanto en el contenido generado como en la jerarquía predicha.
- Extraer conclusiones sobre la utilidad de los mecanismos de control estructural en tareas de generación técnica, y sobre la conveniencia de tratar la jerarquía como una parte más de la superficie textual o como una señal estructural diferenciada.

Capítulo 2

Estado del arte

La generación automática de manifiestos de Kubernetes a partir de instrucciones en lenguaje natural no se puede entender como una simple tarea de generación de texto. A primera vista, el esquema es familiar: dado un prompt, el modelo produce una respuesta. Pero la salida esperada en este caso no es una explicación ni un resumen, sino un artefacto técnico cuya corrección depende de una jerarquía interna, de relaciones entre campos y de convenciones de dominio que el mecanismo autoregresivo estándar no representa de forma explícita. Esta diferencia no es solo cosmética: un modelo que entiende correctamente lo que se le pide puede, aun así, fallar al serializar la respuesta en el formato correcto, producir un YAML sintácticamente parseable pero estructuralmente incoherente, o generar un manifiesto donde los selectores de un **Service** no corresponden a las etiquetas de los **Pods** que debe exponer.

Por eso, este capítulo no revisa la literatura como si fuera una lista de técnicas que convergen hacia la solución, sino como una cadena de problemas que se van precisando. Primero se sitúa el contexto de la generación autoregresiva y la tensión que introduce en tareas con salida jerárquica. Después se analiza el salto desde el texto libre hasta los artefactos técnicos estructurados, con atención especial a los lenguajes de configuración y a los benchmarks más próximos al problema de este trabajo. A continuación se revisan los estudios sobre fiabilidad de salidas estructuradas, que muestran que el problema no está resuelto incluso en modelos avanzados. Sobre esa base se discuten dos familias de respuesta: los mecanismos de control externo mediante gramáticas y parsers, y la representación explícita de la jerarquía como señal de entrenamiento. La revisión cierra con Kubernetes como dominio de evaluación, situando el hueco concreto que motiva la comparación central del trabajo.

2.1 Modelos de lenguaje autoregresivos y la tensión con las salidas jerárquicas

La arquitectura Transformer, introducida por Vaswani et al. como alternativa basada exclusivamente en mecanismos de atención, sentó las bases del paradigma actual de modelado del lenguaje [39]. Desde entonces, el escalado de estos modelos sobre grandes corpus ha producido sistemas capaces de capturar regularidades lingüísticas, razonar sobre problemas abiertos y

adaptarse a dominios nuevos con pocas demostraciones [50]. El ajuste mediante instrucciones ha refinado adicionalmente esa capacidad general: trabajos como FLAN muestran que el entrenamiento con tareas formuladas como instrucciones mejora de forma sistemática la generalización a tareas no vistas [41], y técnicas como Self-Instruct han ampliado la escala de los datos de ajuste usando el propio modelo como generador de ejemplos [40]. Esta combinación hace que un modelo de tipo instruct sea un punto de partida razonable para transformar una petición en lenguaje natural en una salida técnica.

Sin embargo, el mecanismo fundamental de generación no cambia con el ajuste. Un LLM produce salida de forma autoregresiva: cada token se decide condicionado al prompt y a todos los tokens generados hasta ese momento. En términos probabilísticos, esto se puede escribir como:

$$p_{\theta}(y \mid x) = \prod_{t=1}^T p_{\theta}(y_t \mid x, y_{<t}),$$

donde x es el prompt, y_t el token generado en la posición t , e $y_{<t}$ la historia previa de generación. Este proceso es natural para texto libre, donde la validez local de cada fragmento puede evaluarse con cierta independencia de lo que vendrá después. En cambio, cuando la salida esperada es una estructura jerárquica, la generación secuencial introduce una tensión difícil de resolver: el modelo toma decisiones locales una a una, pero la corrección de la salida depende de propiedades globales. En un manifiesto YAML de Kubernetes, el nivel de indentación de una línea determina si su contenido pertenece a una lista, a un campo de un mapa anidado o al nivel raíz del documento. Y ese nivel no puede inferirse solo de la línea actual: depende de lo que ya se ha generado antes y de lo que el esquema espera que venga después. El resultado práctico es que un modelo puede entender que la petición describe un `Deployment` con varios contenedores y, aun así, fallar al serializar correctamente la profundidad de campos como `containers`, `volumeMounts` o `resources`.

El modelo base utilizado en este proyecto, perteneciente a la familia Qwen2.5 [31], es un punto de partida potente: su variante instruct ha sido ajustada con instrucciones diversas y ofrece conocimiento de dominio técnico suficiente para reconocer los recursos de Kubernetes y sus convenciones habituales. Pero ese conocimiento no resuelve el problema estructural. Que el modelo sepa qué es un `CronJob` o un `PersistentVolumeClaim` no garantiza que serialice correctamente su jerarquía de campos cuando genera token a token. Precisamente en esa tensión se sitúa la pregunta central de este trabajo.

2.2 De texto libre a elementos técnicos estructurados

La evaluación funcional del código, introducida con HumanEval por Chen et al., supuso un punto de inflexión en la forma de medir la calidad de los LLMs cuando la salida es un artefacto técnico [8]. Antes, las métricas de referencia eran textuales: similitud de n-gramas, distancia de edición, BLEU. Con el código, esas métricas pierden buena parte de su sentido, porque dos programas pueden ser textualmente distintos y funcionalmente equivalentes, o parecer similares superficialmente y diferir en un detalle que los hace semánticamente incompatibles.

Esta idea es igualmente válida, y si acaso más urgente, para los lenguajes de configuración. Los modelos especializados en código, como Code Llama, mejoran la capacidad de generar artefactos formales [35], pero los manifiestos YAML de Kubernetes no son código en el sentido de que el modelo los ejecute directamente: son especificaciones declarativas que un sistema externo interpreta para decidir cómo desplegar, gestionar o exponer recursos. El criterio de validez, en consecuencia, no es solo sintáctico. Un YAML puede parsearse sin errores y aun así describir un recurso que Kubernetes rechazará por campos inválidos, relaciones rotas o convenciones incumplidas.

Este matiz separa los lenguajes de configuración de los lenguajes de programación generalistas, y tiene consecuencias directas sobre cómo debe plantearse la evaluación. Trabajos sobre DSLs como Ansible YAML ya habían señalado que la gramática, el esquema y la dependencia de documentación específica del dominio introducen dificultades que los modelos generalistas no resuelven bien, y que las métricas de texto no son suficientes para capturarlas [29, 30]. Kubernetes añade una capa adicional: no solo hay convenciones locales en cada campo, sino dependencias cruzadas entre recursos. La correspondencia entre el selector de un **Service** y las etiquetas de los **Pods** que expone, el vínculo entre un **PersistentVolumeClaim** y el **PersistentVolume** que debe satisfacerlo, o la relación entre los nombres de los contenedores y los puertos que el manifiesto abre hacia el exterior son relaciones que no pueden verificarse leyendo solo una parte del documento.

Más allá de esa complejidad de dominio, hay una abstracción que ilumina el problema desde un ángulo más preciso. Un documento YAML es, en esencia, un árbol etiquetado: cada clave, valor y elemento de lista corresponde a un nodo, la relación padre-hijo viene determinada por la indentación, y el documento completo define una estructura con raíz. Generar ese documento con un modelo autoregresivo es, en el fondo, serializar ese árbol en un recorrido en profundidad: el orden de generación, línea a línea, proporciona el orden de visita de los nodos, y lo que no queda determinado de forma directa es la profundidad de cada uno. En YAML esa profundidad se expresa como indentación, pero el modelo la infiere token a token sin ninguna señal estructural explícita. Esta observación transforma la pregunta de investigación: no se trata de estudiar si el modelo puede producir texto con aspecto de YAML, sino de si puede aprender a serializar correctamente un árbol, situando cada nodo en su nivel exacto. Combinado con el texto de la línea, el valor `level` es suficiente para reconstruir el árbol completo de forma determinista, y por eso su predicción no es una anotación auxiliar, sino la variable estructural central del proyecto.

El benchmark más cercano al problema de este proyecto es CloudEval-YAML, que propone una metodología sistemática para evaluar la generación de configuraciones YAML cloud-native a partir de instrucciones en lenguaje natural [45]. A diferencia de los benchmarks de código general, CloudEval-YAML evalúa manifiestos de Kubernetes y herramientas relacionadas mediante criterios funcionales y de esquema, e identifica categorías de fallo que una métrica textual no detectaría: errores de sintaxis YAML, violaciones de esquema de campo, inconsistencias en referencias cruzadas entre recursos y desajuste con la intención del prompt. Sus resultados muestran que los modelos actuales tienen dificultades especialmente cuando el manifiesto esperado involucra múltiples recursos coordinados. Precisamente por eso, en este proyecto la evaluación no se organiza alrededor de una única puntuación de similitud, sino

como un conjunto de comprobaciones en varios niveles: parseabilidad de la salida reconstruida, exactitud de los niveles jerárquicos predichos, presencia de campos de dominio requeridos y adecuación al prompt.

2.3 Fiabilidad de salidas estructuradas en LLMs

La práctica de pedir a los LLMs que devuelvan JSON, YAML, XML o respuestas ajustadas a un esquema se ha generalizado rápidamente, impulsada por la necesidad de integrar modelos de lenguaje en sistemas que esperan salidas procesables automáticamente. Pero la fiabilidad de esas salidas sigue siendo un problema abierto que la literatura reciente ha empezado a medir con más sistematicidad. StructEval evalúa específicamente la capacidad de los modelos para generar y convertir múltiples formatos estructurados, incluyendo YAML, y encuentra que la adherencia al formato y la corrección estructural no pueden darse por supuestas incluso en modelos avanzados [46]. JSONSchemaBench llega a una conclusión similar desde la perspectiva de la generación restringida por esquemas: la complejidad del esquema afecta tanto a la cobertura de los métodos de constrained decoding como a la calidad final de las respuestas, y los fallos no son uniformes [16]. StructuredRAG muestra que seguir instrucciones de formato no elimina los fallos de consistencia cuando el modelo debe además integrar información recuperada [25]. Struc-Bench confirma que los errores en datos estructurados complejos afectan a campos, relaciones internas y restricciones de esquema, no solo a detalles superficiales [26].

Lo que estos trabajos tienen en común es que distinguen, aunque no siempre con esa terminología, tres niveles de corrección que se pueden fallar de forma independiente. El primero es la validez sintáctica: ¿puede el documento parsearse? El segundo es la validez de esquema: ¿respeta los campos y tipos que el dominio espera? El tercero es la corrección semántica: ¿describe lo que el prompt pedía? Un modelo puede superar el primer nivel y fallar en el segundo, o superar los dos y fallar en el tercero. Esta separación no es un refinamiento teórico; es la arquitectura de evaluación que este proyecto necesita para estudiar dónde exactamente aparecen los errores cuando se genera YAML de Kubernetes. Sin esa distinción, sería imposible saber si una salida incorrecta falla por un error de indentación, por una clave de esquema inválida, o porque el modelo ha interpretado mal el prompt desde el principio. La consecuencia metodológica directa es que ninguna métrica única puede caracterizar la calidad de la salida, y que construir un evaluador en varios niveles simultáneamente es parte de la contribución del proyecto.

A partir de esta evidencia, la literatura ha ido articulando dos grandes familias de respuesta al problema de las salidas estructuradas poco fiables. La primera consiste en restringir la generación mediante mecanismos externos que bloqueen continuaciones inválidas en tiempo de decodificación. La segunda consiste en modificar cómo el modelo aprende y representa la estructura internamente. Las dos secciones siguientes revisan cada una de esas direcciones y explican en qué medida se aplican a las decisiones de diseño de este proyecto.

2.4 Control estructural: gramáticas, parsers y constrained decoding

La idea detrás del constrained decoding es directa: si el modelo puede escoger cualquier token en cada paso, nada impide que produzca una secuencia sintácticamente inválida. Restringir el espacio de salida en tiempo de decodificación —mediante léxicos, gramáticas formales o máscaras sobre el vocabulario— permite garantizar que la salida pertenece al lenguaje objetivo incluso cuando el modelo solo ve los tokens que ya ha generado. Grid Beam Search introduce este mecanismo de forma general para restricciones léxicas [20], mientras que el uso de gramáticas formales extiende la idea a restricciones sobre la forma completa de la salida [7]. Shin et al. muestran que los modelos de lenguaje pueden usarse como parsers semánticos cuando se guían hacia un sublenguaje controlado que después se transforma en una representación formal [37], y Grammar-Constrained Decoding propone una formulación general donde la gramática se incorpora directamente al proceso de decodificación sin necesidad de ajuste adicional [17]. PICARD lleva esta idea al dominio de la generación de SQL: un parser incremental verifica en cada paso si la continuación parcial es compatible con la gramática SQL, bloqueando tokens que no pertenecen a ninguna extensión válida de lo generado hasta ese momento [36]. Enfoques más generales como Efficient Guided Generation y Guidance permiten programar restricciones sobre la salida de forma más flexible, incluyendo esquemas, expresiones regulares y funciones arbitrarias [42, 5].

El atractivo de estas técnicas es claro: eliminan una categoría entera de errores sintácticos sin necesidad de reentrenar el modelo. Pero tienen dos limitaciones que son relevantes para las decisiones de diseño de este proyecto. La primera es que exigen una gramática o un esquema del lenguaje de salida disponible y utilizable en tiempo de decodificación. En este trabajo, el modelo no genera YAML directamente, sino una representación intermedia en bloques donde cada línea contiene el texto visible y su nivel jerárquico. Esta representación no tiene una gramática estándar, y restringir la decodificación sobre ella añadiría una complejidad que no contribuye a la pregunta de investigación. La segunda limitación es más de fondo: el objetivo del proyecto no es garantizar que el modelo produzca algo parseable enmascarando sus errores con restricciones externas. El objetivo es estudiar qué aprende el modelo y dónde aparecen los fallos estructurales. Si la decodificación está restringida por una gramática, la señal de error queda bloqueada antes de que pueda observarse.

El parser de este proyecto cumple una función distinta a la de PICARD o Grammar-Constrained Decoding. No actúa durante la generación, sino después. Su entrada no es YAML libre, sino la secuencia de bloques que el modelo ha producido, donde cada bloque contiene el texto de la línea y el valor `level` predicho. A partir de esa representación, el parser aplica la indentación de forma determinista y reconstruye el manifiesto final. No repara errores semánticos, no inventa campos ausentes y no ajusta niveles que el modelo ha predicho incorrectamente. Si los bloques son inconsistentes con la gramática YAML, el manifiesto resultante fallará al parsearse, y ese fallo se registra como parte de la evaluación.

2.5 Predicción estructurada y representación explícita de jerarquía

La segunda familia de respuesta al problema de las salidas estructuradas poco fiables consiste en tratar la jerarquía no como una restricción externa que hay que imponer, sino como una parte explícita de lo que el modelo debe aprender. Esta idea tiene una historia larga en predicción estructurada. A diferencia de la clasificación independiente de etiquetas, la predicción estructurada considera salidas donde los componentes son interdependientes: la corrección de una parte no puede evaluarse sin tener en cuenta las demás [22]. Los Structured Prediction Energy Networks continúan esa línea modelando las dependencias entre componentes mediante una función de energía aprendida [4]. Los modelos de transición globalmente normalizados muestran que decidir localmente el mejor token siguiente sin considerar la trayectoria completa puede perjudicar la calidad global de la salida [2]. Aplicado al YAML, esto se vuelve concreto: el nivel jerárquico de una línea no puede decidirse de forma puramente local, sino que depende de la estructura que ya se ha generado y de la que se espera a continuación.

Una pregunta que surge de inmediato es si esa información jerárquica ya está codificada en las representaciones internas del modelo. Los structural probes de Hewitt y Manning muestran que una parte de la información sintáctica puede recuperarse de los estados internos de los Transformers mediante operaciones lineales simples [18], y los análisis de los patrones de atención en BERT sugieren que determinadas cabezas aprenden a atender a relaciones sintácticas sin supervisión explícita [9]. Si se aplica este razonamiento al caso de YAML de Kubernetes, es razonable pensar que un modelo con conocimiento del dominio codifica de alguna forma la profundidad esperada de cada campo en sus representaciones internas. Pero que esa información esté implícita no equivale a que esté siendo usada de forma efectiva durante la generación. La diferencia entre tener la información disponible y actuar sobre ella de forma fiable es precisamente el problema que se quiere estudiar: si hacer la jerarquía una señal de entrenamiento explícita mejora la serialización, eso sugiere que el modelo no estaba usando esa información de forma óptima cuando quedaba enterrada en la superficie textual.

En generación de código, la idea de modelar explícitamente la estructura de la salida tiene precedentes claros. Abstract Syntax Networks generan código siguiendo la estructura del árbol de sintaxis abstracta correspondiente, de modo que cada decisión de generación corresponde a un nodo en el árbol y no a un token arbitrario [32]. El modelo sintáctico de Yin y Neubig incorpora reglas gramaticales directamente en el proceso de generación, de forma que el árbol se construye expansión a expansión y cada producción está garantizada por el sistema de tipos de la gramática [47]. Estos trabajos parten de la misma intuición que motiva la arquitectura de este proyecto: si la salida es un árbol, lo más natural es modelar el árbol, no la secuencia plana que resulta de serializarlo. La diferencia es que en código la gramática del lenguaje define explícitamente las reglas de expansión del árbol, mientras que en YAML la jerarquía no tiene una gramática formal estándar. Lo que sí existe es una representación equivalente: el recorrido en profundidad del árbol YAML produce una secuencia de líneas, y cada línea queda caracterizada por su texto y su profundidad en el árbol:

$$B = \left((t_i, \ell_i) \right)_{i=1}^n.$$

En esta representación, t_i es el contenido visible de la línea y ℓ_i su nivel jerárquico. Dado el orden de generación, que reproduce el recorrido, conocer la profundidad de cada nodo es condición suficiente para reconstruir el árbol completo de forma determinista. Esa es la razón por la que predecir `line_level` de forma explícita, con un cabezal estructural separado, es una hipótesis con sentido: no como un truco de ingeniería, sino como una traducción directa de la estructura del árbol subyacente a una señal supervisada.

En la arquitectura `two_head_sft`, el modelo base genera el contenido de cada línea como tarea estándar de modelado del lenguaje, mientras que un cabezal estructural adicional predice el valor `level` de esa línea a partir de los estados internos del transformer. Las dos tareas coexisten en el mismo modelo pero tienen señales de supervisión distintas. Cuando hay dos pérdidas de naturaleza diferente, la pregunta de cómo equilibrarlas no es trivial: la literatura sobre aprendizaje multitarea ofrece técnicas como la ponderación por incertidumbre de Kendall et al., que asigna pesos automáticamente a partir de la varianza de cada objetivo [24]. La comparación con `serialized_sft`, donde el nivel se aprende como texto adicional dentro de la secuencia, permite aislar el efecto de esa separación. Si las predicciones de jerarquía mejoran cuando se proporciona una señal diferenciada, eso es evidencia de que el mecanismo autoregresivo no era suficiente para aprender el nivel de forma fiable solo desde la superficie textual. Si no mejoran, o si la parseabilidad global empeora, eso también es informativo sobre los límites y los costes de la separación explícita.

2.6 Adaptación supervisada eficiente: SFT, LoRA y QLoRA

El ajuste supervisado, o SFT, es la forma más directa de especializar un modelo cuando se dispone de pares de entrada y salida que representan el comportamiento deseado. En lugar de optimizar una recompensa externa, el modelo aprende a imitar demostraciones del formato esperado minimizando la pérdida de verosimilitud sobre la secuencia objetivo:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t}).$$

Esta estrategia es especialmente adecuada en la fase inicial de un proyecto de generación estructurada: antes de saber si el modelo puede aprender la tarea, lo más informativo es ver hasta dónde llega la imitación supervisada con los recursos disponibles [43]. En este caso, el dataset procesado contiene pares de prompt y manifiesto YAML normalizado, además de la representación estructural derivada en bloques con `level`. Esa representación es la que permite construir los targets para las dos ramas del experimento con los mismos datos de entrenamiento.

Ajustar completamente un modelo de varios miles de millones de parámetros no es viable en la mayoría de los entornos académicos. LoRA resuelve este problema congelando los pesos

principales del modelo e introduciendo matrices de bajo rango entrenables en las capas de atención, reduciendo el número de parámetros actualizados en varios órdenes de magnitud [21]. QLoRA extiende la idea al entrenamiento sobre modelos cuantizados en 4 bits, lo que hace posible adaptar modelos grandes dentro de las restricciones de memoria de una GPU de consumo [11]. En este proyecto, el modelo base es Qwen2.5-7B-Instruct cuantizado en 4 bits, y el ajuste se aplica mediante adaptadores LoRA tanto en la rama `serialized_sft` como en `two_head_sft`. La segunda rama añade además un cabezal estructural que se entrena simultáneamente con los adaptadores, por lo que la eficiencia del ajuste no es solo una concesión a los recursos disponibles, sino una condición para que el experimento sea reproducible y comparable entre ramas.

El papel del SFT en este trabajo no es únicamente mejorar el estilo de la salida. Es la primera etapa de una comparación supervisada que tiene como objetivo aislar si la jerarquía se aprende mejor cuando se modela de forma explícita o cuando se deja que el modelo la absorba como parte del texto serializado. La literatura sobre post-entrenamiento coincide en que el SFT sigue siendo una fase necesaria antes de aplicar métodos de preferencias más complejos [41], y que la calidad de ese ajuste inicial condiciona fuertemente la estabilidad de etapas posteriores [44]. Por tanto, la comparación entre las dos ramas supervisadas debe resolverse antes de plantear ninguna extensión mediante alineamiento por preferencias.

2.7 Kubernetes como dominio de evaluación estructural

Kubernetes utiliza manifiestos YAML para describir recursos como `Pod`, `Deployment`, `Service`, `ConfigMap`, `CronJob`, `DaemonSet` o `PersistentVolumeClaim`. El manifiesto se escribe como texto, pero el sistema que lo procesa espera objetos con campos específicos, tipos concretos y relaciones internas que no se deducen de la sintaxis. Un `Service` con `selector` incorrecto no falla al parsearse: simplemente no encontrará los `Pods` que debería exponer. Un `Deployment` sin límites de recursos en sus contenedores es válido para el API de Kubernetes, pero puede comprometer la estabilidad del clúster. Esta diferencia entre lo que se ve y lo que se valida es el rasgo que hace de Kubernetes un dominio de evaluación especialmente útil para este trabajo: la corrección es comprobable de forma aproximada y automática, a distintos niveles, sin necesidad de ejecutar el sistema ni de evaluación humana.

La literatura empírica sobre manifiestos Kubernetes reales muestra que los errores de configuración son frecuentes y variados. Zhang et al. estudian sistemáticamente los defectos de configuración en repositorios de Kubernetes y encuentran que las herramientas estáticas solo cubren una fracción de los fallos observados, porque muchos involucran relaciones entre objetos que no pueden verificarse analizando un recurso aislado [49]. Rahman et al. estudian las misconfigurations de seguridad en manifiestos de código abierto y muestran que los errores más peligrosos suelen ser consecuencia de campos omitidos o de valores por defecto que el usuario no ha considerado [34]. Fonseca et al. y Bufalino et al. analizan los modos de fallo de Kubernetes desde la perspectiva de la infraestructura, documentando escenarios donde pequeños errores de configuración de red o de política producen comportamientos difíciles de depurar [15, 6]. Kakarla et al. abordan el problema de la detección automática de bugs

en configuraciones mediante técnicas de análisis de datos, mostrando que la variabilidad estructural de los ficheros reales complica la definición de reglas de validación genéricas [23].

Frente a ese panorama de errores en manifiestos escritos por humanos, la pregunta de qué ocurre cuando los manifiestos los genera un LLM ya tiene algunas respuestas concretas, aunque escasas. Zhang et al. estudian qué tipos de smells introducen los modelos generativos en manifiestos de Kubernetes y encuentran que los patrones más frecuentes incluyen la ausencia de límites de recursos en los contenedores, el uso de imágenes sin versión fija y la omisión de campos de seguridad que no son exigidos sintácticamente pero que son necesarios en producción [48]. Estos resultados son relevantes para este proyecto porque muestran que los fallos de los modelos generativos en este dominio no son simplemente errores de formato: son errores de conocimiento de dominio que se manifiestan en manifiestos sintácticamente válidos pero semánticamente deficientes. Ueno y Uchiumi estudian la migración de workloads de contenedores a Kubernetes usando LLMs y proponen una metodología de evaluación basada en la corrección de los manifiestos generados, incluyendo la comprobación de campos esenciales [38]. Angi et al. proponen un sistema de generación de manifiestos desde una descripción de intención usando pocos ejemplos como contexto, y evalúan la calidad de las salidas en un entorno de industria con métricas que van más allá de la similitud textual [3].

El benchmark más próximo al problema central de este trabajo es CloudEval-YAML, que propone una metodología sistemática para evaluar la generación de configuraciones YAML cloud-native a partir de instrucciones en lenguaje natural, incluyendo Kubernetes como dominio principal [45]. Su contribución más relevante para este proyecto no son solo los resultados de los modelos que evalúa, sino las categorías de fallo que identifica y jerarquiza: errores de sintaxis YAML, violaciones de esquema de campo, inconsistencias de referencia cruzada entre recursos y desajustes con la intención del prompt. Estas categorías reflejan directamente la estructura de la evaluación diseñada en este proyecto: parseabilidad de la salida YAML reconstruida, exactitud de los niveles jerárquicos predichos, presencia de campos de dominio requeridos y adecuación al prompt.

Lo que ninguno de estos trabajos estudia de forma directa es la cuestión que este proyecto plantea. CloudEval-YAML, los trabajos de Angi et al. y Ueno et al., y los análisis de smells y defectos se centran en el *qué*: si el manifiesto generado es correcto, si los campos están presentes, si las relaciones son coherentes. Este proyecto se centra también en el *cómo*: cómo se representa y aprende la estructura jerárquica del manifiesto, y si hacer ese aprendizaje explícito, separando la predicción de contenido de la predicción de profundidad, mejora la fiabilidad de la serialización. Esa pregunta no ha sido planteada en la literatura específica de Kubernetes, y responderla requiere una metodología que haga la jerarquía observable y evaluable por separado del contenido textual.

2.8 Optimización por preferencias y alineamiento post-SFT

Una vez que el modelo ha sido ajustado mediante SFT, puede construirse una señal de preferencia automática a partir de comprobaciones de validez sobre las salidas generadas.

Un manifiesto que parsea correctamente y supera las comprobaciones de dominio puede tratarse como preferido frente a uno que falla en alguno de esos criterios. Esta señal, aunque automática y aproximada, es suficiente para plantear una etapa de optimización directa de preferencias sin necesidad de anotación humana.

DPO reformula el problema del alineamiento por preferencias de forma que el modelo se ajusta directamente sobre pares de salidas preferidas y rechazadas, sin entrenar un modelo de recompensa separado ni aplicar RL *online* [33]. Esto reduce considerablemente la complejidad de la infraestructura y hace que la técnica sea accesible en entornos con recursos limitados. La comparación con PPO muestra que DPO es competitivo en múltiples configuraciones, aunque su comportamiento ante distribuciones muy distintas de las del SFT puede ser menos estable [44]. La literatura sobre las limitaciones del ajuste supervisado sugiere que la generalización fuera de distribución puede mejorarse con fases de alineamiento posteriores, siempre que el punto de partida supervisado sea suficientemente competente [43].

En el contexto de este proyecto, el alineamiento por preferencias es una extensión posible una vez resuelta la comparación supervisada central. Primero es necesario establecer cuál de las dos ramas produce manifiestos con mejor consistencia estructural y mayor tasa de parseabilidad. Solo cuando esa pregunta tiene respuesta cobra sentido preguntarse si una etapa adicional de DPO con señales automáticas de validez puede reducir el porcentaje de salidas no parseables o mejorar la adecuación al prompt. El alineamiento por preferencias no sustituye al SFT, sino que depende de él.

Capítulo 3

Análisis inicial

El objetivo de este capítulo es caracterizar el conjunto de datos y los prompts con los que trabajamos, no como una simple enumeración de estadísticas, sino como un análisis que muestre las tensiones estructurales del problema. El análisis inicial sirve para anclar las decisiones de diseño posteriores: explica por qué ciertos modelos fallan de formas específicas, por qué ciertas métricas son necesarias y qué esperamos ver en los experimentos. Además, contrasta estas observaciones con los resultados ya obtenidos, de forma que cada característica del dataset encuentra confirmación (o matiz) en lo que el modelo descubre durante el entrenamiento.

Todos los manifiestos analizados aquí son *canónicos*: han sido parseados, validados y serializados de forma determinista, partiendo del mismo tipo de pares prompt-manifiesto que emplean benchmarks recientes de generación de YAML cloud-native [45, 1]. El proceso seguido es simple: dado un prompt en lenguaje natural, el modelo debe generar una secuencia de bloques estructurados (documento, índice de línea, nivel de indentación, texto) cuya reconstrucción mediante indentación produce un manifest YAML válido. Esto supone ya que el YAML está normalizado.

3.1 El conjunto de datos

3.1.1 Descripción general

El dataset base está compuesto por 283 muestras que contienen un total de 338 manifiestos Kubernetes válidos. Cada muestra incluye dos variantes de prompt en lenguaje natural: una descripción más elaborada y una versión simplificada. Esto suma un total de 566 pares prompt-manifest en la distribución de entrenamiento. El dataset se divide en tres conjuntos: 213 muestras para entrenamiento, 35 para validación y 35 para evaluación final.

Un aspecto importante del preprocesado fue el control de fugas (*leakage*). Se identificaron 51 muestras que contienen manifiestos exactamente duplicados, y todas ellas fueron agrupadas en el mismo conjunto (siempre entrenamiento). Además, se detectaron 51 pares de prompts duplicados exactamente y aproximadamente 15 pares con similitud mayor a 0.97, que también fueron agrupados para evitar que el modelo entrenara en una muestra y evaluara en su

duplicado.

Cabe destacar que cada muestra es usada 2 veces. Por un lado aparecerá como objetivo del prompt más elaborado (question) y por otro como objetivo del prompt simplificado (simplified). Esto no es un problema de fuga, ya que el modelo no ve ambos prompts durante el entrenamiento.

Tabla 3.1: Resumen cuantitativo del dataset canónico utilizado en este capítulo.

Magnitud	Valor
Muestras totales	283
Documentos Kubernetes totales	338
Pares prompt-manifest	566
Muestras de entrenamiento	213
Muestras de validación	35
Muestras de test	35
Muestras con YAML duplicado (agrupadas)	51
Pares de prompt duplicado exacto	51
Pares de prompt casi duplicado (similitud > 0.97)	15

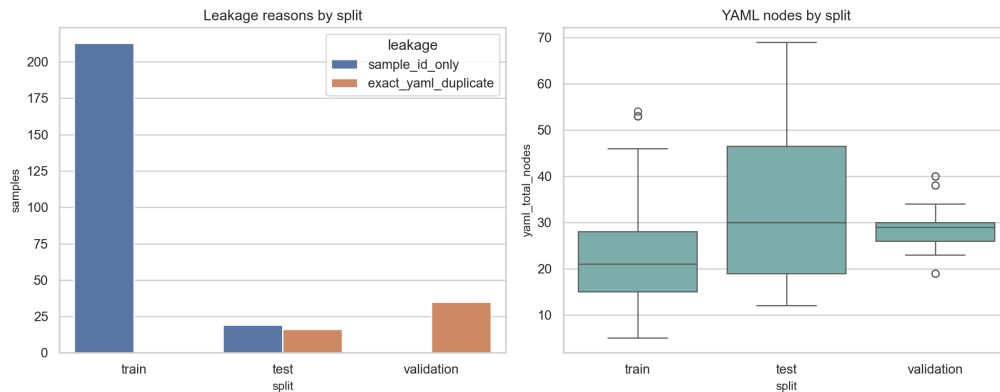


Figura 3.1: Balance del dataset por split y control de fugas entre entrenamiento, validación y test.

3.1.2 Distribución de tipos de recurso

Los manifiestos en el dataset corresponden a varios tipos de recursos Kubernetes (*kinds*). La distribución no es uniforme. Los ocho tipos más frecuentes: (Pod (56), DaemonSet (55), Service (31), Deployment (21), Job (20), PersistentVolume (16), Secret (14), StatefulSet (14)) representan aproximadamente el 64 % del total de documentos. El resto distribuye entre tipos menos comunes: ConfigMap (12), ReplicaSet (9), Namespace (5), CronJob (4) y otros. Esta concentración tiene implicaciones para el aprendizaje del modelo, no solo en términos de frecuencia sino de complejidad estructural, que veremos luego.

La distribución de versiones de API refleja el ecosistema real de Kubernetes: v1 (núcleo del API, 48 %), apps/v1 (workloads, 30 %), batch/v1 (tareas programadas, 8 %), rbac.authorization.k8s.io/v1

(control de acceso, 6%) y otras versiones. Por último, respecto a la estructura de los manifiestos, casi todas las muestras (238 de 283) contienen un único recurso por manifiesto. Solo 36 muestras contienen dos recursos y 8 contienen tres. La mayoría de los problemas jerárquicos que encontramos están confinados a la estructura dentro de un único recurso, no en la orquestación de múltiples documentos.

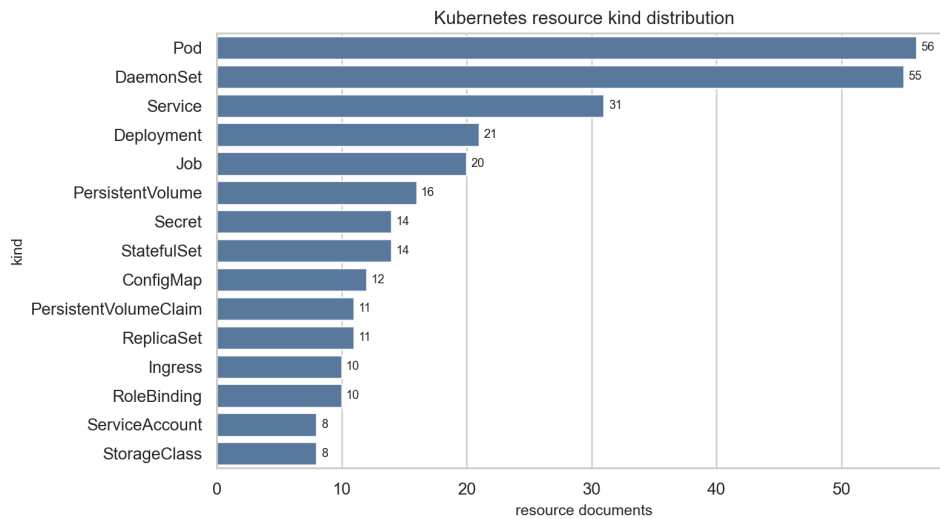


Figura 3.2: Distribución de *kinds* en el corpus Kubernetes.

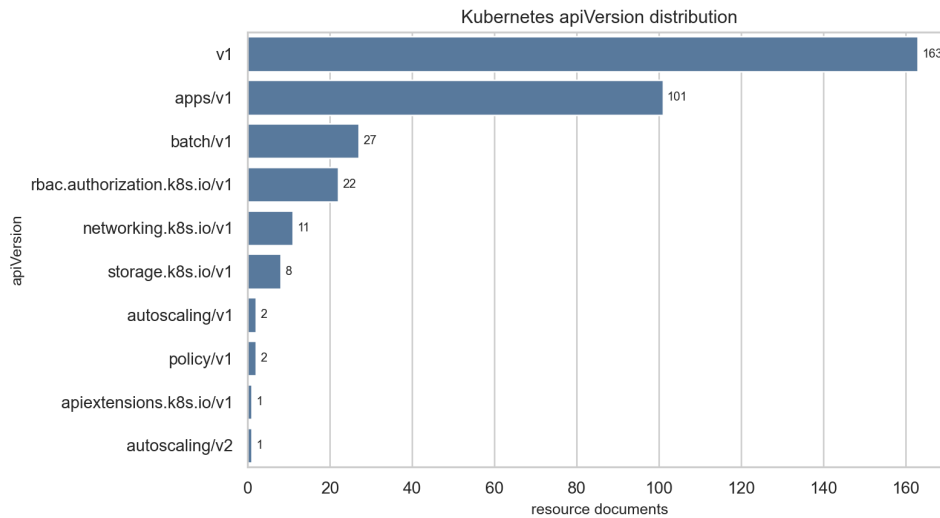


Figura 3.3: Distribución de versiones de API en los manifiestos del dataset.

3.2 Representación jerárquica: la secuencia nivel-línea

3.2.1 El contrato de estructura

En este capítulo usamos la representación secuencia-nivel-línea como lente descriptiva para estudiar distribución de complejidad y patrones de dificultad. La especificación formal del contrato de datos, la definición operacional de `document_index/line_index/level` y el pipeline de transformación de YAML canónico a formato tabular se desarrolla de forma completa en el capítulo 4, sección 4.2. Aquí nos centramos en la evidencia empírica que motiva ese diseño metodológico.

3.2.2 Estadísticas de profundidad y complejidad

Las muestras del dataset presentan variaciones significativas en tamaño y profundidad. El número de líneas por manifest (`block_count`) varía desde un mínimo de 4 líneas hasta un máximo de 59, con mediana de 20 líneas. Esto refleja la heterogeneidad de los manifests: algunos son objetos de configuración simples (como un `ConfigMap` con unos pocos pares clave-valor), mientras que otros son deployments complejos con especificaciones de plantillas, volúmenes y selectores.

La profundidad máxima de indentación por muestra (`max_block_level`) también varía considerablemente, con mínimo de 3, percentil 75 en 9 y máximo de 13. Esto significa que algunos manifests tienen nesting profundo (caso típico: `Deployment` con `spec` → `template` → `spec` → `containers`), mientras que otros permanecen en niveles superficiales.

Pero el rasgo más importante es la *distribución de niveles en el conjunto completo de bloques*. Si N_b denota el número total de bloques y ℓ_i el nivel del bloque i , la proporción de bloques en cada nivel k puede definirse como:

$$p_k = \frac{1}{N_b} \sum_{i=1}^{N_b} \mathbf{1}[\ell_i = k].$$

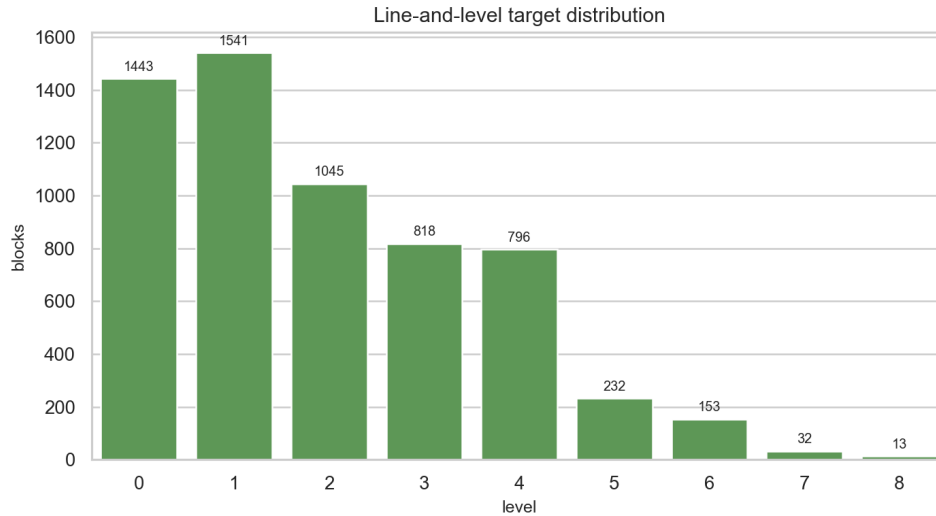
Esta definición permite separar la intuición visual de la figura de una magnitud concreta: cuando contamos cuántos bloques del dataset completo caen en cada nivel de indentación, emerge un patrón claro:

- Nivel 0: 1.443 bloques (primeras claves del manifest: `apiVersion`, `kind`, `metadata`, `spec`)
- Nivel 1: 1.541 bloques (contenido de `metadata` y `spec`)
- Nivel 2: 1.045 bloques (subestructuras como `labels`, `selector`, `containers`)
- Nivel 3: 818 bloques
- Nivel 4: 796 bloques
- Nivel 5: 232 bloques (7.8 % del total)
- Nivel 6: 153 bloques (4.5 %)

- Niveles 7 - 8: 45 bloques combinados (1.5 %)

Tabla 3.2: Distribución de bloques por nivel de indentación.

Nivel	Bloques	Porcentaje
0	1.443	23.8 %
1	1.541	25.4 %
2	1.045	17.2 %
3	818	13.5 %
4	796	13.1 %
5	232	3.8 %
6	153	2.5 %
7 - 8	45	0.7 %


Figura 3.4: Distribución de bloques por nivel de indentación en el conjunto canónico.

Existe un *desbalanceo severo* hacia niveles bajos. A partir del nivel 5, la cantidad de ejemplos cae de forma brusca. Este hecho tiene implicaciones directas para el aprendizaje: un modelo que intenta predecir `level` como un número dentro de una secuencia tendrá muy pocos ejemplos de niveles profundos y, por tanto, poco incentivo para aprender a predecirlos correctamente.

Precisamente en este punto se introdujo un experimento diagnóstico previo al entrenamiento estructural. Su objetivo fue verificar una pregunta metodológica más básica: si los estados ocultos del modelo base ya contienen señal recuperable sobre la variable `level`, siguiendo la intuición de los trabajos de *structural probing* sobre representaciones internas de modelos de lenguaje [18, 9]. En este sentido, el experimento actúa como una comprobación de factibilidad para la hipótesis estructural del trabajo.

Formalmente, el probe puede verse como un clasificador ligero que intenta reconstruir el nivel jerárquico a partir de un estado oculto del modelo:

$$\hat{\ell}_i = g_\phi(h_i),$$

donde h_i es la representación latente asociada a la línea i , g_ϕ es el clasificador entrenado sobre esas representaciones, y $\hat{\ell}_i$ es el nivel estimado. Esta formulación es deliberadamente limitada: no demuestra que la arquitectura final vaya a generar mejor YAML, pero sí permite medir si existe una señal estructural aprovechable antes de introducir un cabezal explícito.

El diseño del experimento se planteó de forma conservadora para evitar conclusiones infladas. Primero, la etiqueta `level` se eliminó de la superficie de entrada del modelo durante la extracción de representaciones, de modo que el clasificador no pudiera “leer” la respuesta directamente. Segundo, la evaluación se realizó sobre *train* y *validation*, reservando *test* para fases posteriores del proyecto. Tercero, se contrastaron probes aprendidos frente a baselines simples (clase mayoritaria y nivel previo), precisamente para distinguir señal latente real de regularidades triviales del corpus.

Bajo ese marco, los resultados más sólidos muestran un patrón coherente con el análisis de distribución de niveles: la recuperación de `level` es alta en zonas superficiales (niveles 0 - 2), se degrada en niveles intermedios (3 - 4) y cae de forma marcada en nivel 5, con mayor inestabilidad en niveles superiores por escasez de muestras. En la configuración lineal seleccionada para la figura 3.6, el error absoluto medio fue de 0.436. A primera vista, esta evidencia no permite afirmar que el problema estructural esté resuelto, pero sí respalda que existe una señal jerárquica latente sobre la que puede apoyarse una supervisión explícita de nivel en etapas posteriores.

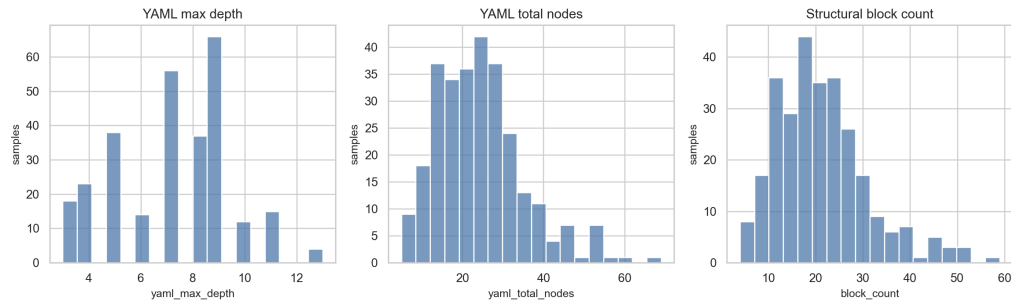


Figura 3.5: Distribuciones de complejidad estructural en el dataset canónico.

3.2.3 Heterogeneidad estructural por tipo de recurso

Diferentes tipos de Kubernetes tienen “firmas” estructurales muy distintas. Un `ConfigMap` es una estructura plana: contiene solo claves y valores al nivel 1 bajo el campo `data`, lo que produce nesting de profundidad 3 - 4. Un `Secret` es similar. Por el contrario, un `Deployment` o `StatefulSet` introduce templating: el campo `spec` contiene un `template`, que a su vez contiene otro `spec` para los pods, que contiene `containers`, que contiene `volumeMounts`. Esto produce nesting de profundidad 8 - 10 o más.

Esto significa que la dificultad estructural no es uniforme: cuando el modelo ve un `Pod`, tiene muchas más muestras de nesting profundo que cuando ve un `ConfigMap`. En este sentido,

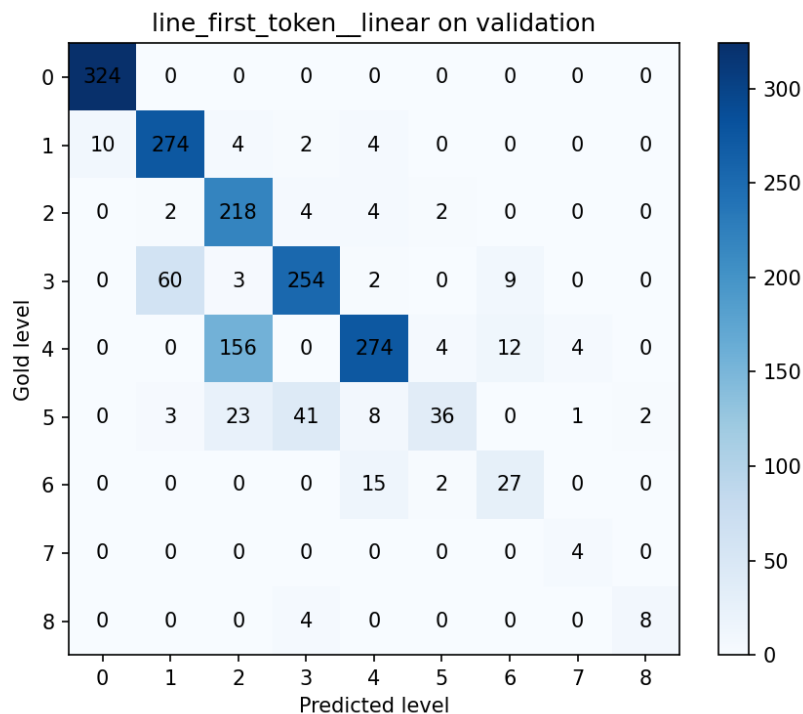


Figura 3.6: Matriz de confusión del *latent probe* para predicción de nivel de indentación.

el sesgo de frecuencia en el dataset (Pod y DaemonSet dominan con 33 % del total) está correlacionado con una mayor complejidad estructural. Esto es un matiz importante: no es que el modelo tenga sesgo hacia tipos simples (que sería un problema de falta de cobertura), sino que ha visto muchos ejemplos de estructuras complejas.

3.2.4 Presencia de campos semánticos

Algunos campos Kubernetes son obligatorios en todos los manifests (como `metadata`), mientras que otros son opcionales y aparecen solo en ciertos tipos de recursos. Analizando el dataset:

- `metadata`: presente en el 100 % de los manifests (requerida por la API)
- `spec`: presente en el 86.2 % (casi todos excepto Namespace y algunos objetos triviales)
- `containers`: presente en el 64.7 % (Pods, Deployments, y otros workloads)
- `image`: presente en el 64.7 % (corolario de containers)
- `ports`: presente en el 21.2 % (Services, Ingresses, algunos Pods)
- `volumes`: presente en el 13.4 % (solo algunos workloads)
- `env`: presente en el 7.77 % (algunas aplicaciones con configuración)

Esta distribución refleja la naturaleza del dominio: la mayoría de manifests son workloads (Pods, Deployments, etc.) que necesitan `spec` y `containers`, pero una minoría significativa son resources de configuración o red con campos mucho más específicos. La implicación para

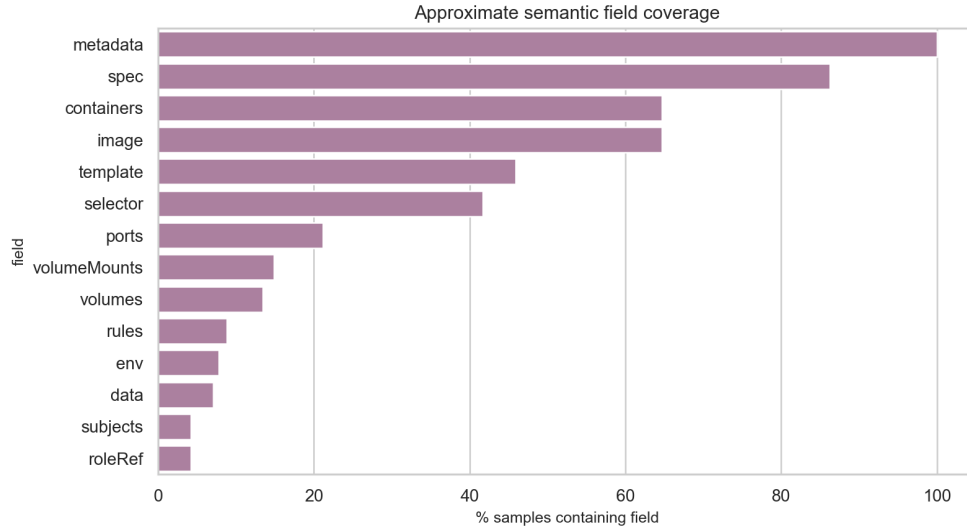


Figura 3.7: Cobertura de campos semánticos relevantes en manifiestos Kubernetes del dataset.

el modelo es que debe aprender patrones estructurales de alta varianza: a veces `spec` contiene `containers`, a veces contiene replicas, a veces contiene `selector`.

3.3 Análisis de los prompts

3.3.1 Variantes y longitud

Como ya hemos visto, cada muestra tiene asociados dos prompts: una versión más descriptiva (*question*) y una versión simplificada (*simplified*). El objetivo es capturar la robustez del sistema ante diferentes formas de expresar la misma especificación. La versión *question* tiene una longitud mediana de 54 palabras, con máximo de 176. La versión *simplified* tiene mediana de 36 palabras, con máximo de 139. El rango es amplio en ambos casos, reflejando que algunos manifests requieren especificación detallada (por ejemplo, un Deployment con múltiples contenedores y restricciones de scheduling) mientras que otros son triviales.

En cuanto a tokens del tokenizador del modelo base (Qwen 2.5 - 7B), los prompts *question* tienen longitud mediana de aproximadamente 45 tokens, mientras que *simplified* tiene mediana de 27 tokens. La diferencia entre conteo de palabras y conteo de tokens es relevante porque ciertos identificadores de Kubernetes (como nombres en camelCase o paths) se tokenizan de forma subóptima, fragmentándose en múltiples tokens. Esto aumenta la longitud efectiva del prompt para el modelo respecto a lo que el conteo de palabras sugeriría.

3.3.2 Similitud entre variantes

La similitud semántica entre el prompt *question* y el prompt *simplified* se mide mediante similitud coseno entre embeddings de frase. Si $\mathbf{e}_q$ y $\mathbf{e}_s$ representan los embeddings de las variantes *question* y *simplified*, respectivamente, la medida utilizada es:

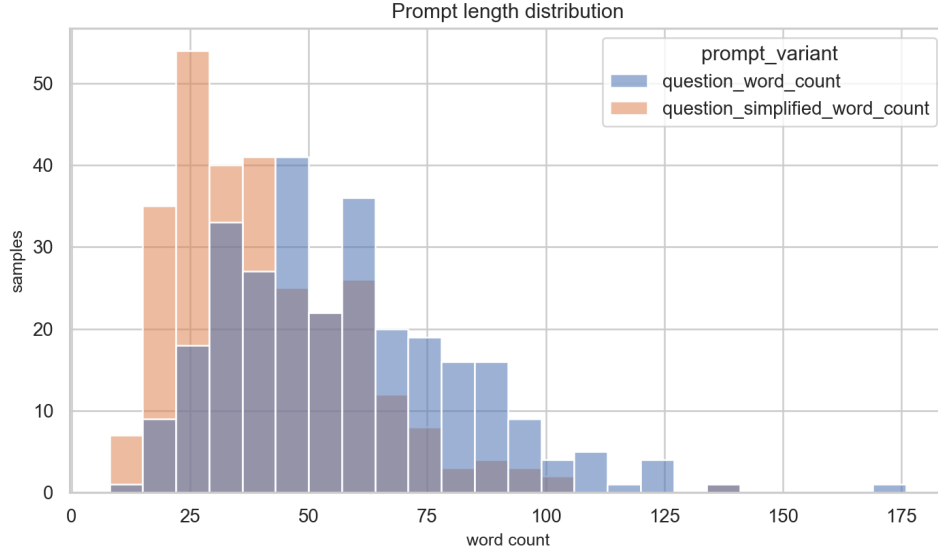


Figura 3.8: Distribución de longitud de prompt por variante (*question* y *simplified*).

$$\text{sim}(q, s) = \frac{\mathbf{e}_q^\top \mathbf{e}_s}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_s\|_2}.$$

En el dataset, esta similitud varía sustancialmente: el rango va de 0.02 a 1.0, con mediana de 0.68. Esto indica que para algunas muestras, ambas variantes son casi idénticas (casos donde simplificar no aporta casi nada), mientras que para otras, la versión simplificada omite detalles cruciales. Un caso extremo sería un prompt que especifica detalles sobre tolerancias de nodos cuya versión simplificada solo menciona “pod con restricciones”, capturando la esencia pero perdiendo especificidad.

3.3.3 Requisitos capturados

De los prompts se pueden extraer requisitos semánticos mediante patrones de regex: tipo de recurso (Pod, Deployment, etc.), nombres de aplicación, imágenes de contenedor, labels, puertos expuestos, número de replicas, volúmenes y mounts, variables de entorno, namespaces, y restricciones de scheduling (afinidad, tolerancias, etc.). El grado en que estos requisitos aparecen explícitamente en el prompt varía. Algunos prompts especifican casi todos los requisitos; otros proporcionan solo los campos esenciales. Y de estos requisitos mencionados en el prompt, no todos aparecen en el manifest YAML ground truth. Un prompt puede especificar “el contenedor debe ejecutarse como usuario no-root”, pero el manifest del dataset podría no incluir la sección `securityContext`, bien porque fue omitido deliberadamente o porque el contexto del dataset no cubre todos los requisitos mencionados. Esta ambigüedad es un aspecto del problema que veremos reflejado en las métricas de cobertura de requisitos.

De esta extracción de requisitos construiremos métricas de cobertura semántica, que nos ayudarán a evaluar no solo si el manifest es sintácticamente correcto, sino si cumple con los requisitos explícitos del prompt. Por ejemplo, si el prompt menciona un puerto específico,

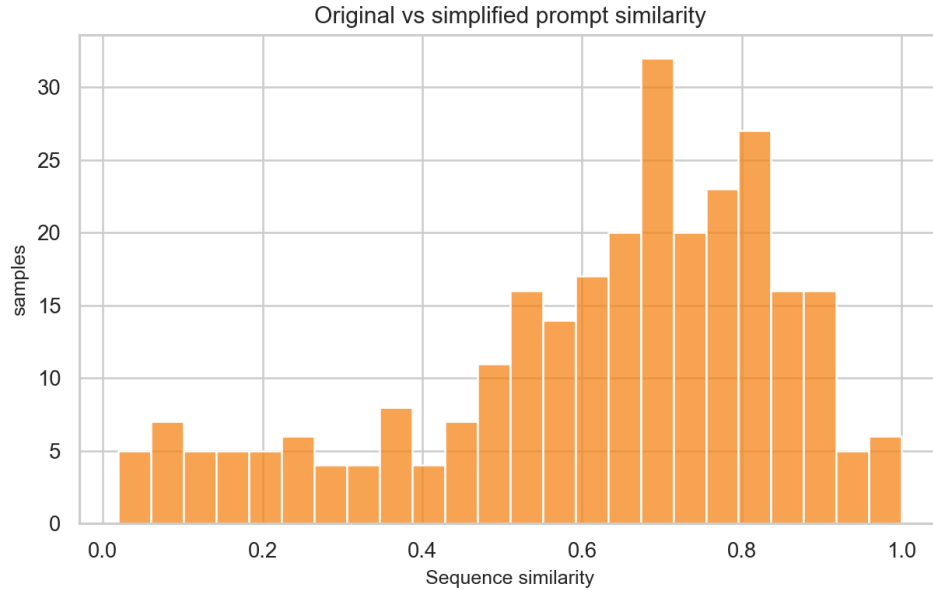


Figura 3.9: Similitud semántica entre variantes de prompt por muestra.

¿está ese puerto presente en el manifest generado? Si el prompt especifica un número de replicas, ¿coincide con el valor en el YAML? Esto nos liberará del uso de métricas como BLEU o ROUGE, que no capturan adecuadamente la corrección semántica en este dominio cuando la salida es un artefacto técnico validable [8, 45].

3.3.4 Correlación entre longitud y complejidad

Un aspecto relevante es si prompts más largos producen manifests más complejos. Existe una correlación positiva moderada entre la longitud del prompt (en palabras o tokens) y la complejidad del manifest, medida como número de líneas (`block_count`) o número de nodos en el árbol parseado (`yaml_total_nodes`). Esto es intuitivo: especificaciones más detalladas tienden a generar manifests más grandes. Pero la correlación no es perfecta: hay prompts relativamente breves que generan manifests complejos (por ejemplo, “crea un Deployment” en un contexto donde Deployment implica naturalmente múltiples contenedores y volúmenes) y prompts largos con manifests simples (por ejemplo, explicaciones verbosas de un ConfigMap simple).

3.4 Observaciones con implicaciones de diseño

El análisis anterior revela patrones del dataset que tienen consecuencias directas para cómo el modelo aprenderá a generar manifests. En esta sección contrastamos estas observaciones con los resultados de los experimentos realizados hasta ahora.

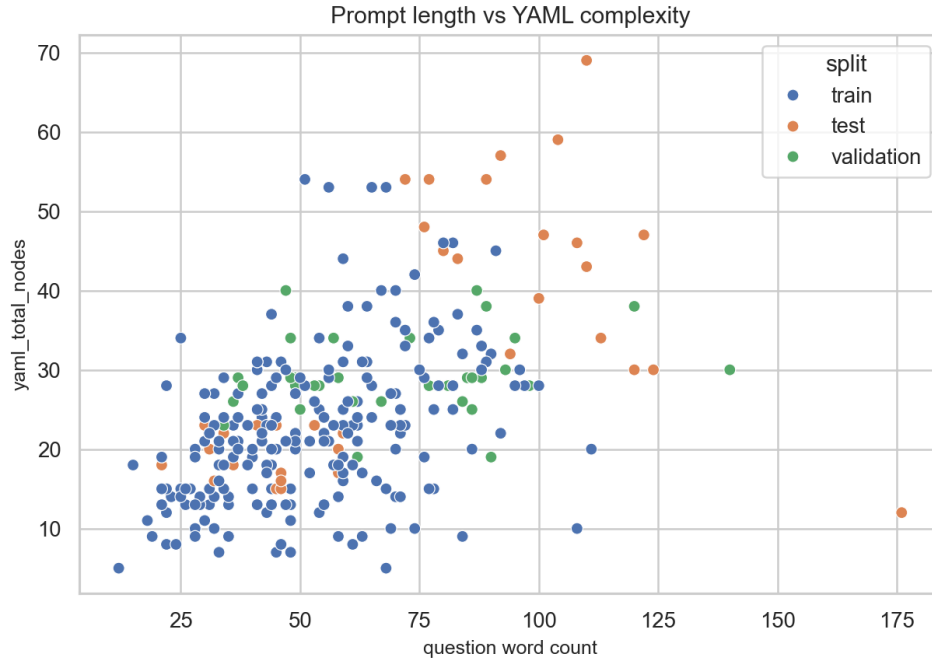


Figura 3.10: Relación entre longitud de prompt y complejidad estructural del YAML.

3.4.1 La pared del nivel 5: desbalanceo de profundidad

El dataset tiene $<8\%$ de bloques en niveles de indentación ≥ 5 . En términos de la distribución anterior, esta región profunda se resume como:

$$p_{\geq 5} = \sum_{k=5}^8 p_k.$$

Esta escasez no es una simple incomodidad estadística: afecta a la capacidad del modelo de aprender a predecir niveles profundos. El experimento anterior confirma que la representación latente del modelo base codifica información de profundidad jerárquica, pero la fidelidad cae en picada a partir del nivel 5. Concretamente, mientras que la capacidad de predecir si una línea está al nivel 0 - 4 es consistente ($\text{recall} > 0.61$), el modelo solo logra recall de 0.32 en el nivel 5, una caída de casi el 50%.

3.4.2 Sintaxis vs. estructura: la brecha del 86 %

Un YAML puede ser sintácticamente válido pero estructuralmente incorrecto. Entendemos por “estructuralmente incorrecto” un manifest que parsea correctamente pero cuya forma canónica (después de parsing y reserialización) difiere del target. Esto puede ocurrir por reordenamiento de claves, por nesting incorrecto, o por omisión/adición de campos.

El modelo fine-tuneado (SFT) alcanza una tasa de éxito de parsing del 98.57% pero una tasa de igualdad estructural (*parsed equal rate*) de apenas 11.43%. La brecha de 86 puntos

porcentuales es significativa: nos viene a decir que la gran mayoría de los manifests que el modelo genera son válidos según el parser YAML, pero no coinciden exactamente con el target canónico. El modelo base sin fine-tuning amplifica este problema: logra parsing en solo 30.77 % de los casos y coincidencia estructural en 3.08 %.

Esta observación estructura el sistema de evaluación en capas: primero, validez de parseo (¿es YAML válido?); segundo, equivalencia estructural (¿es el mismo manifest?); tercero, corrección semántica (¿cumple con los requisitos y las buenas prácticas de seguridad?). Solo la primera es necesaria pero no suficiente.

3.4.3 Anti-patrones de seguridad presentes en el training data

Los manifests del dataset no son perfectos desde el punto de vista de seguridad. El análisis realizado detecta varios anti-patrones en las salidas generadas, muchos de los cuales están presentes también en los targets del training data:

- Ausencia de `resources.requests` y `resources.limits`: 261 ocurrencias en 70 muestras (3.7 por muestra en promedio)
- Ausencia de `runAsNonRoot` en `securityContext`: 71 ocurrencias
- Ausencia de `readOnlyRootFilesystem`: 71 ocurrencias
- Uso de tag `latest` de imagen o ausencia de tag específico: 67 ocurrencias
- Otras issues menores: volúmenes sin mounts correspondientes, etc.

El evaluador de dominio de Kubernetes reporta que solo el 14.29% de las muestras pasan todos los niveles de validación de buenas prácticas. Sin embargo, este número oculta un dato importante: el modelo *sí* genera configuraciones completamente conformes en algunas muestras (10 de 70), lo que sugiere que posee la capacidad estructural de producir código seguro pero no está sesgado hacia ello. El modelo aprende de los datos y replica los anti-patrones que encuentra, un riesgo coherente con la literatura sobre errores y smells de seguridad en manifests Kubernetes [34, 48]. Esto tiene implicación directa para fases posteriores: una señal explícita de preferencia o recompensa sobre seguridad, por ejemplo mediante optimización directa de preferencias, puede redirigir el comportamiento sin necesidad de reentrenamiento desde cero [33].

Capítulo 4

Diseño Experimental y Métricas de Evaluación

Este capítulo formaliza el diseño experimental que guía todo el trabajo. El objetivo del capítulo es, definir con precisión el contrato de datos, el flujo de representación y el sistema de métricas con el que se evalúan las dos formulaciones supervisadas. En otras palabras, aquí se fija qué significa que una predicción sea válida, estructuralmente fiel y útil en términos de dominio Kubernetes.

Además, el capítulo hace explícita la relación entre decisiones de representación y señales de evaluación. Esta conexión es importante porque una salida puede parecer correcta en superficie textual y, sin embargo, fallar al reconstruir un YAML válido o incumplir requisitos de dominio. Precisamente por eso se adopta una evaluación multinivel y no una única métrica agregada, en línea con trabajos recientes que muestran las limitaciones de evaluar salidas estructuradas únicamente desde su parecido superficial [46, 45].

4.1 Planteamiento experimental y comparabilidad

La pregunta experimental central es: hasta qué punto una formulación puramente secuencial puede capturar jerarquía, y qué impacto tiene sobre la calidad estructural supervisar esa jerarquía de forma explícita en lugar de aprenderla como texto serializado, una tensión cercana a la formulación clásica de la predicción estructurada [22, 4]. Para responder de forma controlada, ambas ramas comparten modelo base, conjunto de datos canónico, parser determinista y protocolo de evaluación.

De este modo, cualquier diferencia observada en resultados puede atribuirse a la formulación de salida y a la señal de entrenamiento estructural, no a cambios en datos o infraestructura. Esta decisión metodológica evita comparaciones injustas y facilita interpretar causalmente el efecto de cada elección de representación.

4.1.1 Formulaciones supervisadas

El estudio compara dos formulaciones:

Tabla 4.1: Comparativa de formulaciones supervisadas utilizadas en el estudio.

Aspecto	Serialized SFT	Two-Head SFT
Superficie objetivo	Secuencia TSV con <code>document_index</code> , <code>line_index</code> , <code>level</code> , <code>line_text</code> .	Secuencia TSV con contenido y metadatos de posición; <code>level</code> fuera de la superficie textual.
Señal de jerarquía	Implícita en la propia cadena generada token a token.	Explícita mediante cabezal estructural dedicado para predecir <code>level</code> .
Parser posterior	Reconstrucción determinista a YAML, sin reparación heurística.	Reconstrucción determinista a YAML, sin reparación heurística.
Objetivo de la comparación	Medir hasta dónde llega la codificación superficial de estructura.	Medir ganancia al desacoplar contenido textual y jerarquía.

4.2 Formato de datos y representación habitual

El formato de las muestras usado en este TFM parte de un principio simple: la unidad de aprendizaje no es el árbol YAML completo, sino una secuencia de bloques lineales que conserva información suficiente para reconstruirlo de forma determinista. Cada bloque representa una línea física con metadatos estructurales.

4.2.1 Unidad básica: bloque estructural

Cada manifiesto se representa como una secuencia de tuplas de la forma:

`(document_index, line_index, level, line_text)`

donde `document_index` identifica el documento YAML en escenarios multi-documento, `line_index` fija el orden local de líneas en cada documento, `line_text` contiene el contenido sin sangría inicial y `level` codifica la indentación en unidades de dos espacios.

Una distinción clave es que `level` no equivale necesariamente a profundidad semántica del árbol parseado. Para entenderla, es útil partir de un ejemplo concreto. Considérese el siguiente manifiesto `ConfigMap`, un tipo de recurso frecuente en el dataset de este trabajo, anotado con el `level` que corresponde a cada línea:

```
apiVersion: v1      # level = 0
kind: ConfigMap     # level = 0
metadata:           # level = 0
  name: app-config  # level = 1
```

```
labels:           # level = 1
  app: my-app     # level = 2
```

`level` cuenta unidades de sangría en el texto renderizado: cada dos espacios de indentación suman uno. La línea `app: my-app` tiene `level = 2` porque está desplazada cuatro espacios del margen. Hasta aquí, la lectura es directa. El contraste aparece al parsear el documento como árbol de objetos. La profundidad de un nodo se define como el número de aristas que lo separan de la raíz: la raíz tiene profundidad 0, sus hijos directos tienen profundidad 1, y así sucesivamente. Bajo esa definición, el escalar `my-app` está a tres aristas de la raíz (raíz $\rightarrow$ `metadata` $\rightarrow$ `labels` $\rightarrow$ `my-app`). De modo que `yaml_depth` para este documento vale 3, mientras que el `level` máximo observado en los bloques es 2. Las dos magnitudes suelen correlacionar, pero se calculan desde representaciones distintas: una trabaja sobre el texto renderizado, la otra sobre la estructura del árbol parseado. En la práctica, la variable de `level` se corresponde con `yaml_depth - 1`.

4.2.2 Granularidad del dataset durante los experimentos

Para evitar ambigüedades terminológicas, el dataset se maneja con cuatro niveles de granularidad:

- **Muestra** (`sample`): unidad conceptual asociada a una petición en lenguaje natural y su salida esperada.
- **Documento** (`document`): subunidad YAML dentro de una muestra, indexada por `document_index`.
- **Línea/Bloque** (`line` o `block`): fila estructural con `line_index`, `level` y `line_text`.
- **Nivel de línea** (`line level`): profundidad de indentación de una línea concreta, usada tanto en entrenamiento como en evaluación estructural.

Esta jerarquía de granularidad permite construir métricas locales (por línea), métricas de consistencia (por documento) y métricas de éxito operativo (por muestra). En la práctica, las métricas se van a construir a nivel de muestra o a nivel de línea, dependiendo de la señal que se quiera capturar. Por ejemplo, la tasa de parseo es una métrica de muestra (¿parsea el documento completo?), mientras que la exactitud de nivel es una métrica de línea (¿predice el nivel correcto para esta línea?).

4.2.3 Pipeline de transformación: YAML a CSV/TSV

El flujo de datos habitual durante los experimentos sigue cuatro etapas:

1. Normalización de YAML de referencia para fijar una forma canónica estable.
2. Conversión de YAML canónico a secuencia de bloques estructurados.
3. Serialización de bloques a formato tabular de entrenamiento (principalmente TSV, con exportación CSV/TSV para análisis externo).
4. Reconstrucción determinista de YAML desde bloques para validación *round-trip*.

La elección de una representación tabular se justifica por su versatilidad. Permite entrenar con una interfaz textual homogénea, inspeccionar errores por columna y trazar fallos estructurales de forma reproducible. Además, al mantener `document_index` y `line_index`, la evaluación conserva orden y segmentación sin depender de heurísticas posteriores.

El parser cumple aquí una función de control estructural: no repara la predicción ni inventa campos, sino que delimita qué salidas pueden reconstruirse de forma segura. Esta idea se relaciona con trabajos de generación guiada y parsers incrementales, donde la validez formal de la salida se incorpora como frontera explícita del proceso de generación o validación [36, 17, 42].

Tabla 4.2: Serializaciones operativas usadas en el proyecto.

Formato	Columnas visibles	Uso principal
<code>blocks_tsv_v1</code>	<code>document_index</code> , <code>line_index</code> , <code>level</code> , <code>line_text</code>	Supervisión secuencial estructurada en Serialized SFT.
<code>blocks_tsv_compact_v1</code>	<code>document_index</code> , <code>level</code> , <code>line_text</code>	Variante compacta para análisis y baselines controlados.
<code>content_blocks_v1</code>	<code>document_index</code> , <code>line_index</code> , <code>line_text</code>	Superficie textual para Two-Head SFT, con <code>level</code> como etiqueta separada.

Esquema del pipeline (visión conceptual)

YAML canónico → Bloques estructurados → Serialización tabular (TSV/CSV) → Predicción del modelo → Reconstrucción YAML → Evaluación multinivel.

Figura 4.1: Pipeline metodológico desde la representación canónica hasta la evaluación.

4.3 Protocolo de entrenamiento y evaluación

La comparativa experimental se apoya en tres principios: mismo split de datos entre formulaciones, misma base de modelo y mismo mecanismo de reconstrucción y evaluación. El modelo base es Qwen2.5-7B-Instruct [31] en cuantización de 4 bits. La cuantización consiste en representar los pesos del modelo con una precisión reducida (cuatro bits por parámetro frente a los 16 o 32 habituales), lo que reduce de forma importante el consumo de memoria de la GPU sin comprometer en exceso la calidad de las representaciones internas [11]. La adaptación al dominio se realiza mediante LoRA (*Low-Rank Adaptation*), una técnica de ajuste fino que introduce matrices de adaptación de bajo rango (rango 8 en este trabajo) en las proyecciones de atención del modelo. Esto permite modificar su comportamiento sin tocar los pesos originales ni requerir los recursos de un entrenamiento completo [21].

Un matiz relevante para la interpretación de los experimentos es que el presupuesto de tokens de salida difiere entre ramas: la rama *Serialized SFT* opera con un límite de 512 tokens,

mientras que *Two-Head SFT* dispone de 1024. La diferencia refleja la asimetría entre las dos superficies objetivo: la rama serializada incluye `level` como columna explícita en la cadena generada, mientras que la rama Two-Head lo delega al cabezal estructural y genera únicamente contenido y metadatos de posición. Esta asimetría es una decisión de diseño documentada, pero el lector debe tenerla presente al interpretar métricas sensibles a truncación o completitud de salida.

En este marco, la diferencia relevante entre ramas no es el tamaño del modelo ni el conjunto de datos, sino la forma en la que se representa y supervisa la jerarquía. Esa decisión es coherente con la hipótesis central del trabajo: los errores de generación estructurada no se explican solo por semántica de contenido, sino también por cómo la arquitectura aprende la señal de profundidad.

4.4 Métricas de evaluación: familias y utilidad

La evaluación se organiza en familias complementarias. Ninguna métrica individual basta para caracterizar calidad estructural en YAML/Kubernetes. Por eso, el análisis debe combinar la validez sintáctica, fidelidad de estructura, adecuación textual y consistencia de dominio si quiere ser útil como medida cuantitativa.

Tabla 4.3: Familias de métricas y pregunta metodológica que responden.

Familia	Ejemplos	Utilidad metodológica
Sintácticas de parseo	<code>yaml_parse_ok</code> , <code>block_parse_ok</code> , ratio de bloques válidos	Determinar si la salida es procesable sin intervención manual.
Estructurales de jerarquía	<code>level_exact_match_rate</code> , <code>level_mae</code> , secuencia de índices	Medir fidelidad de la geometría de indentación y orden interno.
Fidelidad textual	exactitud de contenido de línea, precisión/recobrado/F1 textual, BLEU/-ROUGE auxiliares	Cuantificar cuánto contenido de referencia se conserva.
Semántica Kubernetes	coincidencia de <code>kind</code> , <code>apiVersion</code> , claves semánticas, selectores/-volúmenes	Verificar corrección funcional básica de recursos generados.
Requisitos de prompt y campos requeridos	precisión/recobrado de requisitos, tasa de campos obligatorios presentes	Evaluar alineación entre petición natural y manifiesto resultante.
Validez de dominio multi-nivel	nivel de validez 0–5, tasa de paso de compuertas de dominio	Integrar señales estructurales y semánticas en un criterio operativo único.

4.4.1 Métricas sintácticas y estructurales

Las métricas de parseo son la primera barrera: sin parseo correcto, no tiene sentido discutir calidad semántica. Sin embargo, parsear no implica necesariamente fidelidad al objetivo. Por eso se añaden métricas estructurales por línea y por documento que detectan desorden de índices, errores de nivel o fuga de indentación.

Este bloque de métricas responde a la pregunta: *¿la salida es internamente coherente como objeto estructurado?* En un problema de generación YAML, esta pregunta es necesaria y no puede reemplazarse por una métrica textual global.

Las principales métricas de este bloque son la tasa de parseo YAML correcto (`yaml_parse_ok`), calculada simplemente como la fracción de muestras cuya salida el parser acepta sin error, y las métricas de fidelidad estructural por línea. De estas últimas, las dos más informativas son la tasa de coincidencia exacta de nivel (`level_exact_match_rate`) y el error absoluto medio de nivel (`level_mae`). Sean $R = (r_1, \dots, r_n)$ los niveles de referencia y $P = (p_1, \dots, p_m)$ los niveles predichos, y sea $k = \min(n, m)$ el número de líneas comparables:

$$\text{level_exact_match_rate} = \frac{1}{n} \sum_{i=1}^k \mathbf{1}[r_i = p_i]$$

$$\text{level_mae} = \frac{1}{k} \sum_{i=1}^k |r_i - p_i|$$

Ambas se calculan solo sobre el prefijo común de longitud k : si la predicción genera menos líneas que la referencia, las líneas no cubiertas no cuentan como coincidencias pero tampoco penalizan directamente el MAE. Esa elección de implementación implica que errores de recuento de líneas se capturan mejor con `line_count_match` que con `level_mae`.

4.4.2 Métricas de contenido y semántica de dominio

Una vez garantizada la estructura mínima, se analiza contenido. En este punto conviene precisar el papel de las métricas de solapamiento textual como BLEU o ROUGE. Estas métricas, ampliamente utilizadas en tareas de generación de lenguaje natural como traducción automática o resumen, miden en qué medida la salida comparte n-gramas con una referencia. En un dominio de texto libre esa señal es razonablemente informativa. En el caso de manifiestos Kubernetes, sin embargo, resulta insuficiente como criterio principal: una salida puede conservar la mayoría de los tokens de referencia y aun así producir un YAML que no parsea, porque una sola línea con la indentación incorrecta rompe la estructura del documento. A la inversa, una salida estructuralmente válida pero que emplea nombres de campo o valores distintos puede tener un ROUGE alto sin ser funcionalmente equivalente. Por eso BLEU y ROUGE se usan aquí en un rol auxiliar, como señal secundaria de solapamiento de contenido, pero no como criterio de corrección estructural ni de validez semántica en el dominio, una distinción coherente con la evaluación funcional de artefactos técnicos y con benchmarks específicos de YAML cloud-native [8, 45].

Dentro de las métricas de fidelidad textual sí intervienen directamente en el análisis la precisión, el recobrado y el F1 de contenido de línea. Sean $\hat{T}$ el multiconjunto de líneas de texto predichas y T el de referencia; los verdaderos positivos son el solapamiento multiconjunto entre ambos, $TP = |\hat{T} \cap T|$:

$$\text{Precision} = \frac{TP}{|\hat{T}|}, \quad \text{Recall} = \frac{TP}{|T|}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

donde $|\cdot|$ denota cardinalidad con multiplicidades. Esta familia es sensible a qué contenido se genera, pero ciega al orden de las líneas dentro del manifiesto, que es precisamente lo que capturan las métricas estructurales del bloque anterior.

Las métricas semánticas de Kubernetes, por su parte, evalúan si la salida conserva campos críticos y relaciones internas relevantes: coherencia entre selectores y etiquetas, correspondencia entre `volumeMounts` y `volumes`, presencia y corrección de `kind` y `apiVersion`. Estas señales apuntan a si el manifiesto generado es funcionalmente plausible, algo que ninguna métrica puramente textual puede capturar.

Aquí la pregunta cambia: *¿además de estar bien formado, el manifiesto representa lo que debe representar?* La combinación de señales textuales y semánticas evita dos falsos positivos habituales: salidas gramaticalmente válidas pero funcionalmente incorrectas, y salidas semánticamente plausibles pero estructuralmente frágiles.

4.4.3 Validez de dominio Kubernetes: escala por niveles

Las métricas anteriores responden a preguntas locales: si una línea tiene el nivel correcto, si el YAML parsea, si cierto campo está presente. Ninguna de ellas, por sí sola, dice si un manifiesto es operativamente aceptable. Para eso el sistema define una escala de validez de dominio estructurada en seis compuertas secuenciales, numeradas del 0 al 5, que se evalúan en orden y se detienen en la primera que falla.

La idea es imitar la cadena de checks que un operador real aplicaría antes de aplicar un manifiesto a un clúster: primero que el YAML sea parseable, luego que la representación interna sea coherente, después que el recurso sea reconocible como Kubernetes, y así progresivamente hasta llegar a requisitos de calidad estática. El nivel que alcanza la predicción antes de fallar (o 5 si supera todas las compuertas) es el `kubernetes_domain_validity_level`.

La escala es acumulativa: superar el nivel ℓ implica haber superado todos los anteriores. Si $c_r(y_i)$ representa que la muestra i supera la compuerta r , la tasa de paso del nivel ℓ sobre un conjunto de evaluación S puede escribirse como:

$$\text{pass}_\ell = \frac{1}{|S|} \sum_{i \in S} \prod_{r=0}^{\ell} \mathbf{1}[c_r(y_i)].$$

Esta definición la convierte en un criterio legible de comparación entre formulaciones, más informativo que una media de métricas heterogéneas. Si un modelo pasa sistemáticamente el

Tabla 4.4: Escala de validez de dominio Kubernetes: compuertas secuenciales.

Nivel	Compuerta	Condición
0	Parseo YAML	El texto generado es YAML válido y puede cargarse sin error de sintaxis.
1	Contrato de bloques	Los bloques del parser satisfacen el contrato estructural completo: índices monótonos, sin fugas de indentación y ratio de bloques válidos igual a 1.
2	Identidad Kubernetes	Cada documento tiene <code>apiVersion</code> , <code>kind</code> reconocido y <code>metadata.name</code> no vacíos, y los campos obligatorios del tipo de recurso están presentes.
3	Invariantes intra-recurso	Coherencia dentro de cada recurso: selectores con sus etiquetas de template, imágenes de contenedor presentes, puertos en rango válido, <code>volumeMounts</code> con su <code>volume</code> correspondiente.
4	Invariantes inter-recurso	Referencias entre recursos: selector de <code>Service</code> apunta a etiquetas de un workload local, <code>RoleBinding</code> apunta a un <code>Role</code> existente, <code>ServiceAccount</code> referenciada existe en el manifiesto.
5	Calidad estática	Ausencia de olores de seguridad: sin imagen <code>latest</code> , <code>runAsNonRoot: true</code> , <code>readOnlyRootFilesystem: true</code> , recursos de CPU y memoria declarados, sin namespaces de host habilitados.

nivel 2 pero no el 3, el diagnóstico es claro: genera recursos identificables pero con incoherencias internas. Si se queda en el nivel 0, el problema es más básico: no produce YAML parseable.

La compuerta final, el nivel 5, recoge requisitos de calidad que no son estrictamente obligatorios para que Kubernetes acepte el manifiesto, pero que sí son buenas prácticas habituales en entornos de producción. Precisamente por eso es la más difícil de superar y la que actúa como criterio de excelencia operativa, especialmente porque las configuraciones Kubernetes pueden ser sintácticamente válidas y aun así introducir riesgos de seguridad o mantenimiento [34, 48]. La tasa de muestras que alcanzan el nivel 5 se registra como `kubernetes_domain_gate_pass_rate`, y será uno de los indicadores principales en el capítulo de resultados.

4.4.4 Métricas de cumplimiento de prompt y agregación

El sistema también evalúa la alineación con requisitos extraídos del prompt. La idea es que una predicción puede estar bien formada semánticamente y aun así ignorar restricciones concretas del usuario: el tipo de recurso pedido, el nombre, la imagen del contenedor o el número de réplicas. Para capturar esa brecha, el sistema extrae automáticamente un conjunto de requisitos Q desde el texto del prompt y un conjunto de campos cubiertos C desde el YAML predicho. La precisión y el recobrado de requisitos se definen entonces como:

$$\text{Req-Precision} = \frac{|Q \cap C|}{|C|}, \quad \text{Req-Recall} = \frac{|Q \cap C|}{|Q|}$$

Cuando $|Q| = 0$ —es decir, el prompt no contiene requisitos extraíbles— la métrica se marca como no aplicable y no entra en los promedios del dataset. Cuando $|Q| = |C|$ y el conjunto coincide exactamente, la muestra cuenta como `prompt_requirement_exact_match`.

Finalmente, todas las métricas se agregan a nivel de dataset para obtener tasas medias, varianzas y criterios de comparación entre formulaciones. Sea S el conjunto de muestras evaluadas; para cualquier métrica binaria $x_i \in \{0, 1\}$ la tasa agregada es simplemente $\bar{x} = \frac{1}{|S|} \sum_{i \in S} x_i$, y para métricas continuas como F1 o MAE se calcula la media aritmética excluyendo muestras no aplicables. Esta agregación permite pasar de diagnósticos locales a conclusiones experimentales robustas, que se desarrollarán en el capítulo de resultados.

4.5 Criterios de trazabilidad hacia resultados

El resultado de este capítulo es un marco de evaluación reproducible: contrato de datos explícito, pipeline de representación verificable y conjunto de métricas con interpretación clara. Con este marco, el capítulo siguiente puede centrarse en responder la pregunta comparativa sin reabrir definiciones metodológicas.

En concreto, la discusión de resultados se apoyará en tres ejes: robustez estructural, fidelidad semántica y cumplimiento de requisitos de prompt. Mantener estos ejes separados facilita identificar dónde mejora cada formulación y dónde persisten limitaciones.

Capítulo 5

Resultados I: Línea base y ajuste supervisado serializado

El capítulo 4 fijó el protocolo que guía la evaluación: representación canónica en bloques estructurados, pipeline determinista de reconstrucción y un sistema de métricas multinivel que diferencia validez sintáctica, fidelidad estructural, cobertura semántica y conformidad con el dominio Kubernetes. Este capítulo aplica ese protocolo a las dos primeras etapas de la comparativa: la línea base zero-shot y el ajuste fino supervisado con serialización de jerarquía en superficie (`serialized_sft`). El split de test queda reservado para la evaluación conjunta de todas las arquitecturas al final del trabajo. Todos los resultados presentados aquí corresponden al split de validación.

Conviene señalar una asimetría de representación antes de interpretar las cifras. La línea base genera su salida en formato `blocks_tsv_compact_v1`, que omite la columna `line_index` y la recupera implícitamente del orden de las filas. El SFT serializado genera en `blocks_tsv_v1`, que incluye `line_index` de forma explícita. Ambas variantes pasan por el mismo parser determinista y el mismo protocolo de evaluación, por lo que las métricas sobre el YAML reconstruido son comparables entre sí. La diferencia de superficie es relevante para entender el diseño de cada etapa, pero no distorsiona la comparación de calidad.

5.1 Línea base zero-shot

Para entender qué aportan las etapas supervisadas, es útil empezar por una pregunta más sencilla: ¿qué puede hacer el modelo base sin ningún ajuste, con una instrucción en lenguaje natural y un formato de salida explícito en el prompt? Esta configuración se apoya en la capacidad de los modelos de instrucción para seguir especificaciones dadas en lenguaje natural [41, 27]. La respuesta no es trivial a priori. Qwen2.5-7B-Instruct [31] es un modelo de instrucción de propósito general que ha procesado cantidades significativas de código y configuración YAML durante su preentrenamiento, y el prompt del baseline describe el formato de bloque esperado con ejemplos concretos. La pregunta relevante no es si el modelo entiende Kubernetes en algún grado, que probablemente sí, sino si puede mantener la consistencia

jerárquica que exige la reconstrucción determinista del parser para producir YAML válido de principio a fin.

La Tabla 5.1 recoge las métricas principales del baseline en el split de validación.

Tabla 5.1: Métricas del baseline zero-shot en el split de validación (70 muestras totales; 65 con salida estructurada parseable).

Métrica	Valor
Muestras evaluadas	65 / 70
<code>structured_output_parse_success_rate</code>	92,86 %
<code>yaml_parse_success_rate</code>	30,77 %
<code>average_level_mae</code>	0,7896
<code>average_level_exact_match_rate</code>	11,31 %
<code>average_line_text_f1</code>	20,86 %
<code>average_semantic_key_f1</code>	27,78 %
<code>average_prompt_requirement_f1</code>	22,13 %
<code>required_field_complete_sample_rate</code>	70,59 %

La primera observación que surge es una aparente paradoja: la tasa de parseo estructurado asciende al 92,86 %, lo que indica que el modelo sigue el formato de bloque en casi todos los casos. Sin embargo, solo el 30,77 % de las muestras produce un YAML válido tras la reconstrucción con el parser. La Figura 5.1 desglosa ese resultado en tres categorías sobre el total de 70 muestras.

Esta diferencia puede resumirse mediante una brecha de parseabilidad entre la superficie estructurada y el YAML reconstruido:

$$g_{\text{parse}} = \text{structured_output_parse_success_rate} - \text{yaml_parse_success_rate}.$$

En el baseline, esta brecha alcanza $0,9286 - 0,3077 = 0,6209$, es decir, algo más de 62 puntos porcentuales. El valor refleja que el cuello de botella no está en producir una tabla reconocible, sino en que los niveles generados sean compatibles con una reconstrucción YAML válida.

La paradoja se disuelve cuando se examina qué falla exactamente. El modelo produce un output estructurado reconocible, es decir, el bloque TSV se puede parsear. Sin embargo, la asignación de la columna `level` es inconsistente a medida que avanza la secuencia generada. En particular, los desajustes acumulativos en la indentación a lo largo de varias decenas de tokens introducen errores que el parser no puede recuperar: el resultado es un `ScannerError` o `ParserError` en el momento de reconstruir el árbol YAML. El `average_level_mae` de 0,79 lo cuantifica: en promedio, el nivel predicho se aleja casi una unidad completa de sangría del nivel correcto. Para una estructura YAML donde cada unidad representa dos espacios de indentación, ese margen es suficiente para romper la coherencia del árbol.

Las métricas semánticas confirman el mismo límite desde otra perspectiva. Un `semantic_key_f1` del 27,78 % indica que poco más de una cuarta parte de las claves semánticas esperadas

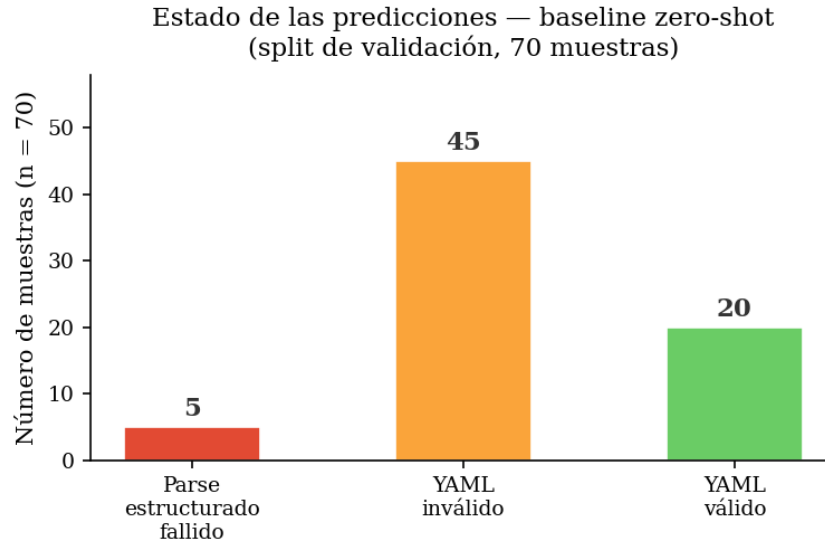


Figura 5.1: Estado de las predicciones del baseline zero-shot en el split de validación. De las 70 muestras, 65 producen una salida estructurada parseable, pero solo 20 dan lugar a un YAML válido tras la reconstrucción determinista. El cuello de botella no es el formato de bloque sino la consistencia jerárquica dentro de él.

aparecen en la predicción. El `average_prompt_requirement_f1` del 22,13 % añade que los requisitos explícitos del prompt (el tipo de recurso solicitado, el nombre, el número de réplicas, el namespace, etc.) se satisfacen en apenas una de cada cinco muestras. Y solo el 70,59 % de las muestras produce recursos con todos los campos obligatorios presentes. En conjunto, el baseline refleja que el modelo reconoce el dominio a nivel de superficie, y produce un output con aspecto de manifiesto. Sin embargo, carece tanto del control jerárquico como de la cobertura semántica que necesita el parser para que ese manifiesto sea estructuralmente válido y funcionalmente completo. Ese es el suelo empírico a partir del cual operan las etapas supervisadas.

5.2 `serialized_sft`: ajuste con jerarquía en superficie

El `serialized_sft` parte de la hipótesis de que el problema del baseline admite una solución mediante ajuste fino supervisado con LoRA [21, 11, 43]: si el modelo aprende a generar secuencias de bloques bien formadas en el formato `blocks_tsv_v1`, el parser determinista debería poder reconstruir YAML válido en casi todos los casos. La jerarquía no se predice mediante ningún mecanismo estructural separado; se serializa en la superficie textual como una columna más del TSV. La pregunta de fondo es si esa codificación implícita es suficiente para resolver el problema de consistencia de niveles que el zero-shot no puede superar. En la arquitectura del trabajo, esta variante actúa como modelo de control: su función es establecer hasta dónde llega la serialización de jerarquía en superficie antes de comparar con el `two_head_sft`, donde la predicción de `level` se desacopla de la generación de contenido de línea mediante una cabeza estructural dedicada.

La Tabla 5.2 presenta la comparativa completa entre el baseline y el SFT serializado en el split de validación. Para cada métrica m , la diferencia se calcula como:

$$\Delta_m = m(\text{serialized_sft}) - m(\text{baseline}).$$

En las métricas donde valores mayores son mejores, un Δ_m positivo indica mejora; en el caso de `average_level_mae`, la mejora aparece como una diferencia negativa porque se trata de un error.

Tabla 5.2: Comparativa de métricas entre el baseline zero-shot y `serialized_sft` en el split de validación. Δ indica la diferencia en puntos porcentuales (métricas de tasa) o en unidades absolutas (`level_mae`).

Métrica	Baseline	<code>serialized_sft</code>	Δ
Muestras evaluadas	65 / 70	70 / 70	—
<code>structured_output_parse_success_rate</code>	92,86 %	100,00 %	+7,14
<code>yaml_parse_success_rate</code>	30,77 %	98,57 %	+67,80
<code>average_level_mae</code>	0,7896	0,2723	−0,517
<code>average_level_exact_match_rate</code>	11,31 %	75,78 %	+64,47
<code>average_line_text_f1</code>	20,86 %	82,06 %	+61,20
<code>average_semantic_key_f1</code>	27,78 %	95,52 %	+67,74
<code>average_prompt_requirement_f1</code>	22,13 %	85,31 %	+63,18
<code>required_field_complete_sample_rate</code>	70,59 %	100,00 %	+29,41

El salto es consistente en todas las métricas: ninguna empeora y varias mejoran más de 60 puntos porcentuales. La tasa de parseo YAML pasa de 30,77 % a 98,57 %: el problema de consistencia estructural que dominaba el baseline queda resuelto en casi todos los casos. El `average_level_mae` cae de 0,79 a 0,27, lo que implica que el error promedio de indentación por línea se reduce a poco más de un cuarto de unidad. Las métricas semánticas reflejan el mismo patrón: `semantic_key_f1` pasa de 27,78 % a 95,52 %, y los campos obligatorios, que el baseline completaba en apenas el 70,59 % de las muestras, aparecen completos en el 100 % de los manifiestos generados por el SFT. La predicción de nivel exacto (`level_exact_match_rate`) mejora de 11,31 % a 75,78 %, lo que confirma que, en esta primera formulación, la serialización superficial de la jerarquía es suficiente para que el modelo aprenda a asignar indentaciones correctas en la gran mayoría de las líneas.

La Figura 5.2 muestra la evolución de la pérdida durante las tres épocas de entrenamiento.

La convergencia es rápida, especialmente en los primeros pasos. La pérdida cae de 1,32 a valores inferiores a 0,2 antes del fin de la primera época (paso 53), lo que sugiere que el modelo asimila la estructura del formato `blocks_tsv_v1` con pocas iteraciones. Las épocas segunda y tercera refinan los casos residuales —posiblemente manifiestos con mayor profundidad jerárquica o con múltiples documentos, aunque el log de entrenamiento no desglosa la pérdida por muestra— sin descensos tan abruptos, sino con una reducción progresiva y ruidosa que es esperable en un entrenamiento con batch efectivo pequeño. Cabe

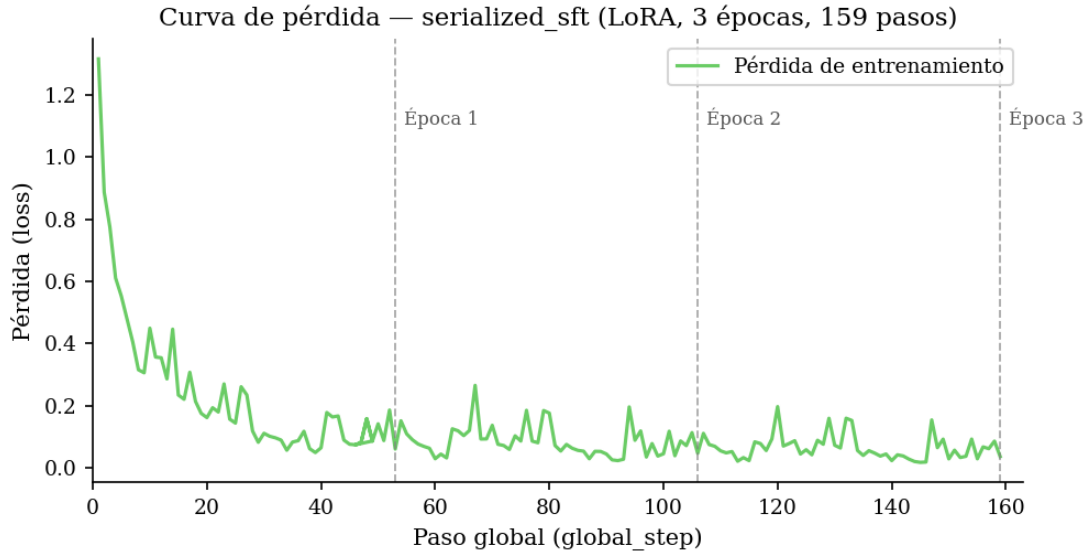


Figura 5.2: Curva de pérdida de entrenamiento del `serialized_sft` (LoRA, 3 épocas, 159 pasos globales, 424 muestras de entrenamiento efectivas (426 nominales, 2 omitidas por OOM)). La pérdida desciende de 1,32 en el primer paso a valores inferiores a 0,2 al final de la primera época. Las épocas segunda y tercera refinan los casos más complejos, con valores de pérdida entre 0,03 y 0,09.

señalar que durante el proceso se omitieron dos microbatches por recuperación de OOM: los ejemplos `q229::question` y `q229::question_simplified`, que superan los 3100 caracteres de prompt y objetivo combinados. La omisión está registrada en `oom_skipped_batches.jsonl`; el conjunto de entrenamiento efectivo es de 424 muestras.

La Figura 5.3 resume el salto en las cuatro métricas principales.

A pesar de ese avance, la compuerta de validez de dominio Kubernetes descubre un cuello de botella que la supervisión pura no resuelve: solo el 14,29 % de las muestras superan la compuerta completa. La Tabla 5.3 desglosa la tasa de superación por nivel de compuerta.

Los niveles 0 a 4 se superan con tasas del 90 % al 99 %, lo que confirma que el SFT gestiona bien los aspectos estructurales y semánticos básicos de los manifiestos. El colapso se produce en el nivel 5, que evalúa prácticas de seguridad: especificación de límites de recursos de CPU y memoria, configuración del contexto de seguridad del contenedor (`runAsNonRoot`, `readOnlyRootFilesystem`) y uso de etiquetas de imagen no flotantes. En ese nivel la tasa cae al 14,29 %. El nivel de validez de dominio promedio por muestra es de 3,9, coherente con el perfil observado: la mayoría de los manifiestos alcanzan el nivel 4 y fallan en el umbral de seguridad.

La magnitud de ese cuello de botella puede expresarse como la caída entre la última compuerta semántica superada de forma mayoritaria y la compuerta de seguridad:

$$\text{drop}_5 = \text{pass}_4 - \text{pass}_5 = 0,9000 - 0,1429 = 0,7571.$$

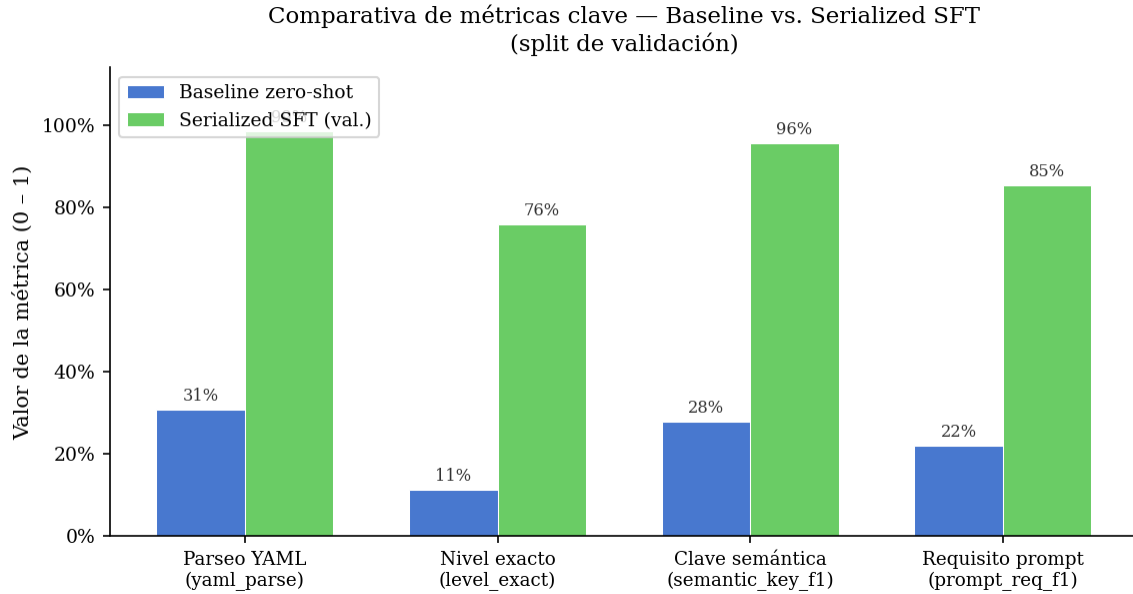


Figura 5.3: Comparativa de métricas clave entre el baseline zero-shot y el `serialized_sft` en el split de validación. El salto de 31 % a 99 % en parseo YAML y de 28 % a 96 % en clave semántica muestra que el SFT resuelve de forma simultánea el problema estructural y el de cobertura semántica. La predicción de nivel exacto mejora de 11 % a 76 %, confirmando que la serialización implícita de jerarquía es suficiente para aprender indentaciones correctas.

Tabla 5.3: Tasa de superación de la compuerta de validez de dominio Kubernetes por nivel para el `serialized_sft` en el split de validación (70 muestras). Los niveles son acumulativos: superar el nivel k implica haber superado los niveles $0, \dots, k - 1$. La compuerta final corresponde al nivel 5.

Nivel	Criterio evaluado	Tasa de paso
0	Identidad básica: <code>apiVersion</code> y <code>kind</code> reconocibles	98,57 %
1	Metadatos mínimos: <code>metadata.name</code> presente	98,57 %
2	Campos obligatorios según el <code>kind</code>	97,14 %
3	Estructura <code>spec</code> completa (contenedores, puertos)	91,43 %
4	Consistencia selector-template y coherencia de volúmenes	90,00 %
5	Prácticas de seguridad (límites de recursos, contexto, imagen)	14,29 %
kubernetes_domain_gate_pass_rate (compuerta final)		14,29 %

Esta caída de 75,71 puntos porcentuales resume por qué el problema residual no es ya principalmente sintáctico ni estructural, sino de alineación con criterios de calidad estática.

Este resultado no implica que la representación serializada sea incorrecta ni insuficiente para capturar jerarquía. Lo que ocurre es distinto: el SFT ha aprendido a generar manifiestos estructuralmente correctos y funcionalmente coherentes, pero ha aprendido también a reproducir los patrones que predominan en los datos de entrenamiento, que probablemente incluyen manifiestos de ejemplo o de desarrollo sin restricciones de seguridad, un patrón coherente con estudios empíricos sobre misconfigurations y smells de seguridad en Kubernetes [34, 48]. La compuerta de nivel 5 detecta precisamente esa ausencia sistemática. No es un fallo de formato sino de alineación con las prácticas que el evaluador de dominio considera obligatorias.

Un experimento adicional permite precisar esa afirmación. Se evaluó un run de entrenamiento independiente de una sola época con configuración idéntica (`serialized-sft-a-v1-1epoch-20260507-1528`) cuyo checkpoint final (paso 53) corresponde al cierre de la primera época; el checkpoint equivalente del run de tres épocas fue descartado por la política de retención de checkpoints. El objetivo es desagregar cuánto del salto total produce cada fase de entrenamiento. Con una sola época el modelo ya alcanza una tasa de parseo YAML del 90 %, un `semantic_key_f1` del 85,83 % y un MAE de nivel de 0,297. Las dos épocas restantes refinan esas cifras de forma apreciable: el parseo YAML sube hasta el 98,57 %, el `semantic_key_f1` escala al 95,52 % y el MAE de nivel descende a 0,272. Lo que no cambia en absoluto es la tasa de superación de la compuerta de nivel 5: 14,29 % al final de la primera época y exactamente 14,29 % al final de la tercera. Más iteraciones sobre los mismos datos refinan la calidad estructural y semántica del modelo, pero no acercan ni un punto porcentual al techo impuesto por los requisitos de seguridad. Esa invarianza es el argumento empírico más directo contra una lectura de insuficiencia de entrenamiento: el cuello de botella no está en cuántas veces ve el modelo los datos, sino en qué señal contienen esos datos. La Tabla 5.4 recoge la comparativa completa entre ambos checkpoints.

Tabla 5.4: Comparativa de métricas entre el run de 1 época (paso 53) y el modelo final de 3 épocas (paso 159) del `serialized_sft` en el split de validación (70 muestras). Δ indica la diferencia en puntos porcentuales o en unidades absolutas (`level_mae`).

Métrica	1 época	3 épocas	Δ
<code>yaml_parse_success_rate</code>	90,00 %	98,57 %	+8,57
<code>average_level_mae</code>	0,2974	0,2723	−0,025
<code>average_level_exact_match_rate</code>	66,44 %	75,78 %	+9,34
<code>average_line_text_f1</code>	73,37 %	82,06 %	+8,69
<code>average_semantic_key_f1</code>	85,83 %	95,52 %	+9,69
<code>average_prompt_requirement_f1</code>	77,16 %	85,31 %	+8,15
<code>required_field_complete_sample_rate</code>	98,41 %	100,00 %	+1,59
<code>kubernetes_domain_gate_pass_rate</code>	14,29 %	14,29 %	0,00

5.3 Análisis de errores del SFT serializado

Confrontar el perfil de errores del baseline con el del SFT resulta revelador porque los dos fallan de formas cualitativamente distintas. El baseline fallaba principalmente por inconsistencia estructural: niveles de indentación incorrectos, subtrees truncados, confusión entre mappings y listas en posiciones profundas del árbol YAML. El SFT serializado elimina casi por completo ese tipo de error. Lo que permanece es diferente en naturaleza.

La Tabla 5.5 muestra los conteos de errores de dominio detectados por el evaluador automático en el split de validación, acumulados sobre todos los recursos generados.

Tabla 5.5: Perfil de errores de dominio del `serialized_sft` en el split de validación (70 muestras). Los conteos son acumulados sobre todos los recursos generados; una muestra puede contener varios recursos y contribuir con múltiples instancias del mismo tipo de error.

Tipo de error	Descripción	Inst.	Categoría
<code>missing_resource_requirement</code>	Sin límites de CPU/memoria	261	Seguridad
<code>missing_run_as_non_root</code>	Contenedor ejecutable como root	71	Seguridad
<code>missing_read_only_root_filesystem</code>	En raíz no read-only	71	Seguridad
<code>latest_image_tag</code>	Imagen con etiqueta <code>:latest</code>	67	Seguridad
<code>volume_mount_without_volume</code>	Volumen montado sin declaración	5	Semántica
<code>service_selector_without_workload</code>	Selector sin workload asociado	1	Semántica
<code>yaml_parse</code>	Fallo residual de parseo YAML	1	Estructura
<code>kubernetes_identity</code>	Identidad de recurso inválida	1	Estructura
Antipatrones de seguridad (subtotal)		470	
Errores estructurales y semánticos (subtotal)		8	

La distribución es llamativa por su asimetría. Los cuatro antipatrones de seguridad acumulan 470 instancias frente a 8 instancias de errores estructurales o de semántica local. En términos relativos, la proporción de errores atribuibles a seguridad dentro del perfil medido es:

$$\rho_{\text{seguridad}} = \frac{470}{470 + 8} = 0,983.$$

Es decir, el 98,3 % de las instancias registradas en este perfil pertenecen a la misma familia de fallo. La Figura 5.4 lo visualiza.

Este perfil tiene una interpretación concreta. El SFT ha aprendido a reproducir la superficie de los manifiestos Kubernetes con alta fidelidad, incluyendo los patrones de los ejemplos de

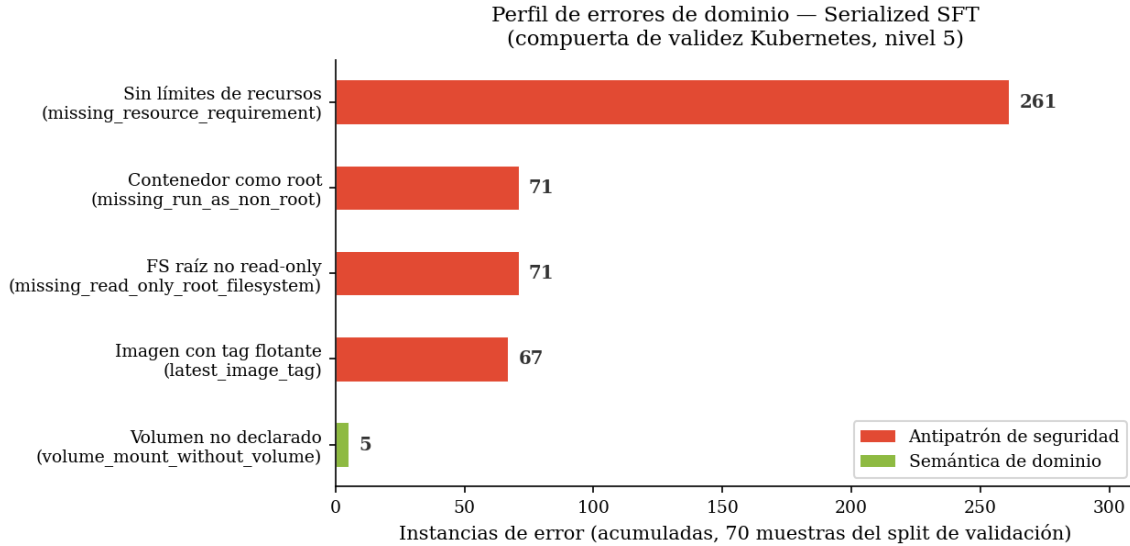


Figura 5.4: Distribución de errores de dominio del `serialized_sft` en el split de validación. Los cuatro antipatrones de seguridad (rojo) acumulan 470 instancias; los errores de semántica de dominio (verde) se limitan a 7 instancias. El perfil refleja que el SFT ha resuelto el problema estructural pero ha heredado los patrones inseguros presentes en los datos de entrenamiento.

entrenamiento. Si esos ejemplos no especifican límites de recursos, no configuran `runAsNonRoot` ni `readOnlyRootFilesystem`, y emplean la etiqueta `:latest` en las imágenes, el modelo aprende a hacer lo mismo. La estructura resultante es correcta, los campos requeridos están presentes y la jerarquía es coherente; pero las restricciones de seguridad que un entorno de producción exigiría están sistemáticamente ausentes. No es un límite de la representación serializada: es un límite de lo que la supervisión sola puede transmitir.

Los errores estructurales residuales merecen un comentario separado. Las 5 instancias de `volume_mount_without_volume` representan un error de coherencia semántica local: el modelo declara una referencia a un volumen en la spec del contenedor, pero no incluye la declaración correspondiente en la sección `volumes` del pod. Se trata de un problema de alcance contextual—el modelo no conecta las dos partes de la especificación— y no de indentación ni de parseo. El único caso de `service_selector_without_workload` y el único fallo residual de parseo YAML son instancias aisladas sin patrón sistemático.

El análisis de errores del SFT serializado señala, por tanto, un cambio cualitativo respecto al baseline. Antes de la supervisión, el límite era estructural: el modelo no mantenía la jerarquía. Después de la supervisión, el límite es de alineación: el modelo no incorpora las prácticas de seguridad que el evaluador de dominio requiere. Esa distinción importa en dos direcciones. Por un lado, el gap de seguridad muestra que la supervisión pura sobre los datos actuales no es suficiente para adquirir las prácticas de dominio que el evaluador requiere; la señal que falta no es de cantidad sino de preferencia, y eso es lo que abre la puerta a una fase de alineación basada en DPO [33, 44]. Por otro lado, la pregunta estructural que motiva el diseño del trabajo sigue abierta: el capítulo siguiente evalúa si una cabeza de predicción explícita de

`level` —en lugar de su serialización en la superficie del texto— produce mejoras adicionales en la calidad jerárquica que la variante serializada no alcanza. Ambas líneas de continuidad arrancan del perfil de errores documentado aquí.

Capítulo 6

Resultados II: Ajuste Fino con Predicción Explícita de Nivel

El capítulo anterior dejó abierta una pregunta que el SFT serializado no puede responder por sí mismo: ¿hasta qué punto la jerarquía mejora cuando el modelo dispone de una señal estructural dedicada en lugar de tener que deducirla del texto generado? En el `serialized_sft`, la predicción del nivel es una consecuencia implícita del aprendizaje lingüístico: el modelo aprende que tras ciertos contextos tienden a aparecer ciertos dígitos. No hay nada en la función de pérdida que le diga explícitamente que ese dígito representa una posición en el árbol jerárquico del YAML. Este capítulo explora una arquitectura alternativa que supervisa directamente esta variable.

La idea central de este capítulo es sencilla de enunciar: añadir un cabezal adicional que se supervisa directamente sobre la variable `level`, de modo que el modelo no solo aprenda a generar texto sino también a predecir la profundidad jerárquica de cada línea como una tarea separada. La arquitectura resultante, denominada THM (*Two-Headed Model*), mantiene la misma superficie de generación que el SFT serializado pero desacopla la señal estructural de la señal textual. El texto y el nivel dejan de competir por el mismo token: el nivel se convierte en una etiqueta supervisada que el modelo aprende en paralelo a la generación.

Como se verá, la idea es más fácil de enunciar que de hacer funcionar.

6.1 Motivación: los límites de la supervisión implícita

El `serialized_sft` del capítulo anterior logró resultados notables: una tasa de parseo YAML del 98,57 % y una coincidencia exacta de nivel del 75,78 %. Sin embargo, si se examina de cerca la naturaleza de esos aciertos, aparece una limitación estructural importante. En el formato `blocks_tsv_v1`, el nivel es simplemente el primer campo de cada línea generada: un dígito entre 0 y 8, seguido de un tabulador y del contenido textual. El modelo aprende a colocar ese dígito correctamente mediante la misma mecánica con la que aprende a escribir cualquier otra secuencia de tokens: por asociación contextual y exposición supervisada. Hay señal suficiente en los datos para que el aprendizaje funcione razonablemente, pero esa señal

llega al modelo de forma indirecta, mezclada con la señal de contenido textual.

Este diseño tiene una consecuencia particular ya manifestada en el análisis del capítulo 3: el modelo aprende bien los niveles frecuentes (0 a 4) pero falla de forma sistemática en los niveles profundos (5 a 8). A primera vista, podría atribuirse al desbalanceo de clases: el nivel 5 representa solo el 2,9 % de las líneas de entrenamiento, y el nivel 8 aparece en apenas el 0,1 % de ellas. Pero el problema no se reduce a una cuestión de frecuencia. En el SFT serializado, el modelo llega a predecir dígitos de nivel razonablemente bien en promedio, pero no existe ningún mecanismo que le diga que ese dígito tiene una interpretación estructural: que un nivel 5 implica que la línea vive bajo cuatro niveles de anidamiento previo, que eso depende de qué recursos se hayan generado antes, y que una predicción incorrecta del nivel no solo produce un dígito malo sino que puede comprometer toda la estructura del documento.

La hipótesis de partida de este capítulo es que separar la señal de nivel de la señal textual podría ayudar en dos sentidos. Por un lado, la función de pérdida vería el nivel como una variable explícita a optimizar, no como un subproducto de la generación. Por otro lado, el modelo podría apoyarse en representaciones latentes que, como demostró la sonda diagnóstica del capítulo 3, ya codifican información jerárquica aprovechable. Esa doble intuición es la que motiva el diseño del THM (*Two-Headed Model*).

6.1.1 Análisis de la señal latente

Antes de diseñar una arquitectura con supervisión explícita de nivel, conviene recuperar un resultado del capítulo 3 que sirvió como comprobación de factibilidad. El experimento del espacio latente consistió en entrenar clasificadores ligeros sobre las representaciones ocultas del modelo base Qwen2.5-7B-Instruct para predecir la variable `level` sin que el modelo hubiera recibido ningún entrenamiento específico para ello. El resultado más sólido, obtenido con un clasificador lineal sobre el estado oculto en la primera posición token de cada línea (`line_first_token`), alcanzó una exactitud del 78,8 % y un error absoluto medio de 0,436 en el split de validación. Ese resultado no demostraba que el modelo pudiera generar niveles correctos, pero sí que la información estructural ya estaba presente en sus representaciones internas de alguna forma recuperable.

Sin embargo, como muestra la Figura 6.1, la señal no es uniforme a lo largo de la jerarquía. Los niveles 0 a 4 presentan un recall alto y relativamente estable. A partir del nivel 5 la recuperación cae de forma brusca: la aproximación realizada solo recupera el 31,6 % de las instancias de nivel 5 en validación, y los niveles 7 y 8 tienen soporte tan reducido que los resultados son poco fiables. Esto tiene una implicación directa para el diseño: la señal latente existe, pero está concentrada en los niveles superficiales de la jerarquía. Precisamente los niveles en los que el sistema ya funcionaba razonablemente bien. Para los niveles profundos, que son los más problemáticos, la sonda confirma que la señal es débil e irregular.

Este patrón orientó varias decisiones de diseño. En particular, la elección de la política de alineación del cabezal de nivel: en lugar de leer el estado oculto en la posición media de la línea o en el último token, se optó por leer el estado en el momento previo al primer token de cada bloque (`record_prefix_state`). Esta posición capta el estado del modelo justo antes de comenzar a generar el contenido del bloque, lo que intuitivamente debería concentrar más

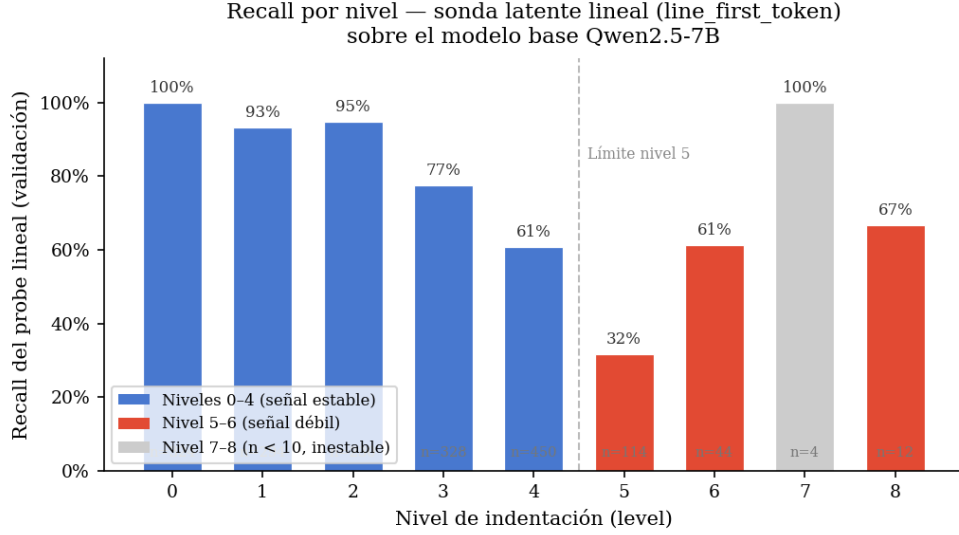


Figura 6.1: Recall por nivel del mejor aproximador lineal (`line_first_token`) sobre las representaciones del modelo base. Los niveles 0–4 muestran señal sólida; a partir del nivel 5 el recall cae por debajo del 32 %, y los niveles 7–8 son inestables por escasez de muestras.

información estructural sobre el contexto previo que sobre el contenido en curso.

6.2 Diseño de la arquitectura de dos cabezales

La idea del THM (*Two-Headed Model*) puede describirse de forma concisa: se toma el modelo base con LoRA y se le añade un segundo cabezal de predicción que opera en paralelo al cabezal de lenguaje estándar. El cabezal de lenguaje sigue generando el texto de cada bloque token a token, como en cualquier modelo autoregresivo. El cabezal de nivel, por su parte, toma el estado oculto en una posición específica antes de cada bloque y produce una predicción de clase sobre los nueve niveles posibles (0 a 8). Las dos pérdidas se combinan de forma aditiva con un coeficiente λ que controla el peso relativo de la señal estructural.

La Figura 6.2 ilustra el esquema general. El backbone Qwen2.5-7B con adaptadores LoRA procesa la secuencia de entrada y genera una sucesión de estados ocultos $h_t \in \mathbb{R}^{3584}$. En cada posición que marca el inicio de un nuevo bloque, el cabezal de nivel lee ese estado oculto y produce una distribución sobre los nueve niveles de indentación. El cabezal de lenguaje opera de forma estándar sobre todos los estados ocultos de la secuencia, generando el texto del bloque token a token. La predicción final del nivel proviene exclusivamente del cabezal estructural, no del texto generado: a diferencia del `serialized_sft`, el nivel no aparece como un campo en la superficie de texto del bloque.

La Tabla 6.1 resume las diferencias de formato entre ambas variantes supervisadas.

El cabezal de nivel es un perceptrón multicapa con una capa oculta de dimensión 256 y dropout del 5 %. La función de pérdida es entropía cruzada sobre las nueve clases, con $\lambda = 1$ para el peso del cabezal de nivel respecto al cabezal de lenguaje. Los adaptadores LoRA

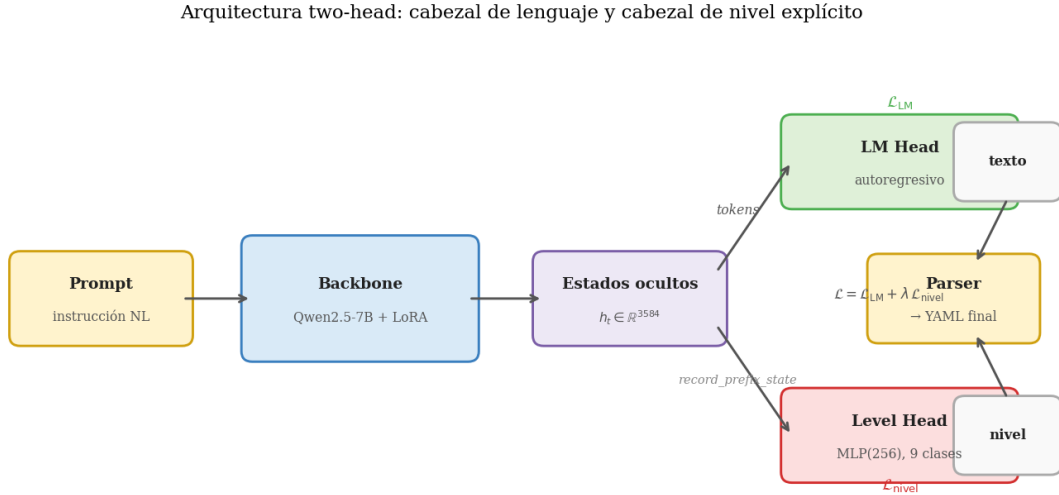


Figura 6.2: Arquitectura del THM: el backbone compartido alimenta simultáneamente el cabezal de lenguaje y el cabezal de nivel. La política `record_prefix_state` lee el estado oculto justo antes del primer token de cada bloque para supervisar la predicción de nivel.

Tabla 6.1: Comparación de formatos de superficie entre `serialized_sft` y `two_head_sft`. En la variante de dos cabezales, el campo de nivel desaparece del texto generado y pasa a ser una etiqueta supervisada exclusiva del cabezal estructural.

Aspecto	<code>serialized_sft</code>	<code>two_head_sft</code>
Formato de bloque	<code>blocks_tsv_v1</code>	<code>content_blocks_v1</code>
Nivel en superficie	Sí (primer campo: 0, 1...)	No (etiqueta supervisada)
Cabezales de predicción	1 (LM head)	2 (LM head + level head)
Política de alineación	N/A	<code>record_prefix_state</code>
Pérdida combinada	$\mathcal{L}_{\text{LM}}$	$\mathcal{L}_{\text{LM}} + \lambda \mathcal{L}_{\text{nivel}}$

tienen rango 8, escala 16 y se aplican a las proyecciones de atención (Q, K, V, O). El resto de hiperparámetros de entrenamiento son idénticos a los del `serialized_sft`: tasa de aprendizaje 2×10^{-4} , tres épocas, tamaño de lote efectivo 8 (acumulación de gradientes sobre batches de tamaño 1), semilla 42 y 426 muestras de entrenamiento. Esta identidad de configuración es deliberada: permite atribuir cualquier diferencia en resultados a la arquitectura, no a diferencias de entrenamiento.

La transformación del formato de datos es sencilla pero significativa. El formato `blocks_tsv_v1` incluye cuatro columnas por bloque: `doc_idx`, `line_idx`, `level` y `line_text`. El formato `content_blocks_v1` elimina la columna `level`: solo contiene `doc_idx`, `line_idx` y `line_text`. El nivel desaparece por completo de la superficie textual y existe únicamente como etiqueta en el proceso de supervisión. Esto significa que durante la inferencia, el modelo no puede “leer” el nivel que acaba de predecir en el siguiente token: la predicción del cabezal de nivel es invisible para el cabezal de lenguaje.

6.3 Resultados del experimento base: THM v1

El experimento base (THM v1, checkpoint final, paso 159) se evaluó sobre el split de validación completo de 70 muestras. La Tabla 6.2 presenta la comparativa principal respecto al baseline zero-shot definido en el Capítulo 5. Esta es la comparación relevante en este punto del argumento: antes de confrontar distintas familias supervisadas entre sí, conviene establecer si el cabezal estructural explícito mejora el punto de partida sin ajuste fino.

Tabla 6.2: Comparativa de métricas compartidas en el split de validación: baseline zero-shot vs. THM v1 (70 muestras). La columna Δ indica THM v1 menos baseline; en `average_level_mae`, un valor negativo indica mejora porque la métrica es un error.

Métrica	Baseline	THM v1	Δ
Muestras evaluadas	65 / 70	70 / 70	—
<code>yaml_parse_success_rate</code>	30,77 %	48,57 %	+17,80
<code>average_level_exact_match_rate</code>	11,31 %	35,04 %	+23,73
<code>average_level_mae</code>	0,7896	0,239	−0,551
<code>average_line_text_f1</code>	20,86 %	37,43 %	+16,57
<code>average_semantic_key_f1</code>	27,78 %	45,92 %	+18,14
<code>average_prompt_requirement_f1</code>	22,13 %	42,06 %	+19,93

El resultado es claramente mejor que el punto de partida. El parseo YAML sube de 30,77 % a 48,57 %, la coincidencia exacta de nivel pasa de 11,31 % a 35,04 %, y las métricas de contenido también aumentan: el F1 de línea crece de 20,86 % a 37,43 %, el F1 semántico de 27,78 % a 45,92 %, y el F1 de adecuación al prompt de 22,13 % a 42,06 %. La señal principal, por tanto, no es que el THM v1 sea inferior a otras ramas supervisadas, sino que la separación explícita entre contenido textual y nivel jerárquico produce una mejora sustantiva frente al modelo sin ajuste.

El error absoluto medio de nivel confirma esa lectura inicial: baja de 0,7896 en el baseline a

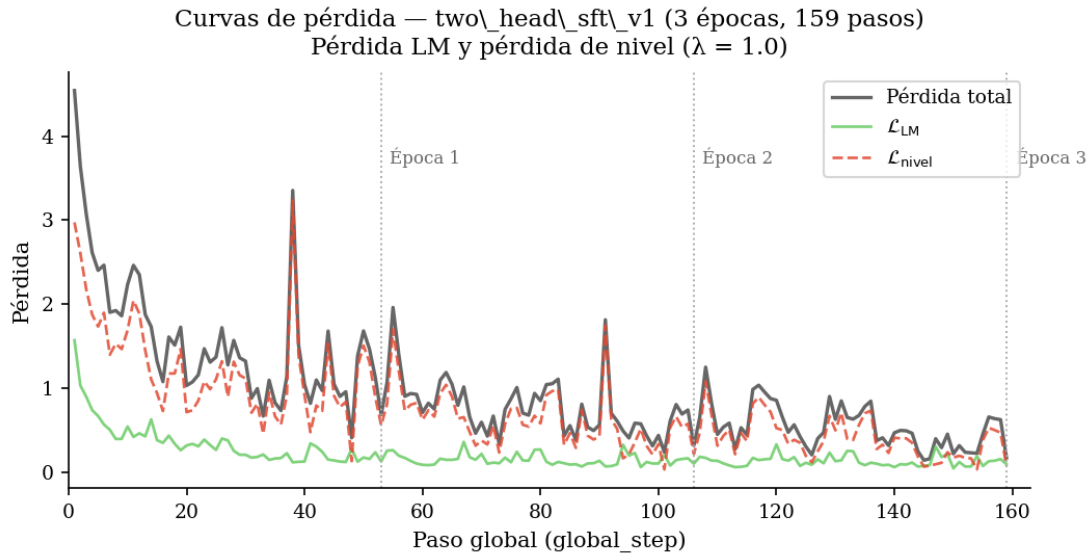


Figura 6.3: Curvas de pérdida durante el entrenamiento del THM v1: pérdida total, componente LM y componente de nivel ($\lambda = 1.0$, 3 épocas, 159 pasos globales).

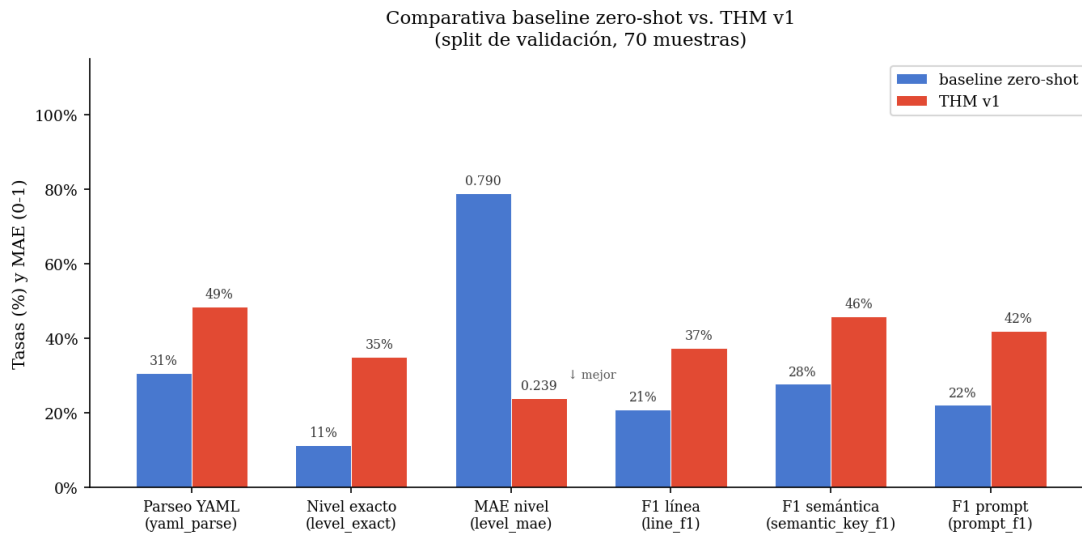


Figura 6.4: Comparativa visual de métricas clave entre el baseline zero-shot y THM v1 en el split de validación.

0,239 en el THM v1. Ahora bien, debemos de tener en cuenta que el MAE medio puede ser bajo si el modelo acierta muchos niveles frecuentes y, al mismo tiempo, ignora por completo la región profunda de la jerarquía.

La Figura 6.3 muestra las curvas de entrenamiento. La pérdida LM desciende de forma estable: caída rápida en la primera época, refinamiento progresivo en las dos siguientes. La pérdida de nivel, en cambio, presenta un comportamiento diferente: también desciende, pero a un ritmo más lento y sin llegar a los niveles de la pérdida LM. El modelo aprende simultáneamente ambas tareas, pero no con la misma eficiencia, lo que sugiere que la señal estructural es más difícil de incorporar que la señal textual.

6.3.1 Colapso hacia los niveles superficiales

El diagnóstico más revelador del experimento base no está en las métricas globales sino en la distribución de predicciones de nivel. La Figura 6.5 muestra la distribución de niveles gold en el split de validación junto con la distribución de los niveles predichos por el cabezal estructural.

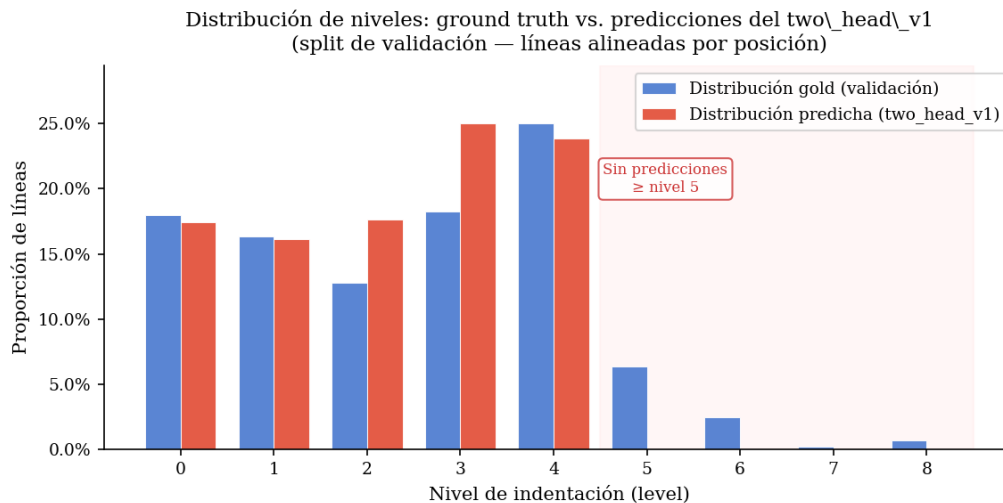


Figura 6.5: Distribución comparada de niveles: ground truth de validación (azul) y predicciones del THM v1 (rojo). El modelo emite cero predicciones para los niveles 5 a 8; la zona sombreada marca la región de colapso.

Se puede vislumbrar un patrón claro: el cabezal de nivel del THM v1 no emite ninguna predicción con valor mayor o igual a 5. De las 1800 líneas de validación, 174 tienen nivel gold igual o superior a 5. Ninguna de ellas recibe una predicción correcta. El recall exacto para niveles 5–8 es del 0 %; el recall con tolerancia de un nivel (off-by-one) es del 28 %, lo que indica que el modelo se acerca a esos niveles desde abajo pero nunca cruza el umbral de decisión.

La causa estructural parecía ser el desbalanceo extremo de clases. El nivel 5 representa el 2,9 % de las líneas de entrenamiento. El nivel 6 el 2,6 %. El nivel 7 el 0,5 %. El nivel 8 el 0,1 %, lo que equivale a 8 líneas de entrenamiento sobre un total de 8336. Con una función

de pérdida de entropía cruzada estándar y sin corrección de clase, el gradiente que proviene de las instancias de nivel 8 es aproximadamente 115 veces más débil que el de las instancias de nivel 0. En la práctica, el cabezal aprende que es seguro ignorar los niveles profundos: el coste de equivocarse en ellos es mínimo comparado con el coste de equivocarse en los niveles frecuentes.

Las muestras con parseo YAML fallido presentan una tasa de presencia de contenedores (`containers`) del 97,2 %, frente al 79,4 % en las muestras parseables. Los manifests de tipo `Deployment` o `StatefulSet`, que son precisamente los que generan estructuras de nivel 5 o superior a través del anidamiento `spec→template→spec→containers→volumeMounts`, concentran de forma desproporcionada los fallos.

6.4 Los modelos clasificadores ordinales

El colapso hacia niveles superficiales está relacionado directamente con el tratamiento de la predicción de nivel como una clasificación categórica estándar. La entropía cruzada multiclase no tiene noción del orden entre etiquetas: equivocarse entre nivel 3 y nivel 4 tiene el mismo coste que equivocarse entre nivel 3 y nivel 8. Pero en este contexto, el orden importa.

La segunda familia de experimentos, denominada THO o variante ordinal versión 2, reformula la predicción de nivel como un problema de regresión ordinal. En lugar de producir directamente una distribución sobre nueve clases, el cabezal proyecta el estado oculto a un escalar continuo z mediante un MLP, y ese escalar se compara con una secuencia de umbrales aprendibles $\tau_0 < \tau_1 < \dots < \tau_7$. El nivel predicho es el número de umbrales que z supera:

$$\hat{\ell} = \sum_{k=0}^7 \mathbf{1}[z > \tau_k].$$

La pérdida se define como la suma de entropías cruzadas binarias para cada uno de los ocho problemas de umbral: “¿es el nivel mayor que k ?”. Esta formulación obliga al modelo a aprender una representación continua que respeta el orden entre clases, y permite que los umbrales sean aprendidos conjuntamente con el resto de la red.

Se probaron cuatro variantes dentro de esta familia, progresivamente modificadas en función de los diagnósticos obtenidos en cada run. La Tabla 6.3 resume los resultados de las cuatro variantes en el split de validación.

6.4.1 Variante inicial y tasa de aprendizaje de umbrales (lr25)

La variante inicial establece los umbrales con espaciado uniforme de 1,0 unidad ($\tau_0 = 0,5$, $\tau_1 = 1,5$, $\dots$, $\tau_7 = 7,5$) y usa la misma tasa de aprendizaje para todos los parámetros. Los resultados son peores que el THM v1 en parseo YAML (25,71 % frente a 48,57 %) pero no mejores en recall de niveles profundos: sigue siendo cero el recall exacto para niveles 5-8, aunque el off-by-one alcanza el 21,9 %, indicando que el escalar z se acerca al umbral pero no lo cruza.

Tabla 6.3: Comparativa de las cuatro variantes ordinales en el split de validación (70 muestras). La columna “Recall exacto 5–8” recoge el recall del cabezal sobre niveles profundos. La columna “Off-by-1 profundo” es el recall con tolerancia de ± 1 para niveles 5–8.

Variante	YAML parse	Nivel exacto	MAE nivel	F1 línea	Exacto 5–8	Off-1 5–8
initial	25,71 %	5,54 %	0,770	18,99 %	0,00 %	21,90 %
lr25 (thresh. $\times 25$)	28,57 %	6,93 %	0,806	22,69 %	0,00 %	25,36 %
mlp3 (MLP 3 cap.)	14,29 %	4,09 %	0,918	11,53 %	0,00 %	0,00 %
centered (gap=0,5)	2,86 %	1,07 %	0,456	1,45 %	21,74 %	56,52 %

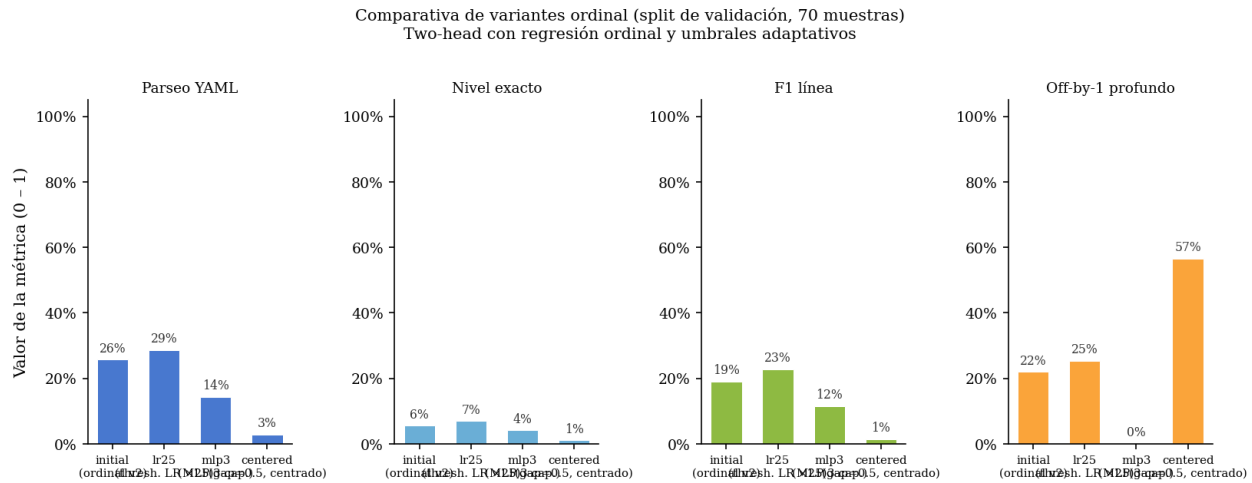


Figura 6.6: Comparativa visual de las cuatro variantes ordinal en cuatro métricas: parseo YAML, coincidencia exacta de nivel, F1 de contenido textual y recall off-by-one en niveles profundos (5–8).

El diagnóstico de los umbrales revela el problema de fondo: durante el entrenamiento completo (159 pasos), los umbrales apenas se desplazan. La variación total de τ_0 a τ_7 es del orden de 10^{-3} , mientras que el escalar latente z oscila en un rango de -3 a $+9$. El modelo aprende ajustando z y no los umbrales. La interpretación es que el gradiente de la pérdida de umbral, pequeño en relación con el gradiente de la pérdida de contenido, es insuficiente para mover parámetros de umbral cuya inicialización ya permite una clasificación aproximada de los casos frecuentes.

La variante lr25 multiplica la tasa de aprendizaje de los parámetros de umbral por un factor de 25 para intentar acelerar su ajuste. El resultado mejora marginalmente en parseo (28,57 %) y en off-by-one profundo (25,36 %), pero los umbrales siguen sin desplazarse de forma significativa. El cambio en la tasa de aprendizaje produce un efecto observable, pero no suficiente para romper el patrón de colapso.

6.4.2 MLP de tres capas (mlp3)

La variante mlp3 aumenta la capacidad del cabezal de nivel a un MLP de tres capas con dimensiones (512, 256), lo que da al sistema más expresividad para proyectar el estado oculto al escalar latente z . También aumenta el factor multiplicador de la tasa de aprendizaje de umbrales a 50. Los resultados son los peores de la familia: 14,29 % de parseo YAML y un off-by-one de nivel profundo del 0 %. La hipótesis más plausible es que la mayor capacidad del cabezal produce gradientes de nivel más grandes que interfieren con la generación textual, empujando al backbone hacia representaciones que dificultan el seguimiento de la secuencia de texto.

6.5 Umbrales centrados: el primer acceso a los niveles profundos

La variante *centered-gap05* parte de una hipótesis diferente: si los umbrales se inicializan cerca de cero con espaciado estrecho (gap = 0,5, centrados en el origen: $\tau_0 = -1,75$, $\tau_1 = -1,25$, ..., $\tau_7 = +1,75$), el escalar z no necesita alejarse tanto de la región central para cruzarlos. En las variantes anteriores, el umbral τ_4 estaba en 4,5, lo que obligaba a z a desplazarse mucho para predecir niveles altos. Con la inicialización centrada, cualquier pequeño movimiento de z en la dirección positiva puede activar predicciones de nivel profundo.

La Figura 6.7 muestra la evolución de los umbrales durante el entrenamiento. Como en las variantes anteriores, los umbrales apenas se mueven: la variación total es del orden de 0,5 unidades a lo largo de 159 pasos, lo que contrasta con la magnitud del escalar z . El aprendizaje sigue ocurriendo principalmente en z , no en los umbrales.

Sin embargo, por primera vez en la familia de experimentos two-head, el modelo emite predicciones de nivel mayor o igual a 5. El recall exacto para niveles 5–8 es del 21,74 %, y el off-by-one alcanza el 56,52 %, lo que indica que el sistema está activando predicciones en la zona profunda de la jerarquía de forma no trivial. Este resultado es cualitativamente distinto al de todas las variantes anteriores: la señal existe, el cabezal puede aprender a cruzar los

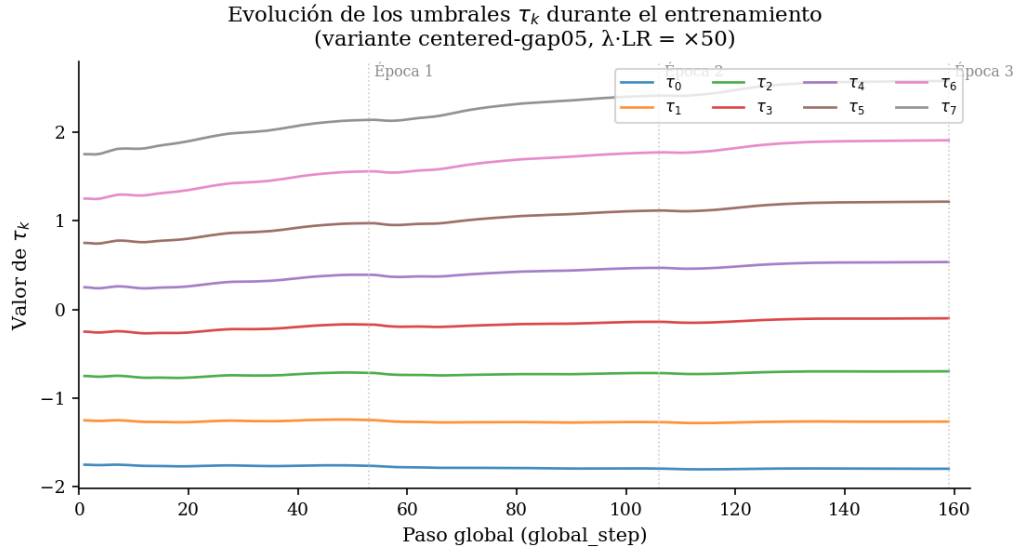


Figura 6.7: Evolución de los ocho umbrales τ_k durante el entrenamiento de la variante *centered-gap05* (159 pasos, 3 épocas). A pesar del alto factor de tasa de aprendizaje ($\times 50$), los umbrales apenas se desplazan respecto a su inicialización.

umbrales profundos si la inicialización lo facilita.

El coste, sin embargo, es severo. La tasa de parseo YAML cae al 2,86 % (2 muestras de 70). La F1 de línea se desploma al 1,45 %. El modelo genera predicciones de nivel profundo pero pierde por completo la capacidad de generar estructuras correctas.

La Figura 6.8 hace visible el trade-off de forma directa: ninguna variante consigue a la vez un parseo YAML razonable y un recall no nulo en los niveles profundos. Las cinco variantes ocupan posiciones opuestas en ese espacio. El *two_head_v1* y las primeras variantes ordinales mantienen una tasa de parseo moderada pero recall profundo nulo. La variante *centered* activa el recall profundo pero destruye el parseo. No existe en el espacio explorado ninguna solución que haga las dos cosas a la vez.

6.6 Análisis de errores del THO centrado

A primera vista, el THO centrado podía leerse como una degradación clara: las métricas globales eran malas, el YAML generado era mayoritariamente inválido y la utilidad práctica del modelo quedaba muy por debajo del THM v1. Sin embargo, precisamente porque esta variante consiguió activar niveles profundos por primera vez, su fallo resultaba informativo. El análisis cualitativo de las salidas mostró que el problema no era solo la aparición de etiquetas raras, sino la posición en la que esas etiquetas aparecían dentro de la trayectoria del manifiesto.

El primer indicio apareció al comparar manualmente algunas predicciones de la variante *centered-gap05* con sus referencias. En varios ejemplos, el contenido textual generado era reconocible: aparecían recursos Kubernetes esperables, claves coherentes y secuencias de

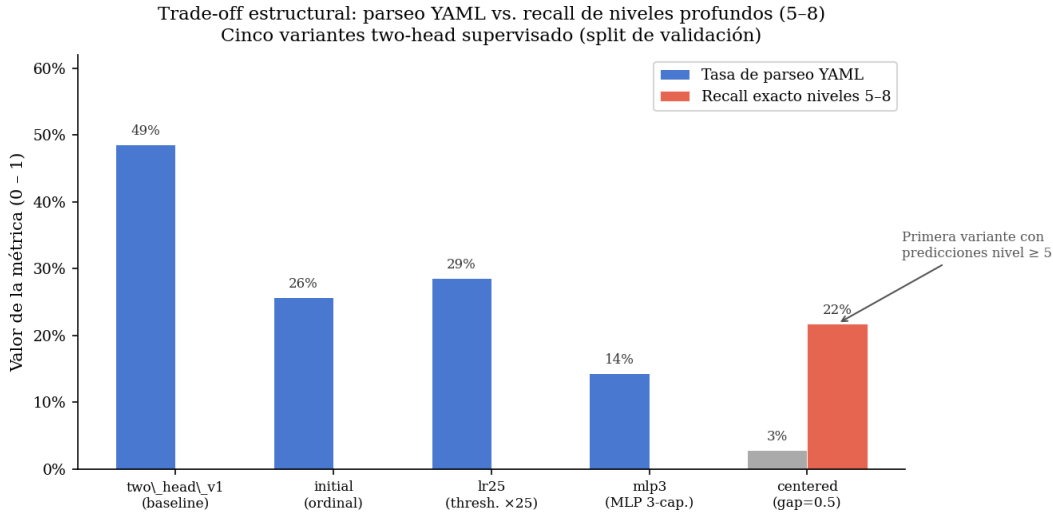


Figura 6.8: Trade-off entre tasa de parseo YAML y recall exacto de niveles profundos (5–8) para las cinco variantes two-head probadas. Solo la variante *centered* activa predicciones profundas, pero a costa de colapsar la generación de contenido.

campos razonables. El problema estaba en la trayectoria de niveles que acompañaba a esas líneas. De hecho, en los 24 ejemplos donde el número de bloques generado coincidía con el número de bloques de la referencia, el YAML reconstruido con los niveles predichos no parseaba en ningún caso, mientras que al sustituir solo esos niveles por los niveles *gold* parseaban 22 de los 24 ejemplos. Esta prueba no convierte la referencia en una solución disponible durante inferencia, pero sí separa claramente dos causas: una parte importante del fallo no procedía del texto de las líneas, sino de la indentación estructural que el cabezal asignaba a esas líneas.

Al inspeccionar esos fallos, el error no parecía distribuirse de forma uniforme a lo largo del manifiesto. El modelo no solo fallaba en los niveles profundos al final de la salida, sino que tendía a retrasar la entrada en la jerarquía desde las primeras fases de generación. En otras palabras, mantenía demasiadas líneas en `level=0` cuando el YAML de referencia ya había descendido a campos como `metadata`, `spec`, `selector` o `template`. Para comprobar si esta impresión era solo anecdótica, cada manifiesto se dividió en cuatro cuartiles relativos de longitud y se comparó la distribución de niveles predicha con la distribución real.

Tabla 6.4: Distribución de niveles por cuartil en el análisis de errores del THO centrado. El patrón más relevante aparece en Q2: el YAML de referencia casi ya no está en `level=0`, pero el modelo conserva una proporción elevada de predicciones superficiales.

Tramo	Media gold	Media pred.	Gold nivel 0	Pred. nivel 0	MAE alineado
Q1 (0–25 %)	0,50	0,16	62,5 %	89,5 %	0,32
Q2 (25–50 %)	2,32	1,39	1,8 %	34,9 %	1,10
Q3 (50–75 %)	3,42	3,04	2,6 %	3,5 %	0,76
Q4 (75–100 %)	3,96	3,55	1,0 %	1,7 %	0,62

La Tabla 6.4 permite matizar el diagnóstico. En el primer cuartil es normal encontrar muchos

niveles superficiales: todo manifiesto empieza con campos como `apiVersion`, `kind` y `metadata`. Por eso el exceso de ceros en Q1 no es, por sí solo, la prueba más importante. El punto decisivo aparece en Q2. Entre el 25 % y el 50 % de la secuencia, la referencia ya ha entrado de forma clara en niveles internos, pero el modelo todavía predice `level=0` en el 34,9 % de las líneas. La lectura más precisa, por tanto, no es simplemente que el THO prefiera clases frecuentes, sino que su predicción de profundidad llega tarde: la estructura real desciende antes de que el cabezal empiece a seguirla.

Esta observación llevó a formular el problema en términos temporales. La predicción de nivel no es una clasificación aislada de líneas independientes. Durante la generación autoregresiva, cada paso produce un estado oculto de alta dimensión y, a partir de ese estado, el cabezal estima el nivel jerárquico correspondiente. Si se denota por h_t el estado latente usado para predecir el nivel en la posición t , la salida puede entenderse como una serie temporal multivariante:

$$(h_1, \ell_1), (h_2, \ell_2), \dots, (h_T, \ell_T), \quad h_t \in \mathbb{R}^{3584}.$$

Desde esta perspectiva, no hay razones para asumir que la distribución de los estados h_t sea estacionaria a lo largo del YAML. Los estados del comienzo se calculan con menos contexto generado; los estados posteriores incorporan una historia más larga de claves, listas, recursos y decisiones previas del propio modelo. Esto no significa necesariamente que los estados iniciales sean peores o menos informativos. La hipótesis más prudente es otra: el espacio latente sobre el que opera el cabezal de nivel cambia con la posición de la línea, de modo que una misma etiqueta `level` puede ocupar regiones distintas al comienzo y al final del manifiesto. El cabezal ordinal, sin embargo, aprende una frontera global que trata todos esos estados como si fueran muestras intercambiables de una única distribución.

Para comprobar si esta hipótesis tenía soporte empírico, se reutilizaron los estados ocultos extraídos en el análisis latente previo y se calcularon distancias entre cuartiles manteniendo fijo el nivel real. La estrategia más relevante fue `record_prefix_state`, porque es la misma política de lectura utilizada por el cabezal del THM. El resultado fue significativo: para esa estrategia, la distancia media entre cuartiles dentro del mismo `level` fue prácticamente igual a la distancia media entre niveles distintos dentro del mismo cuartil. En términos intuitivos, un estado de `level=2` al comienzo del YAML podía estar tan lejos de otro estado de `level=2` al final como de estados pertenecientes a otros niveles. Esta es precisamente la situación en la que una frontera global puede quedar mal calibrada.

El análisis se reforzó con pruebas estadísticas sobre los mismos grupos. Primero, se aplicó MMD con kernel RBF para comparar Q1 frente a Q4 dentro de cada nivel con soporte suficiente. Todas las comparaciones evaluadas resultaron significativas tras corrección por comparaciones múltiples. Después, se aplicó PERMANOVA para comprobar si el cuartil explicaba variación multivariante dentro de un mismo nivel; de nuevo, el efecto apareció de forma sistemática. Finalmente, PERMDISP mostró que en muchos niveles la diferencia no era solo un desplazamiento del centroide, sino también un cambio en la dispersión interna de los estados. Este último matiz es importante: el problema no consiste únicamente en que el espacio se desplace suavemente, sino en que las regiones latentes cambian también su forma y variabilidad a lo largo de la secuencia.

Estos tests no demuestran por sí solos que la deriva posicional sea la causa directa del fallo generativo del THO. Los vectores analizados proceden de una extracción controlada y no de toda la dinámica libre de generación del modelo ajustado. Aun así, ofrecen una explicación coherente para el patrón observado: el cabezal de nivel está intentando aprender una regla global sobre una serie latente que no parece estacionaria. Por eso, incluso cuando la formulación ordinal consigue activar niveles profundos, esos niveles no aparecen necesariamente en las posiciones correctas ni forman una trayectoria de indentación estable. La dificultad no está solo en predecir etiquetas raras, sino en leer correctamente el estado estructural del modelo en cada momento de la generación.

6.6.1 Condicionamiento posicional del cabezal ordinal

El diagnóstico anterior sugiere una intervención bastante natural: si el espacio latente cambia con la posición de la línea, el cabezal de nivel no debería recibir únicamente el estado oculto del modelo, sino también alguna señal explícita sobre el momento de la generación en el que se encuentra. Esta idea no pretende convertir la posición en una nueva etiqueta ni multiplicar el problema en clases del tipo `level` por tramo posicional. La variable que se sigue prediciendo es el mismo nivel ordinal ℓ_t . Lo que cambia es la información disponible para estimarlo: además del estado latente h_t , el cabezal recibe una codificación causal de la posición absoluta de la línea generada.

La primera variante se diseñó de forma simple. La posición utilizada es el índice de línea que se está generando, no la longitud final del manifiesto, por lo que no introduce información no disponible en inferencia. Ese índice se transforma mediante una codificación sinusoidal de 16 dimensiones, siguiendo la intuición general de los encodings posicionales de los Transformers [39], y se concatena justo antes de la proyección final del cabezal ordinal. De forma esquemática, la arquitectura queda:

$$\begin{aligned} u_t &= f_h(h_t) \in \mathbb{R}^{64}, \\ p_t &= f_p(t) \in \mathbb{R}^{16}, \\ z_t &= w^\top [u_t; p_t] + b. \end{aligned}$$

Aquí z_t es el score ordinal que después se compara con los umbrales aprendidos para obtener el nivel predicho. La rama f_h proyecta el estado oculto mediante capas lineales, normalización y activación no lineal, mientras que f_p solo normaliza la codificación posicional sinusoidal. Esta elección tenía una ventaja metodológica: permitía comprobar si una señal temporal sencilla era útil antes de introducir mecanismos de condicionamiento más expresivos.

El experimento asociado, THOP, llegó al final del entrenamiento y se evaluó finalmente sobre las 70 muestras de validación.

La Tabla 6.5 y la Figura 6.9 muestran que la codificación posicional no resolvió el problema de fondo. La tasa de parseo YAML siguió siendo muy baja y el exact match de nivel quedó prácticamente anulado. El diagnóstico por cuartiles confirmó además que el colapso inicial hacia `level=0` persistía: en el primer cuartil, el modelo predijo `level=0` en el 94,7 % de las líneas, frente al 62,2 % de la referencia; en el segundo cuartil, donde la referencia solo tenía

Tabla 6.5: Resultado final del primer experimento con codificación posicional por concatenación final.

Métrica	Valor
Predicciones de validación generadas	70 / 70
Ejemplos evaluados	69
Parseo de la salida estructurada	98,57 %
Parseo YAML	7,25 %
Exact match medio de nivel	2,23 %
MAE medio de nivel	1,024
Recall exacto en niveles 5–8	13,79 %
Recall <i>off-by-one</i> en niveles 5–8	49,66 %
Compresión de niveles 5–8 a 0–4	69,66 %
KDV score medio	5,07 %
KDV gate pass	0,00 %

THOP frente al THO centrado: mejora parcial de superficie, no solución estructural

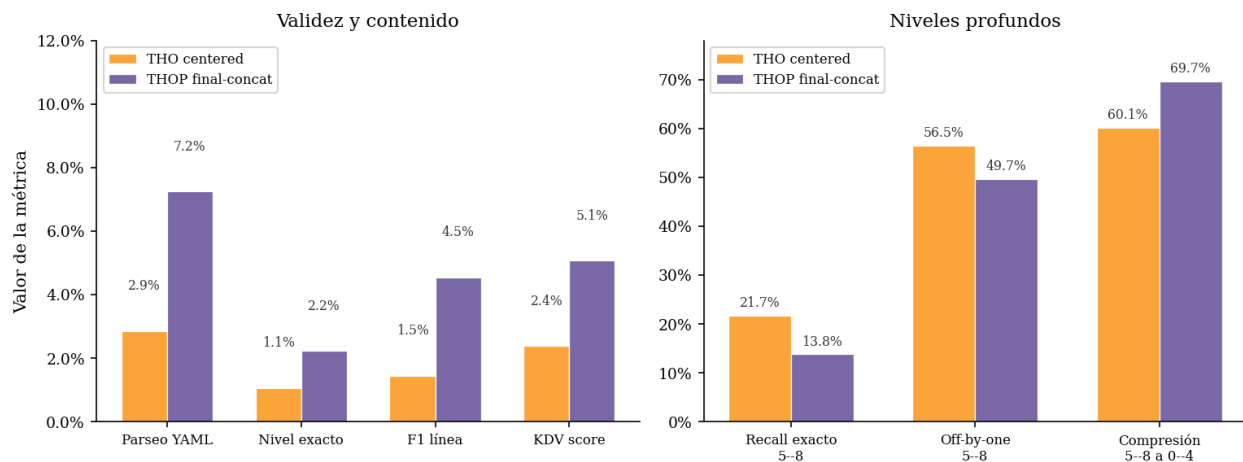


Figura 6.9: Comparación entre el THO centrado y el THOP con concatenación final. La compresión de niveles 5–8 a 0–4 debe leerse en sentido inverso: valores más altos indican más colapso de niveles profundos hacia niveles superficiales.

un 6,6 % de líneas en `level=0`, el modelo todavía asignó ese nivel al 51,8 % de las líneas. Es decir, la señal de posición no bastó para adelantar la entrada en la jerarquía ni para estabilizar la trayectoria de indentación.

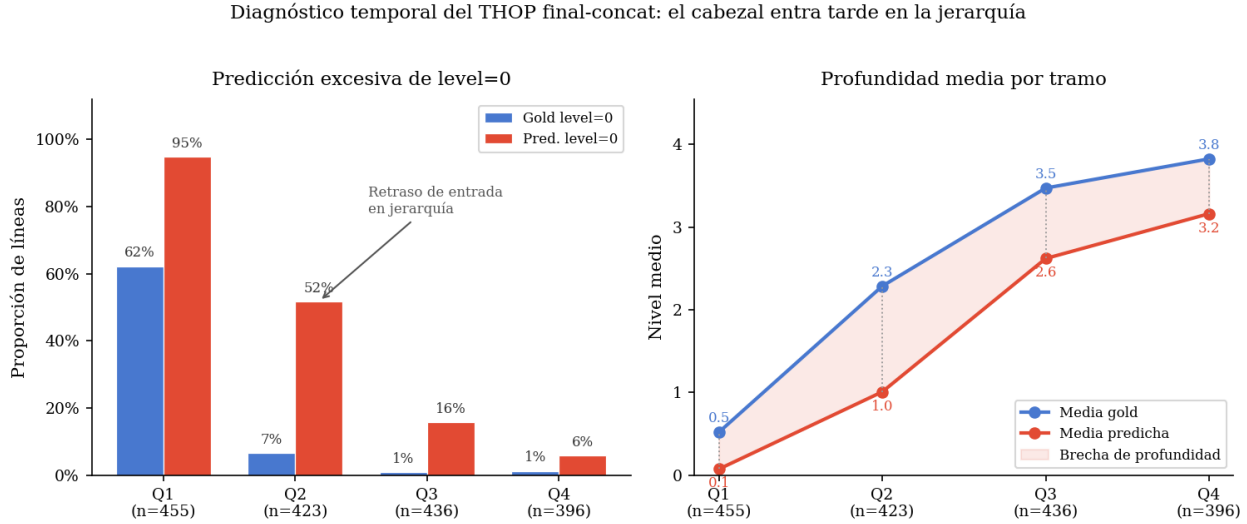


Figura 6.10: Diagnóstico por cuartiles del experimento THOP con concatenación final. La figura muestra, a la izquierda, el exceso de predicciones `level=0`; a la derecha, la brecha entre la profundidad media de la referencia y la profundidad media predicha.

Lo importante, sin embargo, es que el cabezal posicional no fue simplemente ignorado. El análisis del proyector final mostró que la rama posicional tenía un peso no trivial: la norma asociada a sus dimensiones representaba aproximadamente un 47 % de la norma de la rama latente. Además, la correlación entre la posición de línea y el score ordinal fue alta ($r = 0,8460$). Con estos resultados vemos que la posición sí estaba afectando al score, pero lo hacía principalmente como una rampa temporal añadida al valor ordinal. Dado que la interacción se produce en la última capa lineal, el modelo solo puede expresar una forma del tipo:

$$z_t = w_h^\top u_t + w_p^\top p_t + b.$$

En esta formulación, la posición corrige el score mediante un sesgo aditivo, pero no modifica la manera en que se interpreta el estado latente h_t . Y precisamente ahí parece estar la limitación. La jerarquía YAML no es una función suave del índice de línea: una misma posición relativa puede contener una entrada en `metadata`, un descenso hacia `spec.template`, un elemento de lista o una vuelta a un nivel menos profundo. Por eso, una señal posicional tardía puede aprender que, en promedio, las líneas posteriores son más profundas, pero no necesariamente ayudar al cabezal a distinguir las transiciones estructurales locales.

Este primer experimento debe entenderse, por tanto, como un resultado útil dentro de la rama ordinal, no como una mejora respecto al THM v1. La señal posicional empieza a mover algunas métricas parser-facing frente al THO centrado, pero el diagnóstico posterior muestra

que la forma de interacción entre posición y latente no es todavía la adecuada para recuperar la estabilidad que tenía la formulación categórica inicial ni para resolver el problema de fondo.

6.6.2 Segundo experimento posicional: modulación FiLM/gating

El segundo experimento posicional parte de esa lectura. Si la concatenación final solo permite sumar un sesgo temporal al score z_t , la posición llega demasiado tarde: no puede alterar la forma en que el cabezal interpreta las características internas del estado oculto. La alternativa es usar la posición como una condición que module las características antes de la proyección ordinal final. Esta idea se apoya en mecanismos de transformación *feature-wise*, especialmente FiLM, donde una señal condicionante genera parámetros afines para reescalar y desplazar activaciones intermedias [28]. La intuición de *gating* es cercana: permitir que una señal auxiliar abra, cierre o atenúe canales de representación en lugar de limitarse a sumarse al resultado final, una familia de mecanismos ya utilizada en modelos neuronales secuenciales [10].

En la variante THOP-FiLM, la posición de línea sigue siendo causal y absoluta, igual que en THOP. La diferencia está en el lugar de inyección. Primero, el estado oculto se proyecta a un vector intermedio de 512 dimensiones. Después, la codificación posicional sinusoidal genera dos vectores, γ_t y β_t , que modulan esas 512 características antes de pasar al cuello de 64 dimensiones y al score ordinal. De forma esquemática:

$$\begin{aligned} r_t &= f_{\text{pre}}(h_t) \in \mathbb{R}^{512}, \\ (\gamma_t, \beta_t) &= g(p_t), \\ \tilde{r}_t &= r_t \odot (1 + \alpha \tanh(\gamma_t)) + \alpha \beta_t, \\ z_t &= f_{\text{post}}(\tilde{r}_t). \end{aligned}$$

La inicialización también es importante. La última capa del generador FiLM se inicializa a cero, de modo que el modelo empieza cerca de una modulación identidad: al comienzo, $\tilde{r}_t \approx r_t$. Esto evita que la nueva rama posicional destruya desde el primer paso la representación ya aprendida, pero deja espacio para que, durante el entrenamiento, la posición aprenda a reponderar dimensiones concretas del estado latente.

El experimento final con esta variante se evaluó sobre las 70 muestras de validación. A diferencia del THOP por concatenación final, la salida estructurada fue parseable en todos los casos, y la tasa de parseo YAML aumentó hasta el 28,57 %. Esta mejora es real respecto a las variantes ordinal-posicionales inmediatamente anteriores, pero queda lejos del 48,57 % alcanzado por el THM v1. La Tabla 6.6 recoge las métricas principales del experimento.

La diferencia entre las dos variantes posicionales ayuda a sacar conclusiones. En la concatenación final, la posición actúa como un sesgo añadido al score ordinal. En THOP-FiLM, en cambio, la posición modula las características intermedias antes de que el score se calcule. Esta segunda forma de interacción parece más adecuada que la concatenación tardía, porque no fuerza a la posición a competir en la última capa con todo el estado latente, sino que le permite alterar la forma en que ese estado se interpreta. Aun así, el resultado no debe leerse como una arquitectura ganadora: THOP-FiLM sigue degradando de forma severa el contenido, la exactitud de nivel y la parseabilidad global frente al THM v1. La conclusión

Tabla 6.6: Resultado final del experimento THOP-FiLM con modulación posicional sobre el cuello de 512 dimensiones.

Métrica	Valor
Predicciones de validación generadas	70 / 70
Ejemplos evaluados	70
Parseo de la salida estructurada	100,00 %
Parseo YAML	28,57 %
Exact match medio de nivel	2,78 %
MAE medio de nivel	0,843
Recall exacto en niveles 5–8	12,14 %
Recall <i>off-by-one</i> en niveles 5–8	41,43 %
Compresión de niveles 5–8 a 0–4	74,29 %
F1 medio de línea	7,82 %
F1 medio de requisitos del prompt	7,99 %
KDV score medio	12,50 %
KDV gate pass	0,00 %

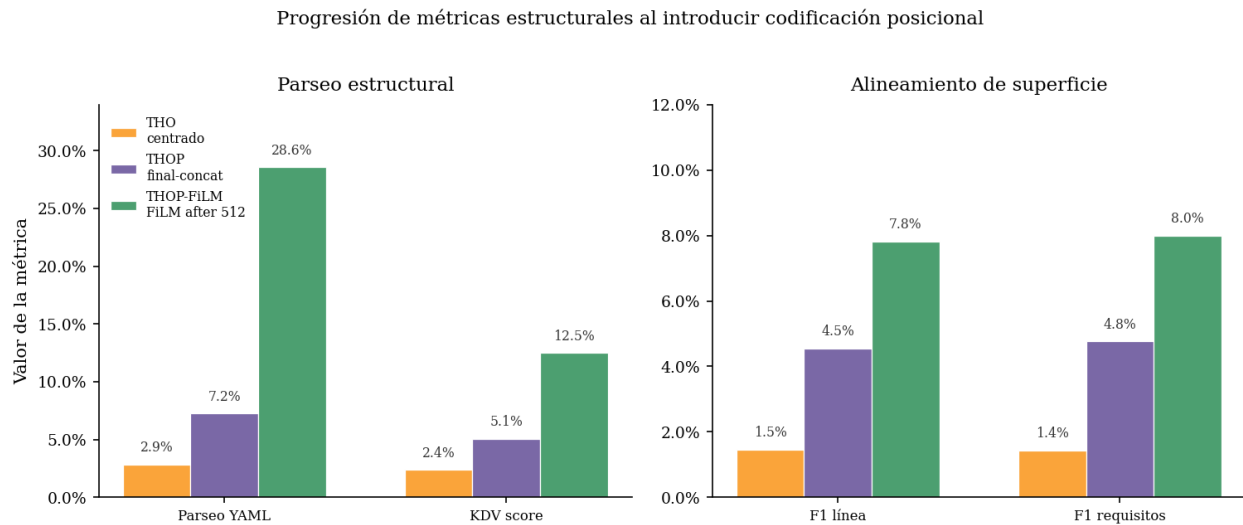


Figura 6.11: Progresión de métricas estructurales al introducir codificación posicional en el cabezal ordinal. El salto de THOP-FiLM es visible dentro de la rama ordinal-posicional, aunque sigue lejos del THM v1 en las métricas globales de validación.

más prudente es que la posición sí contiene información útil para leer el estado latente, pero que incorporarla dentro de una formulación ordinal ya inestable no basta para recuperar un generador robusto. Una línea natural para trabajo futuro sería volver al cabezal categórico del THM v1, que fue el mejor punto de la rama two-head en validación, y estudiar si un condicionamiento tipo FiLM puede ayudarlo a corregir su fallo específico en niveles profundos sin introducir el cuello de botella de un score ordinal único.

Capítulo 7

Alineación Mediante Optimización de Preferencias Directa

El capítulo anterior mostró que las variantes con cabezal explícito de nivel son metodológicamente interesantes, pero no constituyen todavía la base más estable para una fase posterior de alineación. Por ese motivo, la etapa de DPO se aplica sobre el mejor modelo operativo disponible: `serialized_sft`. La pregunta cambia ligeramente respecto a los capítulos anteriores. Ya no se trata de comprobar si el modelo puede aprender la superficie estructurada básica, sino de estudiar si un modelo que ya genera `blocks_tsv_v1` de forma competente puede refinarse mediante preferencias automáticas sin perder la estabilidad estructural que lo hace útil.

DPO (*Direct Preference Optimization*) permite ajustar una política a partir de pares de respuestas preferidas y rechazadas sin entrenar un modelo de recompensa separado ni ejecutar una fase de refuerzo *online* [33]. En este trabajo, esas preferencias no proceden de juicios humanos, sino de señales automáticas derivadas del parser, de la adecuación al prompt y de los checks de dominio Kubernetes. Precisamente por eso, antes de hablar del etiquetado de pares o del entrenamiento DPO, es necesario justificar cómo se generó el conjunto de candidatos sobre el que se construyeron esas preferencias.

7.1 Generación de candidatos para DPO

La construcción de un dataset de preferencias no empieza con el etiquetado, sino con una decisión más básica: cuántas salidas generar para cada prompt y con qué nivel de diversidad. Si el modelo produce candidatos demasiado parecidos, la comparación entre salidas apenas aporta señal. Si, por el contrario, la decodificación es demasiado agresiva, los pares resultantes pueden quedar dominados por errores triviales de formato o de parseabilidad. El objetivo de esta fase fue encontrar un punto intermedio: generar suficientes variantes por prompt para que aparezcan contrastes útiles, pero sin salir de la distribución aprendida por `serialized_sft`.

Para ello se probaron varias estrategias de muestreo sobre el split de entrenamiento. La primera familia generaba cuatro candidatos por prompt con temperaturas moderadas. La

segunda ampliaba a seis candidatos, pero mantenía un rango bajo-medio de temperatura. La tercera, que acabó siendo la base del conjunto final, también generaba seis candidatos, pero cubría un rango más amplio: 0,45, 0,65, 0,85, 1,0, 1,1 y 1,2, con `top_p=0.95`. La Figura 7.1 resume la comparación entre estas estrategias.

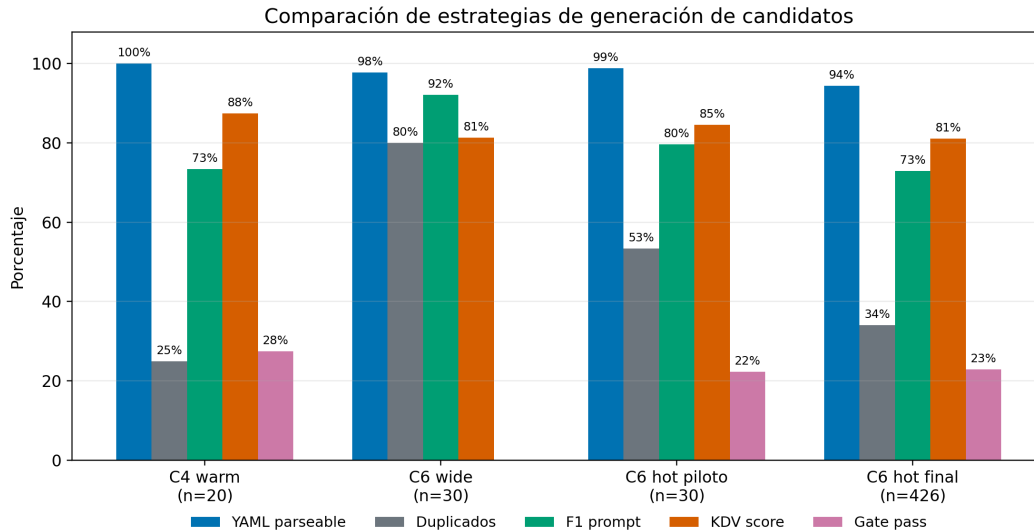


Figura 7.1: Comparación de estrategias de generación de candidatos para DPO. Los pilotos se ejecutaron sobre subconjuntos distintos, por lo que deben leerse como una exploración metodológica y no como una comparación controlada exacta. La fila **C6 hot final** agrega las 426 unidades de prompt usadas en la generación principal.

La lectura principal no es que una estrategia maximice todas las métricas a la vez, sino que cada una ofrece un equilibrio distinto. El piloto **C6 wide** conserva una adecuación al prompt muy alta (92%), pero genera una tasa de duplicados también elevada. El rango **C6 hot**, en cambio, reduce la fidelidad media al prompt, pero introduce más variabilidad y recupera candidatos que sí pasan el *gate* Kubernetes. Esta diferencia es importante porque DPO no necesita únicamente buenas salidas; necesita pares comparables donde una salida sea preferible a otra por razones reconocibles.

El análisis por temperatura confirma esa tensión. Como muestra la Figura 7.2, las temperaturas bajas mantienen mejor parseabilidad, mejor F1 de prompt y mayor score preferencial medio. A 0,45 y 0,65, la tasa de YAML parseable se mantiene cerca del 99%, mientras que a 1,2 cae al 87% y aumenta la proporción de candidatos inválidos. Sin embargo, las temperaturas altas no se descartan por completo: su utilidad no está en producir el mejor candidato medio, sino en generar rechazados informativos cuando el resto de salidas del mismo prompt son estructuralmente válidas.

Esta interpretación se ve con más claridad al observar qué temperatura produce el mejor y el peor candidato dentro de cada prompt. En la Figura 7.3, la temperatura 0,45 genera aproximadamente la mitad de los mejores candidatos, mientras que 1,2 aparece como peor

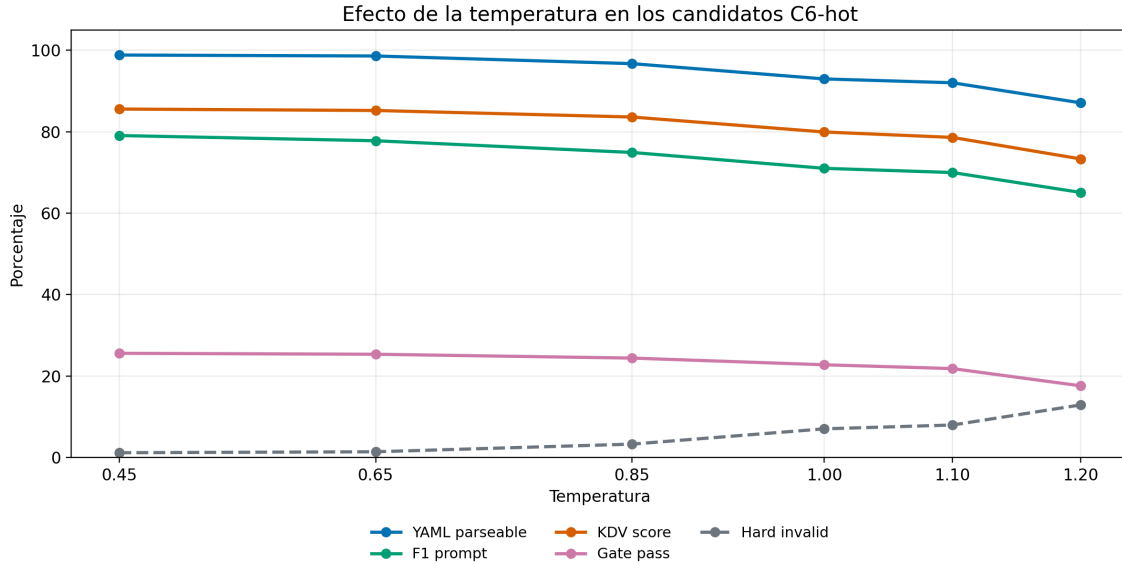


Figura 7.2: Efecto de la temperatura en los candidatos generados con la configuración C6 hot. Las temperaturas bajas conservan mejor la calidad media; las temperaturas altas incrementan el riesgo de candidatos inválidos y reducen la adecuación al prompt.

candidata en casi la mitad de los prompts. La configuración final no busca que todas las temperaturas sean igual de buenas, sino que el conjunto de seis candidatos contenga una mezcla razonable de respuestas fuertes, respuestas aceptables y errores útiles para construir contrastes.

Antes de calcular esos contrastes, cada candidata recibe un score preferencial auxiliar. Para una salida candidata y generada ante un prompt x , el score usado en esta fase se define como:

$$s(y | x) = 1,00 \cdot \text{PRF1} + 0,75 \cdot \text{KDV} + 0,50 \cdot \text{RFC} + 0,25 \cdot \text{LevelAcc} + 0,25 \cdot \text{Gate} - \text{penalizaciones.}$$

En esta expresión, PRF1 mide la adecuación al prompt, KDV resume la validez de dominio Kubernetes, RFC mide la completitud de campos obligatorios, LevelAcc recoge la coincidencia exacta del nivel jerárquico y Gate indica si la candidata supera la compuerta completa de dominio. Las penalizaciones recogen casos simples pero relevantes, como recursos inventados o diferencias severas en el número de líneas. La ponderación da prioridad a la adecuación al prompt y después a la validez de dominio, manteniendo la estructura como señal de retención.

El último criterio fue comprobar si esa diversidad se traducía realmente en pares aprovechables para DPO. Para cada prompt se calculó la diferencia entre el mejor y el peor score preferencial disponible. La Figura 7.4 muestra que 282 de las 426 unidades de prompt alcanzan un margen de al menos 0,25, que es el umbral principal utilizado para conservar pares con una diferencia suficientemente clara. Otros 21 prompts quedan en el intervalo exploratorio entre 0,15 y 0,25, mientras que 123 no producen contraste suficiente.

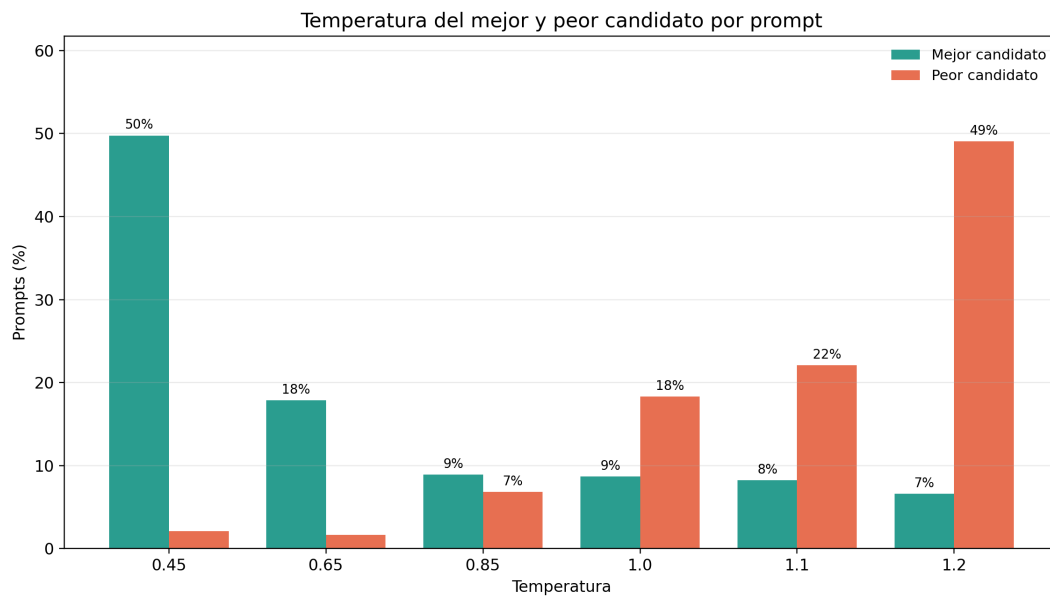


Figura 7.3: Temperatura que produce el mejor y el peor candidato por prompt en la generación principal C6 hot. La distribución muestra que las temperaturas bajas dominan entre los mejores candidatos, mientras que las más altas aportan una parte importante de los peores casos.

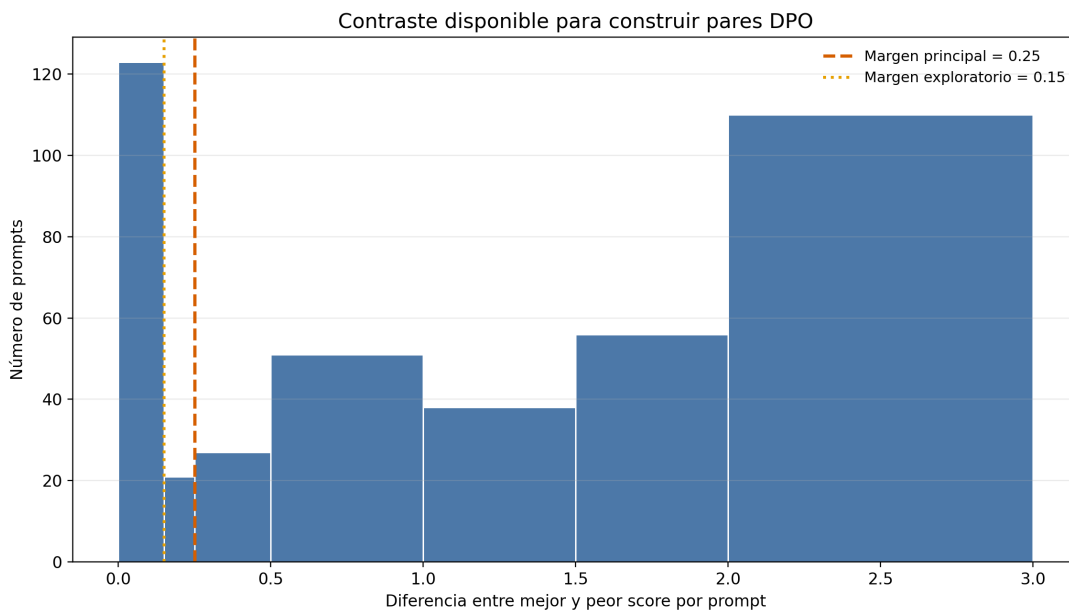


Figura 7.4: Distribución del margen entre el mejor y el peor candidato por prompt. El umbral principal de 0,25 prioriza pares con una diferencia clara, reduciendo el riesgo de entrenar DPO con preferencias ambiguas.

Con estos resultados se adoptó la estrategia **C6 hot** para la generación principal de candidatos. La decisión no se basa en que sea la configuración con mejor calidad media, sino en que ofrece una combinación más útil para una etapa de preferencias: mantiene una tasa alta de parseabilidad, reduce duplicados respecto al rango **wide** y genera suficientes diferencias entre candidatos como para seleccionar pares **chosen/rejected**.

Capítulo 8

Impacto Social y Ética

El propósito central de este trabajo es técnico: estudiar si una representación explícita de la jerarquía estructural mejora la generación de manifiestos Kubernetes respecto a enfoques puramente secuenciales. Sin embargo, el objeto generado no es un texto neutro. Un manifiesto YAML puede definir permisos, límites de recursos, políticas de ejecución, relaciones entre servicios y condiciones de exposición de una aplicación. Por eso, incluso en un prototipo académico, la evaluación del sistema no puede terminar en la parseabilidad ni en la similitud con el corpus de entrenamiento: también debe preguntarse qué tipo de errores podría facilitar si se usara como apoyo en un entorno real.

Este capítulo resume ese análisis desde un enfoque de *Ethics by Design* [12], apoyado en las Directrices Éticas para una IA Fiable del Grupo de Expertos de Alto Nivel [19], el Reglamento de Inteligencia Artificial de la Unión Europea [14] y el RGPD [13]. El detalle tabular de la clasificación regulatoria, los factores RGPD, la checklist ética y la matriz de riesgos se recoge en el anexo Evaluación ética y matriz de riesgos. En el cuerpo principal se conserva únicamente el razonamiento necesario para interpretar el sistema dentro del hilo de la memoria.

8.1 Naturaleza y alcance del sistema

El sistema propuesto transforma instrucciones en lenguaje natural en manifiestos YAML de Kubernetes. Esta delimitación es importante porque reduce de forma notable el espacio de riesgo respecto a un modelo conversacional de propósito general: no se diseña para aconsejar sobre personas, tomar decisiones administrativas, priorizar candidatos ni intervenir en servicios esenciales. Su salida esperada es siempre un artefacto técnico de infraestructura, con una gramática y un vocabulario de dominio relativamente acotados.

Precisamente por esa acotación, los riesgos más relevantes no son los habituales en sistemas de IA aplicados a personas, como discriminación directa, manipulación del usuario o perfilado individual. El problema ético principal aparece en otro punto: una configuración técnica generada por el modelo puede parecer correcta, superar validaciones sintácticas y aun así contener prácticas inseguras. En otras palabras, el riesgo no nace de que el sistema actúe autónomamente sobre individuos, sino de que facilite una confianza excesiva en un artefacto

técnico que requiere revisión experta.

En el escenario analizado, el proveedor sería el equipo responsable del diseño, entrenamiento, documentación y evaluación del sistema. El usuario final sería un ingeniero o técnico de infraestructura que introduce el prompt, revisa el resultado y decide si lo despliega. Los afectados indirectos serían los usuarios de los servicios que se ejecutasen sobre una infraestructura configurada con ayuda del sistema. Esta separación de responsabilidades marca el resto del capítulo: el modelo puede asistir en la generación, pero no debe desplazar el juicio técnico ni la responsabilidad operativa del usuario experto.

8.2 Encaje regulatorio y jurídico

Desde el punto de vista del AI Act, el sistema se interpreta como una herramienta generativa de dominio específico con riesgo limitado. No encaja en los supuestos de riesgo inaceptable ni en los ámbitos de alto riesgo del Anexo III: no evalúa personas, no decide sobre empleo, educación, justicia, migración, acceso a servicios esenciales ni derechos fundamentales. Tampoco se presenta como modelo de propósito general, ya que su funcionalidad queda restringida a la producción de configuraciones Kubernetes. Esta lectura no elimina las obligaciones de transparencia y documentación, pero sí desplaza el foco hacia la supervisión humana y el uso responsable.

La supervisión humana no es solo una exigencia formal. Los resultados del Capítulo 5 muestran que el modelo puede alcanzar una tasa de parseabilidad YAML del 98,57 % tras el ajuste fino serializado, mientras que solo el 14,29 % de las muestras supera la compuerta completa de validez de dominio (Tabla 5.3). Ese contraste resume el riesgo central del sistema: la salida puede estar bien formada como YAML y seguir siendo inadecuada para producción. Por tanto, cualquier uso práctico tendría que comunicar de forma explícita que el resultado es una propuesta técnica que requiere validación, no una configuración lista para desplegarse.

La exposición al RGPD es reducida en el prototipo actual. El ajuste fino se realiza sobre **CloudEval-YAML** [1], un dataset técnico de pares prompt - manifiesto YAML publicado bajo licencia MIT, sin metadatos de desarrollo, nombres de usuario, correos electrónicos ni registros de interacción con personas. Además, el prototipo no almacena consultas de usuarios durante inferencia. Si en una versión posterior se registrasen prompts para mejorar el modelo, esa fase sí exigiría una base jurídica específica, criterios de minimización, mecanismos de seudonimización y una política clara de exclusión u *opt-out*. En el estado actual del trabajo, ese tratamiento no forma parte del sistema.

El análisis de propiedad intelectual apunta en la misma dirección: los dos componentes principales tienen licencias permisivas. El modelo base Qwen2.5-7B-Instruct [31] se distribuye bajo Apache 2.0 y el dataset de ajuste bajo MIT. El riesgo residual de memorización existe, como en cualquier ajuste generativo, pero queda limitado por el tamaño del corpus y por el carácter técnico y estructurado de las muestras. Jurídicamente, el sistema no puede atribuirse autoría sobre los manifiestos generados; la responsabilidad sobre su uso, revisión y eventual despliegue recae en el operador que decide incorporarlos a una infraestructura real.

8.3 Riesgos principales

El primer riesgo es la generación de configuraciones inseguras. El análisis de errores del `serialized_sft` identifica 470 antipatrones de seguridad en 70 muestras de validación: contenedores sin límites de CPU o memoria, ejecución como `root`, sistemas de ficheros raíz con escritura habilitada e imágenes con etiqueta `:latest` (Tabla 5.5). Estos errores son especialmente relevantes porque no impiden necesariamente el parseo del manifiesto. Un YAML puede ser válido, coherente con el prompt e incluso funcional, pero incorporar decisiones que aumentan la superficie de ataque o reducen la reproducibilidad del despliegue.

El segundo riesgo es la dependencia técnica. Una herramienta que convierte lenguaje natural en configuración puede reducir trabajo repetitivo, pero también puede desplazar parte del esfuerzo de comprensión hacia una revisión superficial del resultado. En ese escenario, el usuario no pierde formalmente el control, pero sí puede delegar demasiado pronto en una salida que parece ordenada y profesional. La frontera ética del sistema está precisamente ahí: debe diseñarse como asistente para acelerar la redacción y la exploración de manifiestos, no como sustituto del criterio de un operador con conocimiento de Kubernetes.

El tercer riesgo procede de la cobertura limitada del dataset. El corpus de entrenamiento tiene 213 muestras base, ampliadas mediante variantes de prompt, y reproduce los patrones presentes en `CloudEval-YAML`. Esto introduce sesgos técnicos, no sociales: sobre-representación de ciertos recursos, tecnologías o estilos de configuración, y posible falta de señal en prácticas de seguridad que no aparecen de forma sistemática en los datos. En este sentido, la caída en la compuerta de seguridad no se interpreta como un simple defecto de entrenamiento, sino como una limitación de la señal supervisada disponible.

Por último, existen riesgos residuales de transparencia y trazabilidad. Los modelos basados en transformers no ofrecen una explicación completa de su proceso interno, aunque el trabajo aporta evidencia indirecta mediante análisis de representaciones latentes: un modelo lineal alcanza un 85,94 % de precisión al predecir el nivel de indentación a partir de estados ocultos. Ese resultado ayuda a interpretar la presencia de información estructural en el modelo, pero no convierte sus salidas en explicables ni verificadas por construcción. La trazabilidad debe apoyarse, por tanto, en registros de inferencia, métricas de validación y documentación del alcance del sistema.

8.4 Medidas de mitigación y responsabilidad

La medida de mitigación más importante es mantener una frontera explícita entre generación y despliegue. El sistema no ejecuta comandos, no aplica manifiestos sobre un clúster y no dispone de una ruta automática hacia producción. Su salida es un fichero de texto estructurado que debe pasar por validación sintáctica, revisión de dominio y revisión humana antes de cualquier uso operativo. Esta separación no elimina los errores, pero evita que un fallo del modelo se convierta directamente en un fallo de infraestructura.

La segunda medida es comunicar las limitaciones del sistema de forma concreta. No basta con un aviso genérico de que el contenido ha sido generado por IA. En este caso, la documentación

debe explicar que la parseabilidad YAML no equivale a seguridad, que el parser actúa como control estructural y que las métricas de dominio usadas en la memoria son aproximadas. Esta distinción es coherente con el argumento general del trabajo: producir una estructura válida en superficie es necesario, pero no garantiza que el manifiesto sea adecuado para un despliegue real.

Finalmente, la responsabilidad debe permanecer distribuida de forma clara. El proveedor es responsable de documentar el alcance, las limitaciones, los datos utilizados, las métricas de validación y los riesgos conocidos. El usuario experto es responsable de revisar, adaptar y validar los manifiestos antes de desplegarlos. Esta división no pretende trasladar todo el peso al usuario, sino evitar una ambigüedad peligrosa: el sistema puede ayudar a escribir configuraciones, pero no puede asumir la responsabilidad operativa de una infraestructura.

8.5 Resumen de impacto

El impacto social del sistema es limitado y, en un escenario bien controlado, potencialmente positivo: puede reducir esfuerzo en tareas repetitivas de configuración y ayudar a explorar soluciones estructuradas a partir de descripciones en lenguaje natural. Sin embargo, ese valor solo aparece si el sistema se mantiene dentro de su papel de asistencia. El resultado principal del análisis ético no es que la herramienta sea peligrosa por naturaleza, sino que su apariencia de corrección puede ser más convincente que su seguridad real.

En este sentido, el capítulo refuerza la tesis técnica de la memoria. La generación estructurada no se evalúa únicamente porque produzca YAML más limpio, sino porque permite separar mejor los niveles de validez: sintaxis, jerarquía, reconstrucción parser-facing, dominio y adecuación al prompt. Esa separación también es una medida ética. Hace visible dónde termina el control automático y dónde empieza la responsabilidad humana, que es precisamente la frontera que debe respetarse si el sistema llega a usarse fuera del contexto experimental.

Capítulo 9

Análisis Comparativo de Resultados

Los capítulos anteriores han presentado los experimentos de forma progresiva: primero la línea base y el ajuste supervisado serializado, después las variantes con predicción explícita de nivel y, finalmente, la construcción de preferencias para la etapa de DPO. Este capítulo reúne esos resultados desde una perspectiva comparativa.

Como ya he introducido, el problema estudiado no consiste únicamente en producir texto parecido a YAML, sino en generar una representación intermedia que pueda reconstruirse de forma determinista como un manifiesto Kubernetes. Por tanto, las métricas más informativas son aquellas que atraviesan esa frontera: la parseabilidad de la salida estructurada, la parseabilidad del YAML reconstruido, la precisión de la variable `level`, la adecuación al prompt y las señales aproximadas de validez de dominio. Las métricas de similitud textual o de igualdad exacta con la referencia se conservan como contexto, pero no constituyen el criterio principal de éxito.

9.1 Criterios de comparación

La comparación se organiza alrededor de tres preguntas. La primera es si el ajuste supervisado sobre una superficie estructurada mejora de forma clara la generación zero-shot. Esta pregunta corresponde a la transición entre la línea base y `serialized_sft`. La segunda es si la jerarquía debe permanecer serializada como texto o si conviene predecirla mediante una cabeza estructural explícita. Esta es la comparación central entre `serialized_sft` y las variantes `two_head`. La tercera pregunta aparece después del SFT: si un modelo ya estable en la superficie de bloques puede refinarse mediante preferencias sin perder la regularidad estructural aprendida.

La Tabla 9.1 resume las métricas principales disponibles en validación para las familias más representativas.

La tabla muestra una primera lectura bastante nítida: el salto más grande del proyecto ocurre entre la línea base y el SFT serializado. Dentro de la rama con predicción explícita de nivel, el mejor punto global sigue siendo el THM v1. Las variantes ordinales y posicionales posteriores no deben entenderse como mejoras sucesivas de rendimiento, sino como intentos de explicar y

Tabla 9.1: Resumen comparativo en validación de los modelos principales. Los valores de DPO corresponden a la run **beta=0.10**; la variante DPO v2 se mantiene fuera de la tabla hasta completar su validación final.

Métrica	Baseline	SFT	THM v1	THOP-FiLM	DPO 0.10
yaml_parse_success_rate	30,77	98,57	48,57	28,57	97,14
average_line_text_f1	20,86	82,06	37,43	7,82	80,92
average_level_exact_match	11,31	75,78	35,04	2,78	74,22
average_level_mae	0,7896	0,2723	0,2392	0,8430	0,2630
average_prompt_requirement_f1	22,13	85,31	42,06	7,99	83,68
average_semantic_key_f1	27,78	95,52	45,92	10,13	94,06
kubernetes_domain_score	–	83,10	42,38	12,50	82,86
kubernetes_gate_pass_rate	–	14,29	12,86	0,00	12,86

corregir el fallo específico de esa primera arquitectura en los niveles profundos. DPO, por su parte, conserva buena parte del comportamiento del SFT, pero no lo mejora de forma global en su primera configuración completa.

9.2 De la línea base al SFT serializado

La línea base zero-shot ya permite observar una separación que recorre toda la memoria: seguir un formato textual no equivale a mantener una estructura jerárquica válida. En validación, el baseline produce una salida estructurada parseable en la mayoría de los casos, pero solo el 30,77 % de las muestras se reconstruye como YAML válido. La diferencia entre ambos valores indica que el modelo entiende en parte la instrucción de formato, pero no conserva de forma robusta la relación entre niveles, listas y mappings a lo largo de toda la secuencia.

El `serialized_sft` cambia esa situación de manera sustancial. Al entrenar el modelo directamente sobre `blocks_tsv_v1`, la jerarquía se incorpora a la superficie generada mediante la columna `level`. A primera vista, esta decisión podría parecer poco sofisticada: el nivel sigue siendo texto y no una variable estructural separada. Sin embargo, los resultados del Capítulo 5 muestran que esta formulación es muy efectiva. La tasa de parseabilidad YAML sube hasta el 98,57 %, la coincidencia exacta de nivel alcanza el 75,78 % y las métricas de prompt y claves semánticas mejoran de forma igualmente marcada.

La lectura relevante no es solo que el SFT mejore al baseline, algo esperable en una tarea supervisada, sino que la representación elegida encaja bien con el parser. El modelo aprende a emitir una superficie que contiene suficiente información estructural para reconstruir el YAML casi siempre. Dicho de otro modo, el SFT serializado no resuelve la jerarquía porque entienda formalmente el árbol YAML, sino porque aprende una convención textual que hace visible esa jerarquía para el módulo determinista posterior. Esa combinación entre superficie aprendida y parser explica por qué `serialized_sft` se convierte en la referencia fuerte del resto del trabajo.

También conviene conservar una limitación de esta etapa. El buen resultado estructural no implica corrección Kubernetes completa. Como se discutió en el Capítulo 5, el

`kubernetes_domain_gate_pass_rate` permanece en el 14,29 %, principalmente por anti-patrones de seguridad y buenas prácticas que no impiden el parseo YAML. Por tanto, el SFT serializado demuestra que la superficie estructurada es aprendible, pero no agota la evaluación de dominio.

9.3 Jerarquía serializada frente a jerarquía explícita

La comparación entre `serialized_sft` y `two_head_sft` es la comparación conceptual más importante del trabajo. En la primera formulación, el nivel se genera como un campo textual ordinario dentro de la secuencia. En la segunda, el texto de la línea y la profundidad jerárquica se separan: el modelo genera `content_blocks_v1` y una cabeza adicional predice `level` a partir de estados latentes del transformer. Esta separación responde directamente a la hipótesis de que la jerarquía podría modelarse mejor como una señal estructural explícita.

La comparación global en validación favorece claramente al SFT serializado. Frente al 98,57 % de YAML parseable de `serialized_sft`, el primer `two_head_sft` alcanza el 48,57 %. También cae el F1 de texto de línea, la coincidencia exacta de nivel, la adecuación al prompt y la cobertura semántica aproximada. Si se considerase únicamente esta tabla agregada, la lectura sería sencilla: la serialización de `level` como texto funciona mejor que la cabeza explícita.

Sin embargo, esa lectura sería incompleta. El análisis condicionado a salidas parseables muestra que una parte importante de la caída global procede del cuello de botella `parser-facing`. Cuando `two_head_sft` sí produce YAML parseable, sus métricas se acercan mucho más a las de `serialized_sft`. En los mismos 34 ejemplos donde la rama `two-head` cruza el parser, el F1 de texto queda relativamente próximo, la métrica de claves semánticas es prácticamente equivalente y el error absoluto medio de nivel favorece ligeramente al modelo con cabeza explícita. Esta vista no invalida la comparación global, porque las salidas no parseables siguen siendo fallos reales, pero matiza qué tipo de fallo se está observando.

El problema principal de `two_head_sft` no es que genere siempre contenido irreconocible, sino que la frontera entre contenido generado, nivel predicho y reconstrucción determinista se vuelve frágil. Los errores se concentran en listas y mappings anidados, especialmente en estructuras con `containers`, `command`, `volumeMounts` y `volumes`. En esos casos, desplazar una línea uno o dos niveles no produce solo un error local, sino que puede hacer que el parser interprete una lista como un mapping, un campo como un elemento de secuencia o un fragmento de comando como una clave YAML mal formada.

Esta diferencia permite formular una lectura más justa de la comparación principal. Como sistema final, `serialized_sft` es superior en el estado actual del trabajo. Como experimento de investigación, `two_head_sft` revela algo que el SFT serializado ocultaba: separar explícitamente la jerarquía del texto no basta si el estado latente elegido no contiene una señal suficientemente estable para los niveles profundos y para los cambios locales de estructura.

9.4 Evolución de la rama two-head

Los experimentos posteriores de la rama two-head pueden leerse como intentos de explicar y corregir esa fragilidad, no como una sustitución directa del THM v1. El primer resultado relevante fue que una clasificación categórica estándar de `level` ofrecía el mejor rendimiento global de la familia, pero tendía a refugiarse en los niveles más frecuentes. Esto era coherente con la distribución del dataset, donde los niveles 0 a 4 concentran la mayor parte de las líneas, pero era insuficiente para Kubernetes: muchas estructuras con listas internas requieren niveles 5, 6, 7 u 8.

La reformulación ordinal introdujo una hipótesis más adecuada: los niveles no son clases independientes, sino valores ordenados. Esta decisión permitió por primera vez activar niveles profundos en algunas variantes, pero también hizo visible otro problema. La trayectoria del score ordinal no dependía solo de la estructura YAML, sino también del momento de la generación. El modelo parecía aprender una relación fuerte entre posición temporal y profundidad, una relación parcialmente útil porque los documentos suelen comenzar por niveles superficiales y profundizar más adelante, pero insuficiente para representar la jerarquía real.

El análisis por cuartiles del YAML hizo especialmente visible este fenómeno. En las primeras partes de la secuencia, la cabeza tendía a predecir niveles demasiado superficiales; en las zonas intermedias y finales aparecía una rampa de profundidad más clara, pero los niveles profundos seguían comprimidos. La hipótesis de deriva posicional latente encaja con ese patrón: el mismo nivel jerárquico no se presenta necesariamente igual en el espacio oculto al principio, en la mitad o al final de la generación.

La variante THOP-FiLM se diseñó precisamente para introducir la posición como una señal de condicionamiento más rica. Frente a la concatenación posicional tardía, FiLM modula representaciones intermedias del cabezal ordinal. El resultado mejora la parseabilidad respecto a las variantes ordinal-posicionales anteriores y reduce algunos errores de mapping, pero no resuelve el problema completo: en validación final, el modelo sigue mostrando una comprensión fuerte de niveles profundos y una tasa de parseabilidad muy inferior tanto a la del SFT serializado como a la del THM v1. Por tanto, THOP-FiLM no debe presentarse como una arquitectura ganadora, sino como un diagnóstico adicional: la posición importa, pero puede convertirse en un atajo si no se separa con cuidado de la profundidad estructural.

Esta evolución deja una enseñanza metodológica para el análisis comparativo. La rama two-head no fracasa por una única causa simple. Su comportamiento combina desbalanceo de niveles, calibración ordinal, elección del hidden state, deriva posicional y sensibilidad del parser ante errores locales de indentación. La secuencia experimental sugiere una lectura concreta: el THM v1 era el generador explícito más estable, pero ocultaba un fallo importante en niveles profundos; al atacar ese fallo con umbrales ordinales se hizo visible una nueva dificultad, la no estacionariedad de la señal latente a lo largo de la generación. FiLM ayuda a formular esa dificultad, aunque todavía no la resuelve dentro de la arquitectura ordinal. Esta mezcla explica por qué añadir una cabeza estructural es una intervención conceptualmente atractiva, pero experimentalmente más exigente que serializar la jerarquía como texto.

9.5 Efecto de DPO sobre la referencia serializada

DPO parte de una situación distinta. La pregunta ya no es si el modelo puede aprender una representación estructurada básica, porque `serialized_sft` ya lo ha demostrado. La pregunta es si un modelo que genera bloques estructurados con alta estabilidad puede ser desplazado hacia salidas más adecuadas al prompt y más coherentes con ciertos criterios de dominio mediante preferencias automáticas.

El primer experimento completo, con $\beta = 0,10$, ofrece una respuesta mixta. La parseabilidad YAML baja ligeramente respecto al SFT, del 98,57 % al 97,14 %, y también descienden de forma leve el F1 de texto de línea, la coincidencia exacta de nivel, la adecuación al prompt y las claves semánticas. Al mismo tiempo, DPO mejora algunas señales localizadas: sube el `parsed_equal_rate`, mejora ligeramente el ajuste del número de documentos y reduce ciertos errores de dominio, como `volume_mount_without_volume`. La descomposición por niveles Kubernetes muestra además una mejora en niveles intermedios de validez, aunque el gate final no mejora.

Esta combinación es importante porque muestra el riesgo de una fase de alineamiento automática. DPO no actúa como una mejora monotónica sobre el SFT, sino como una reponderación del comportamiento del modelo. Puede empujar algunas propiedades de dominio en la dirección deseada, pero ese desplazamiento también puede degradar la regularidad superficial que hacía que el SFT fuese tan estable. En una tarea parser-facing, esta regularidad no es secundaria: una pequeña caída de parseabilidad condiciona todas las métricas posteriores.

El experimento con $\beta = 0,30$ no modifica sustancialmente la lectura. Sobre las 42 muestras comparables, las salidas son prácticamente idénticas a las de $\beta = 0,10$ en 41 casos. Esto sugiere que, con el primer dataset de preferencias, aumentar β no proporciona una palanca clara de mejora. El siguiente movimiento metodológico no pasa tanto por seguir elevando β , sino por construir preferencias más discriminativas, especialmente con negativos duros que sean parseables y plausibles.

En el momento de redactar este capítulo, la segunda iteración DPO v2 se mantiene como experimento pendiente de cierre. Su configuración es deliberadamente más agresiva y parte del checkpoint DPO v1, con un conjunto de preferencias más pequeño pero más estricto. Por esta razón, no debe anticiparse su conclusión. En la versión final de este capítulo, DPO v2 debe añadirse solo después de completar su validación, comparándolo al menos con `serialized_sft` y con DPO v1 sobre el mismo protocolo de métricas.

9.6 Evaluación final en test

Hasta este punto, la mayor parte de la discusión se apoya en validación, porque ese split ha servido para desarrollar, comparar y diagnosticar las variantes experimentales. Para cerrar el análisis comparativo sin contaminar el test, la evaluación final debe reservarse a un conjunto reducido de modelos ya seleccionados. La función del test no es elegir hiperparámetros ni decidir entre variantes exploratorias, sino comprobar si la lectura obtenida en validación se mantiene en datos no usados durante el desarrollo.

La Tabla 9.2 recoge la matriz propuesta para esa evaluación final. El baseline ya dispone de una ejecución en test y puede actuar como punto de partida. El SFT serializado debe evaluarse porque es la referencia supervisada fuerte. La rama two-head debe representarse mediante el THM v1, que es el mejor resultado global de esa familia en validación. Las variantes ordinales y posicionales quedan como análisis diagnóstico y como base para trabajo futuro, pero no como candidatas principales a test salvo que se quiera confirmar explícitamente el fallo observado. DPO debe evaluarse solo cuando se haya decidido, usando validación, cuál es el checkpoint final que merece pasar a test.

Tabla 9.2: Matriz de evaluación final en test. Las filas pendientes no deben usarse para selección de modelo, sino como confirmación de modelos previamente fijados en validación.

Modelo	Estado	Función en la comparación
Baseline zero-shot	Ejecutado en test	Punto de partida sin ajuste supervisado
<code>serialized_sft</code>	Pendiente de test	Referencia supervisada principal
THM v1	Pendiente de test	Mejor representante validado de la rama con jerarquía explícita
DPO final	Pendiente de selección y test	Refinamiento post-SFT mediante preferencias automáticas

Las métricas de esta evaluación deberían mantenerse alineadas con el resto del trabajo. En concreto, conviene agruparlas en cuatro bloques:

Bloque	Métricas
Parseabilidad	<code>yaml_parse_success_rate</code> <code>block_parse_success_rate</code>
Fidelidad estructural	<code>average_line_text_f1</code> <code>average_level_exact_match_rate</code> <code>average_level_mae</code>
Adecuación al contenido	<code>average_prompt_requirement_f1</code> <code>average_semantic_key_f1</code>
Dominio Kubernetes	<code>average_kubernetes_domain_validity_score</code> <code>kubernetes_domain_gate_pass_rate</code>

Este conjunto permite comprobar no solo si el modelo genera YAML válido, sino también si conserva la jerarquía, responde al prompt y evita parte de los errores de dominio observados.

9.7 Discusión transversal

Al poner juntas las tres líneas experimentales aparece una lectura más matizada que la que ofrecería cualquier métrica aislada. El SFT serializado es el sistema más fuerte porque convierte una tarea jerárquica en una superficie textual muy regular y fácil de consumir por el parser. Esto no demuestra que el modelo haya aprendido una representación formal del árbol YAML, pero sí demuestra que una serialización explícita de la jerarquía puede ser una estrategia práctica muy eficaz para generación estructurada.

La rama two-head, por el contrario, separa de manera más limpia la pregunta investigadora. Al sacar `level` de la superficie textual y predecirlo con una cabeza estructural, permite

estudiar si el modelo contiene información jerárquica aprovechable en sus estados latentes. El THM v1 muestra que esa separación puede producir un generador razonablemente estable, aunque incapaz de recuperar niveles profundos por el desbalanceo de clases. Las variantes posteriores muestran el reverso del problema: cuando se fuerza al cabezal a entrar en esa zona profunda mediante una formulación ordinal, la calibración de la trayectoria y la posición de la línea se vuelven críticas. La frontera entre generación de contenido, predicción de nivel y reconstrucción parser-facing se convierte así en el punto frágil de la arquitectura.

DPO añade una tercera dimensión a la comparación. En lugar de cambiar la arquitectura, modifica la política aprendida a partir de preferencias automáticas. Su efecto inicial confirma que las señales de dominio pueden mover el comportamiento del modelo, pero también que ese movimiento tiene coste. La mejora de algunas propiedades Kubernetes no compensa todavía las pequeñas pérdidas de parseabilidad y fidelidad al prompt. En este sentido, DPO se interpreta mejor como una herramienta de refinamiento que como una sustitución directa del SFT.

La comparación global refuerza, por tanto, la necesidad de evaluar por niveles. Si solo se observara la parseabilidad YAML, el SFT serializado parecería cerrar el problema. Si solo se observara la predicción de `level`, se perdería el hecho de que una pequeña desviación puede romper todo el documento. Si solo se observaran métricas de dominio, se ignoraría que muchas buenas prácticas Kubernetes dependen de checks más estrictos que la sintaxis. El valor del análisis comparativo está precisamente en mantener juntas estas capas sin confundirlas.

Este capítulo deja así preparada la lectura final del trabajo: los resultados no apuntan a una única técnica dominante en todos los sentidos, sino a una tensión metodológica entre superficie serializada, señal estructural explícita y alineamiento automático. La interpretación de esa tensión, junto con las limitaciones y las líneas de continuidad, se desarrolla en el capítulo siguiente.

Capítulo 10

Conclusiones y Trabajo Futuro

Bibliografía

- [1] Alibaba Cloud Intelligence, 2023. CloudEval-YAML: A practical benchmark for cloud configuration generation. <https://github.com/alibaba/CloudEval-YAML>. MIT License.
- [2] Andor, D., Alberti, C., Weiss, D., Severyn, A., Presta, A., Ganchev, K., Petrov, S., Collins, M., 2016. Globally normalized transition-based neural networks. arXiv preprint arXiv:1603.06042 URL: <https://arxiv.org/abs/1603.06042>.
- [3] Angi, A., Nedoshivina, L., Sacco, A., Braghin, S., Purcell, M., 2025. A perspective on LLM data generation with few-shot examples: from intent to kubernetes manifest, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track), Association for Computational Linguistics. pp. 345–354. URL: <https://aclanthology.org/2025.acl-industry.27/>, doi:10.18653/v1/2025.acl-industry.27.
- [4] Belanger, D., McCallum, A., 2017. Structured prediction energy networks, in: Proceedings of the 34th International Conference on Machine Learning. URL: <https://proceedings.mlr.press/v70/belanger17a.html>.
- [5] Beurer-Kellner, L., Fischer, M., Vechev, M., 2023. Guidance: A programming paradigm for controlling language models. arXiv preprint arXiv:2306.05285 URL: <https://arxiv.org/abs/2306.05285>.
- [6] Bufalino, J., Martin-Navarro, J.L., Di Francesco, M., Aura, T., 2025. Inside job: Defending kubernetes clusters against network misconfigurations. arXiv preprint arXiv:2506.21134 URL: <https://arxiv.org/abs/2506.21134>.
- [7] Bunel, R., Hausknecht, M., Devlin, J., Singh, R., Kohli, P., 2018. Leveraging grammar and reinforcement learning for neural program synthesis, in: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=H1Xw62kRZ>.
- [8] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati,

- M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 URL: <https://arxiv.org/abs/2107.03374>.
- [9] Clark, K., Khandelwal, U., Levy, O., Manning, C.D., 2019. What does bert look at? an analysis of bert’s attention, in: Proceedings of the 2019 ACL Workshop BlackboxNLP. URL: <https://arxiv.org/abs/1906.04341>.
- [10] Dauphin, Y.N., Fan, A., Auli, M., Grangier, D., 2017. Language modeling with gated convolutional networks, in: Proceedings of the 34th International Conference on Machine Learning, pp. 933–941. URL: <https://arxiv.org/abs/1612.08083>.
- [11] Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L., 2023. Qlora: Efficient finetuning of quantized llms, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/2305.14314>.
- [12] European Commission, 2021. Ethics by Design and Ethics of Use Approaches for Artificial Intelligence. Technical Report. Directorate-General for Research and Innovation. URL: <https://op.europa.eu/en/publication-detail/-/publication/d3988569-0f73-11ec-9d68-01aa75ed71a1>.
- [13] European Parliament and Council of the European Union, 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). OJ L 119, 1–88.
- [14] European Parliament and Council of the European Union, 2024. Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act) and amending certain Union legislative acts. OJ L, 2024/1689.
- [15] Fonseca, J., et al., 2024. Mutiny! how does kubernetes fail, and what can we do about it? arXiv preprint arXiv:2404.11169 URL: <https://arxiv.org/abs/2404.11169>.
- [16] Geng, S., Cooper, H., Moskal, M., et al., 2025. Generating structured outputs from language models: Benchmark and studies. arXiv preprint arXiv:2501.10868 URL: <https://arxiv.org/abs/2501.10868>.
- [17] Geng, S., Josifoski, M., Peyrard, M., West, R., 2023. Grammar-constrained decoding for structured nlp tasks without finetuning, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2305.13971>.
- [18] Hewitt, J., Manning, C.D., 2019. A structural probe for finding syntax in word representations, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1906.04284>.
- [19] High-Level Expert Group on Artificial Intelligence, 2019. Ethics Guidelines for Trustworthy AI. Technical Report. European Commission. URL: https://ec.europa.eu/newsroom/dae/document.cfm?doc_id=60419.

-
- [20] Hokamp, C., Liu, Q., 2017. Lexically constrained decoding for sequence generation using grid beam search, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://aclanthology.org/P17-1141/>.
- [21] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2021. Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 URL: <https://arxiv.org/abs/2106.09685>.
- [22] Joachims, T., Hofmann, T., Yue, Y., Yu, C.N., 2009. Predicting structured objects with support vector machines. Communications of the ACM .
- [23] Kakarla, S.K.R., Yan, F.Y., Beckett, R., 2024. Diffy: Data-driven bug finding for configurations. Proceedings of the ACM on Programming Languages 8. URL: <https://doi.org/10.1145/3656385>, doi:10.1145/3656385.
- [24] Kendall, A., Gal, Y., Cipolla, R., 2018. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. URL: <https://arxiv.org/abs/1705.07115>.
- [25] Lee, B., et al., 2024. Structuredrag: Json response formatting with large language models. arXiv preprint arXiv:2408.11061 URL: <https://arxiv.org/abs/2408.11061>.
- [26] Li, M., et al., 2024. Struc-bench: Are large language models really good at generating complex structured data?, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/2309.08963>.
- [27] Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G., 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. ACM Computing Surveys 55. URL: <https://doi.org/10.1145/3560815>, doi:10.1145/3560815.
- [28] Perez, E., Strub, F., de Vries, H., Dumoulin, V., Courville, A., 2018. FiLM: Visual reasoning with a general conditioning layer, in: Proceedings of the AAAI Conference on Artificial Intelligence. URL: <https://arxiv.org/abs/1709.07871>.
- [29] Pimparkhede, S., Kammakomati, M., Tamilselvam, S., Kumar, P., Pon Kumar, A., Bhattacharyya, P., 2024. Doccgen: Document-based controlled code generation, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2406.11925>.
- [30] Pujar, S., Buratti, L., Guo, X., Dupuis, N., Lewis, B., Suneja, S., Sood, A., Nalawade, G., Jones, M., Morari, A., Puri, R., 2023. Automated code generation for information technology tasks in yaml through large language models. arXiv preprint arXiv:2305.02783 URL: <https://arxiv.org/abs/2305.02783>.
- [31] Qwen Team, 2024. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 URL: <https://arxiv.org/abs/2412.15115>.
- [32] Rabinovich, M., Stern, M., Klein, D., 2017. Abstract syntax networks for code generation

- and semantic parsing, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1704.07535>.
- [33] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C., 2023. Direct preference optimization: Your language model is secretly a reward model, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/2305.18290>.
 - [34] Rahman, A., et al., 2023. Security misconfigurations in open source kubernetes manifests: An empirical study. ACM Transactions on Software Engineering and Methodology URL: <https://akondrahman.github.io/files/papers/tosem-k8s.pdf>.
 - [35] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Canton Ferrer, C., Grattafiori, A., Xiong, W., Defossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G., 2023. Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950 URL: <https://arxiv.org/abs/2308.12950>.
 - [36] Scholak, T., Schucher, N., Bahdanau, D., 2021. Picard: Parsing incrementally for constrained auto-regressive decoding from language models, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2109.05093>.
 - [37] Shin, R., Lin, C.H., Thomson, S., Chen, C., Roy, S., Platanios, E.A., Pauls, A., Klein, D., Eisner, J., Van Durme, B., 2021. Constrained language models yield few-shot semantic parsers, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. URL: <https://aclanthology.org/2021.emnlp-main.608/>.
 - [38] Ueno, M., Uchiumi, T., 2024. Migrating existing container workload to kubernetes: LLM based approach and evaluation, in: 2024 IEEE International Conference on Software Maintenance and Evolution, IEEE. pp. 701–706. URL: <https://arxiv.org/abs/2408.11428>, doi:10.1109/ICSME58944.2024.00074.
 - [39] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/1706.03762>.
 - [40] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N.A., Khashabi, D., Hajishirzi, H., 2023. Self-instruct: Aligning language models with self-generated instructions, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/2212.10560>.
 - [41] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2022. Scaling instruction-finetuned language models, in: International Conference on Learning Representations. URL: <https://arxiv.org/abs/2210.11416>.
 - [42] Willard, B.T., Louf, R., 2023. Efficient guided generation for large language models. arXiv preprint arXiv:2307.09702 URL: <https://arxiv.org/abs/2307.09702>.
 - [43] Wu, Y., Zhou, Y., Ziheng, Z., Peng, Y., Ye, X., Hu, X., Zhu, W., Qi, L., Yang, M.H.,

- Yang, X., 2025. On the generalization of sft: A reinforcement learning perspective with reward rectification. OpenReview preprint URL: <https://openreview.net/forum?id=Lv7PjbcaMi>.
- [44] Xu, S., Fu, W., Gao, J., Ye, W., Liu, W., Mei, Z., Wang, G., Yu, C., Wu, Y., 2024a. Is dpo superior to ppo for llm alignment? a comprehensive study, in: Proceedings of the 41st International Conference on Machine Learning. URL: <https://arxiv.org/abs/2404.10719>.
- [45] Xu, Y., Chen, Y., Zhang, X., Lin, X., Hu, P., Ma, Y., Lu, S., Du, W., Mao, Z.M., Zhai, E., Cai, D., 2024b. Cloudeval-yaml: A practical benchmark for cloud native yaml configuration generation, in: Proceedings of the Seventh Annual Conference on Machine Learning and Systems. URL: <https://cloudeval-yaml.github.io/>.
- [46] Yang, J., Jiang, D., He, L., Siu, S., Zhang, Y., Liao, D., Li, Z., Zeng, H., Jia, Y., Wang, H., Schneider, B., Ruan, C., Ma, W., Lyu, Z., Wang, Y., Lu, Y., Do, Q.D., Jiang, Z., Nie, P., Chen, W., 2025. Structeval: Benchmarking llms' capabilities to generate structural outputs. arXiv preprint arXiv:2505.20139 URL: <https://arxiv.org/abs/2505.20139>.
- [47] Yin, P., Neubig, G., 2017. A syntactic neural model for general-purpose code generation, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1704.01696>.
- [48] Zhang, Y., Meredith, R., Reaves, W., Coriolano, J., Babar, M.A., Rahman, A., 2024. Does generative AI generate smells related to container orchestration?: An exploratory study with kubernetes manifests, in: Proceedings of the 21st International Conference on Mining Software Repositories, ACM. pp. 192–196. URL: <https://akondrahman.github.io/files/papers/k8s-msr2024.pdf>, doi:10.1145/3643991.3645079.
- [49] Zhang, Y., Paul, U., d'Amorim, M., Rahman, A., 2025. Configuration defects in kubernetes. arXiv preprint arXiv:2512.05062 URL: <https://arxiv.org/abs/2512.05062>.
- [50] Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.Y., Wen, J.R., 2023. A survey of large language models. arXiv preprint arXiv:2303.18223 URL: <https://arxiv.org/abs/2303.18223>.

Configuración experimental y reproducibilidad

Este anexo recopila los parámetros técnicos necesarios para reproducir los experimentos descritos en la memoria. La información se organiza en cuatro bloques: el entorno de hardware y software, las características del modelo base y su cuantización, la configuración de LoRA compartida entre todas las variantes de ajuste fino, y los hiperparámetros específicos de cada experimento. La motivación de las decisiones más relevantes aparece comentada junto a los valores correspondientes, de modo que el anexo sirva tanto como referencia técnica como como justificación de las elecciones de implementación.

.1 Entorno de hardware y software

Todos los experimentos de entrenamiento, validación e inferencia se realizaron en la misma máquina, sin uso de cómputo en la nube. Esta restricción determinó varias decisiones de diseño: el uso de cuantización de cuatro bits para reducir la huella de memoria del modelo, la elección de un modelo de siete mil millones de parámetros en lugar de variantes mayores, y el tamaño efectivo de batch. El hardware utilizado se detalla en la tabla 1.

Tabla 1: Especificaciones del entorno de hardware.

Componente	Especificación
CPU	Intel Core i7-12700H (12. ^a gen.), 14 núcleos físicos, 20 hilos lógicos
RAM	16 GB DDR4 a 3200 MHz (2 × 8 GB Samsung)
GPU	NVIDIA GeForce RTX 3060 Laptop GPU, 6 GB VRAM GDDR6
VRAM en uso (inferencia)	≈ 4.8 GB
Sistema operativo	Windows 11

El entorno de software se gestiona con `uv` como gestor de dependencias, con un entorno virtual aislado por proyecto. La tabla 2 recoge las versiones mínimas requeridas en el fichero `pyproject.toml`. Las dependencias de aceleración de cómputo se instalan desde el índice oficial de PyTorch con soporte CUDA 12.1.

Tabla 2: Versiones mínimas de las dependencias principales.

Librería	Versión mínima
Python	≥ 3.12
PyTorch	$\geq 2.5.0$ (CUDA 12.1)
Transformers (HuggingFace)	$\geq 4.56.1$
PEFT	$\geq 0.17.1$
bitsandbytes	$\geq 0.43.0$
TRL	$\geq 0.23.1$
accelerate	$\geq 1.10.1$
wandb	$\geq 0.19.0$

.2 Modelo base y cuantización

El modelo base utilizado en todos los experimentos es **Qwen2.5-7B-Instruct**, en su versión cuantizada a cuatro bits. La elección de este modelo responde a tres criterios. Primero, el tamaño de siete mil millones de parámetros permite su carga y ajuste en una GPU de consumo con seis gigabytes de VRAM gracias a la cuantización. Segundo, la variante *instruct* ha sido ajustada previamente con instrucciones diversas en varios idiomas y dominios técnicos, lo que proporciona un punto de partida con conocimiento razonable del vocabulario de Kubernetes sin necesidad de preentrenamiento adicional. Tercero, la familia Qwen2.5 ofrece una arquitectura *decoder-only* estándar, que facilita la integración de cabezales adicionales sobre los estados ocultos del transformer sin modificaciones profundas al flujo de inferencia.

La arquitectura interna del modelo se describe en la tabla 3. La cuantización se aplica mediante bitsandbytes con el esquema NF4 (*Normal Float 4*), que cuantiza los pesos a cuatro bits usando una distribución normal como grid de cuantización. Adicionalmente, se activa la doble cuantización (*double quant*), que cuantiza también las constantes de cuantización del primer nivel, reduciendo el consumo de memoria en aproximadamente 0.4 bits por parámetro. El tipo de dato de cómputo durante el paso forward es `float16`. Esta combinación permite mantener el modelo base en memoria con un consumo inferior a cuatro gigabytes, dejando margen para los adaptadores LoRA, el optimizador y las activaciones intermedias.

.3 Configuración LoRA

Todas las variantes de ajuste fino —`serialized_sft`, `two_head_sft` y sus derivados— comparten la misma configuración de LoRA (*Low-Rank Adaptation*). Los adaptadores se insertan en las cuatro proyecciones de atención del transformer: `q_proj`, `k_proj`, `v_proj` y `o_proj`. No se adaptan los sesgos ni las capas de alimentación directa (*feedforward*).

La elección del rango $r = 8$ responde a un compromiso entre capacidad de adaptación y carga de memoria. Con $r = 8$, cada adaptador introduce dos matrices de bajo rango de dimensiones 3584×8 y 8×3584 , lo que suma aproximadamente 230000 parámetros entrenables por capa de atención. El número total de parámetros ajustados es, en consecuencia, pequeño en

Tabla 3: Arquitectura del modelo base Qwen2.5-7B-Instruct.

Parámetro	Valor
Arquitectura	Qwen2ForCausalLM (decoder-only)
Número de capas	28
Tamaño del espacio oculto (<code>hidden_size</code>)	3584
Cabezas de atención	28 (GQA con 4 cabezas KV)
Tamaño del estado intermedio	18944
Función de activación	SiLU
Ventana de contexto máxima	32768 tokens
Método de cuantización	NF4 + doble cuantización
Tipo de dato de cómputo	float16

Tabla 4: Configuración LoRA común a todos los experimentos de ajuste fino.

Parámetro	Valor
Rango (r)	8
Factor de escala (α)	16
Escala efectiva (α/r)	2.0
Dropout	0.05
Módulos objetivo	q_proj, k_proj, v_proj, o_proj
Adaptación de sesgos	ninguna (<code>bias = none</code>)
Tipo de tarea	CAUSAL_LM

comparación con los 7000 millones del modelo base, lo que hace viable el entrenamiento en la GPU disponible. Rangos mayores aumentarían la capacidad expresiva pero incrementarían también el consumo de VRAM durante el paso de gradientes.

.4 Hiperparámetros de entrenamiento por variante

Este anexo recoge los hiperparámetros de los experimentos que se han ejecutado o documentado durante la evolución del trabajo. No todos ocupan el mismo lugar en la comparación principal: algunos son ramas centrales, otros son ablaciones o ajustes diagnósticos, y otros corresponden a extensiones posteriores como DPO. La tabla 5 recoge la configuración común de los entrenamientos supervisados, mientras que las tablas posteriores separan las decisiones específicas de cada familia experimental.

Tabla 5: Parámetros de entrenamiento comunes a las variantes SFT supervisadas.

Parámetro	Valor
Muestras de entrenamiento	426 (213 muestras $\times$ 2 variantes de prompt)
Muestras de validación	70 (35 muestras $\times$ 2 variantes de prompt)
Tamaño de batch	1
Pasos de acumulación de gradiente	8 (batch efectivo = 8)
Tasa de aprendizaje base	2×10^{-4}
Longitud máxima de secuencia	2048 tokens
Política de tokens de entrada	tokens de prompt enmascarados con -100 ; tokens de salida supervisados
Orden de entrenamiento	secuencial (sin <i>shuffle</i>)
Semilla aleatoria	42
Recuperación ante OOM	omitir el microbatch problemático cuando la variante lo permite

La diferencia más relevante entre `two_head_sft` y las variantes ordinales es la forma en que el cabezal aprende a predecir el nivel. En el primero, el clasificador MLP recibe únicamente el estado oculto del transformer en la posición de alineamiento y produce una distribución categórica sobre las nueve clases. En las variantes ordinales, en cambio, la salida se interpreta como una relación ordenada entre niveles, de modo que los errores cercanos y lejanos no tienen el mismo significado. La variante posicional V1 añade la posición absoluta mediante concatenación final; la variante V2 conserva la misma intuición, pero introduce la posición como una modulación FiLM intermedia.

Tabla 7: Inventario de experimentos respecto al contenido previo del anexo.

Experimento	Estado en el anexo previo	Situación tras la auditoría
baseline zero-shot	documentado en inferencia	Se mantiene como referencia sin hiperparámetros de entrenamiento.
serialized_sft	documentado como una época	Se corrige al entrenamiento principal de 3 épocas y se deja constancia de la ablación de una época.
two_head_sft	documentado	Se mantiene como rama supervisada principal con cabezal categórico.
two_head_ordinal_density	no documentado	Se añade como primera familia ordinal con pérdida ponderada por densidad.
two_head_ordinal_positional	documentado parcialmente	Se mantiene y se precisa como variante V1 con concatenación posicional final.
two_head_ordinal_film_positional	no documentado	Se añade como experimento posterior en curso con modulación FiLM.
serialized_sft_dpo	no documentado	Se añaden las dos ejecuciones DPO completas, con $\beta = 0,10$ y $\beta = 0,30$.

Tabla 9: Parámetros específicos de `serialized_sft`.

Parámetro	Valor
Épocas de entrenamiento	3
Checkpoint principal seleccionado	<code>checkpoint-step-159</code>
Checkpoint de ablación de una época	<code>checkpoint-step-53</code>
Máximo de tokens nuevos (inferencia)	512
Formato de salida objetivo	<code>blocks_tsv_v1</code>
Cabezal estructural	ninguno (nivel codificado en la superficie textual)

Tabla 11: Parámetros específicos de `two_head_sft`.

Parámetro	Valor
Épocas de entrenamiento	3
Checkpoint seleccionado	<code>checkpoint-step-159</code>
Máximo de tokens nuevos (inferencia)	1024
Formato de salida objetivo	<code>content_blocks_v1</code>
Política de alineamiento de nivel	<code>record_prefix_state</code>
<i>Cabecal de nivel (clasificación MLP)</i>	
Tipo	MLP clasificador
Número de clases	9 (niveles 0–8)
Dimensión oculta	256
Dropout	0.05
Función de pérdida	entropía cruzada
Peso relativo de la pérdida de nivel (λ_{level})	1.0

Tabla 13: Serie de variantes ordinales previas a la incorporación explícita de posición.

Run	Diferencia principal	Hiperparámetros específicos
<code>density-v2</code> (2026-05-19)	Primer cabezal ordinal con ponderación por densidad	3 épocas; <code>checkpoint-step-159</code> ; $\lambda_{\text{level}} = 2,0$; pérdida <i>density-weighted ordinal BCE</i> ; 8 umbrales aprendibles; LR base 2×10^{-4} .
<code>thr-lr25</code> (2026-05-20)	Aumento de aprendizaje de umbrales	Misma configuración base; multiplicador de LR de umbrales $\times 25$; LR de umbrales 5×10^{-3} .
<code>mlp3-thr50</code> (2026-05-22)	Aumento conjunto del proyector ordinal y los umbrales	Multiplicador del proyector MLP $\times 3$; LR del proyector 6×10^{-4} ; multiplicador de umbrales $\times 50$; LR de umbrales 1×10^{-2} .
<code>gap05-mlp3-thr50</code> (2026-05-23)	Inicialización centrada de los umbrales	Misma configuración de LR diferenciadas; centro inicial 0.0; separación 0.5; 8 umbrales equidistantes entre $-1,75$ y $1,75$.

Tabla 15: Parámetros específicos de las variantes ordinales con información posicional.

Parámetro	Valor
Épocas de entrenamiento	3
Máximo de tokens nuevos (inferencia)	1024
Formato de salida objetivo	<code>content_blocks_v1</code>
Política de alineamiento de nivel	<code>record_prefix_state</code>
<i>Tasas de aprendizaje diferenciadas</i>	
LM base (LoRA)	2×10^{-4}
Proyector MLP ordinal	6×10^{-4} (multiplicador $\times 3$)
Umbrales aprendibles	1×10^{-2} (multiplicador $\times 50$)
<i>Cabezal ordinal con codificación posicional</i>	
Tipo V1	<code>learned_threshold_ordinal_positional</code>
Tipo V2	<code>learned_threshold_ordinal_film_positional</code>
Número de clases	9 (niveles 0–8)
Dimensiones del proyector	[512, 64]
Dropout por capa	[0.1, 0.0]
Codificación posicional	sinusoidal absoluta, $d = 16, 8$ frecuencias $\{1, 2, 4, 8, \dots, 128\}$
Inyección posicional V1	concatenación final (<i>final_concat</i> , causal)
Inyección posicional V2	modulación FiLM tras la capa de 512 dimensiones (<i>film_after_512</i>)
Configuración FiLM V2	dimensión oculta 128; escala inicial 0.1; última capa inicializada a identidad
Función de pérdida	<i>density-weighted ordinal BCE</i>
Peso relativo de la pérdida de nivel (λ_{level})	2.0
Número de umbrales aprendibles	8 ($\tau_1, \dots, \tau_8$ para 9 clases)
Parametrización de umbrales	$\tau_0 + \sum_i \text{softplus}(\delta_i)$
Inicialización de umbrales	equidistante, centro 0, separación 0.5

Tabla 17: Parámetros específicos de las ejecuciones DPO sobre `serialized_sft`.

Parámetro	Valor
Variante de partida	<code>serialized_sft</code> , <code>checkpoint-step-159</code>
Formato de preferencia	<code>pares chosen/rejected</code> en <code>blocks_tsv_v1</code>
Muestras de preferencia	454
Épocas de entrenamiento	1
Tamaño de batch	1
Pasos de acumulación de gra- diente	8
Tasa de aprendizaje	5×10^{-6}
Longitud máxima de secuencia	2048 tokens
Máximo de tokens nuevos	1024
Pasos entre checkpoints	32
Warmup	0.03
Decaimiento de pesos	0.0
Gradient checkpointing	activado
Recuperación ante OOM	fallar la ejecución
Configuraciones β ejecutadas	0.10 y 0.30
Política de referencia	modelo <code>serialized_sft</code> congelado; log-probabilidades de referencia precomputadas

.5 Configuración de inferencia y evaluación

Durante la evaluación, todos los modelos generan texto en modo greedy, es decir, seleccionando en cada paso el token con mayor probabilidad sin muestreo. Esta elección elimina la varianza introducida por la temperatura y hace que los resultados sean deterministas, lo que facilita la comparación directa entre variantes. El límite de tokens nuevos varía según el formato de salida esperado: la línea base y el SFT serializado generan representaciones más compactas, mientras que el two-head genera únicamente contenido y metadatos de posición sin incluir el nivel en la cadena de texto, lo que puede requerir secuencias más largas para completar el mismo número de líneas.

Tabla 19: Configuración de inferencia por variante experimental.

Parámetro	Baseline	serialized_sft	two_head (todas)
Temperatura	0.0	0.0	0.0
top_p	1.0	1.0	1.0
Máximo de tokens nuevos	320	512	1024
Batch de inferencia	1	1	1

.6 Reproducibilidad y trazabilidad

El diseño del sistema de entrenamiento incorpora varios mecanismos para garantizar que los experimentos puedan reproducirse o reanudarse sin pérdida de estado. En primer lugar, la semilla aleatoria se fija a 42 en todos los experimentos, tanto para la inicialización de los adaptadores LoRA como para los procesos estocásticos internos de PyTorch y Python. En segundo lugar, el conjunto de entrenamiento se procesa en orden secuencial sin mezcla, de modo que el orden de presentación de los ejemplos es idéntico en cada ejecución. En tercer lugar, los puntos de control almacenan no solo los pesos del adaptador sino también el estado completo del optimizador y el planificador de tasa de aprendizaje, lo que permite reanudar el entrenamiento exactamente desde el último paso guardado.

Cuando un microbatch provoca un error de memoria de GPU (*CUDA out of memory*), el sistema lo registra en el fichero `oom_skipped_batches.jsonl`, lo excluye del paso de optimización y continúa con el siguiente. Este mecanismo evita que un único ejemplo problemático interrumpa la ejecución, a costa de que ese ejemplo no contribuya a la actualización del modelo en ese paso. Finalmente, todas las ejecuciones de entrenamiento y validación intermedia se registran en la plataforma Weights & Biases bajo el proyecto `llm-structured-semantic-generation`, con logs de pérdida, métricas de validación por paso de *checkpoint* y configuración completa del experimento.

Evaluación ética y matriz de riesgos

Este anexo conserva el detalle normativo y operativo que respalda el Capítulo 8. La información se separa del cuerpo principal para que el capítulo de impacto social mantenga un hilo argumental más breve, pero sin perder trazabilidad sobre la clasificación regulatoria, los factores RGPD, la checklist ética ni la matriz de riesgos.

.7 Clasificación bajo el AI Act

Tabla 20: Niveles de riesgo del AI Act y aplicabilidad al sistema propuesto.

Nivel de riesgo	Descripción
Inaceptable	Prohibido (art. 5)
Alto riesgo	Anexo III; evaluación de conformidad previa
Riesgo limitado	Art. 52; transparencia y supervisión humana
Riesgo mínimo	Sin obligaciones específicas

El sistema no se encuadra en el Anexo III del AI Act: no opera en educación, empleo, justicia, servicios esenciales ni derechos fundamentales, y tampoco realiza perfilado de personas ni decisiones automatizadas con efectos jurídicos. En consecuencia, la clasificación utilizada en la memoria es la de riesgo limitado, con énfasis en transparencia, documentación técnica y supervisión humana.

.8 Factores RGPD

Si el sistema se distribuyera como herramienta comercial y se empleara para entrenar versiones posteriores del modelo a partir de consultas de usuarios, el tratamiento de esos prompts requeriría una base jurídica explícita. En ausencia de consentimiento, la base aplicable podría ser el interés legítimo del responsable del tratamiento (art. 6.1.f RGPD), condicionado al test de ponderación tripartito: existencia de un interés legítimo real, necesidad y proporcionalidad del tratamiento, y ausencia de prevalencia sobre los derechos e intereses de los afectados.

Tabla 21: Factores de riesgo RGPD relevantes para el sistema en su configuración actual.

Factor analizado	Nivel	Observación
Dataset de entrenamiento sin datos personales (CloudEval-YAML, MIT)	Bajo	No hay metadatos de commits ni identificadores directos de personas.
Tratamiento a escala reducida (213 muestras de entrenamiento)	Bajo	El tamaño limita la complejidad del tratamiento.
Uso de tecnologías emergentes basadas en LLM	Medio	Los modelos generativos requieren un análisis cuidadoso del riesgo de memorización de patrones del corpus.
Posible tratamiento de consultas en fase de explotación	Medio	Solo aplicable si se registran prompts de usuarios en un despliegue futuro; no ocurre en el prototipo actual.

.9 Checklist ética AI HLEG

.10 Matriz de riesgos

Tabla 22: Resumen de cumplimiento de los requisitos éticos del AI HLEG para el sistema propuesto.

Requisito	Estado	Observación
No toma decisiones autónomas sobre asuntos vitales	Sí	—
El usuario puede controlar e intervenir en las operaciones	Sí	—
No subordina, engaña ni manipula al usuario	Sí	—
Tratamiento lícito y transparente de datos personales	Sí	Dataset MIT, sin datos personales
Medidas de seguridad contra brechas y fugas	Sí	Regularización + logging
Diseñado para evitar sesgos algorítmicos	Sí	—
Evita sesgo de representación en el dataset	Parcial	Dataset de 213 muestras; diversificación pendiente
Tiene en cuenta el bienestar de los <i>stakeholders</i>	Parcial	Riesgo residual de automatización excesiva
No reduce la seguridad e integridad en el trabajo	Parcial	470 antipatronos empíricos (Tabla 5.5); revisión experta obligatoria
El usuario es consciente de que interactúa con una IA	Sí	—
Propósito, capacidades y limitaciones se comunican abiertamente	Parcial	Requiere documentación y <i>disclaimers</i> específicos
El sistema permite trazabilidad en todo su ciclo de vida	Sí	Output logging por diseño
Ofrece detalles sobre cómo se toman las decisiones	Parcial	Explicabilidad indirecta mediante análisis latente
Permite supervisión humana durante todo el ciclo de vida	Sí	—

Tabla 23: Principales riesgos del sistema, valoración y medidas de mitigación.

Riesgo	Descripción	Imp./Prob.	Mitigación
Conflicto de licencias	Uso de componentes con licencias restrictivas	Bajo / Bajo	Qwen2.5 (Apache 2.0) y CloudEval-YAML (MIT); ningún riesgo activo
Configuraciones inseguras	Configs sintácticamente válidas pero con anti-patrones de seguridad; 470 instancias en 70 muestras de validación (Tabla 5.5)	Alto / Alto	Revisión experta obligatoria; alineación posterior mediante señales de preferencia
Leak de datos personales	Memorización de patrones del corpus en outputs	Bajo / Bajo	Dataset sin datos personales; regularización durante SFT
Sesgo de evaluadores	Preferencias sesgadas en anotaciones para fases futuras de alineación	Medio / Medio	Selección diversificada del panel; análisis de consistencia entre anotadores
Dependencia técnica	Erosión de competencias del equipo si el sistema se usa sin supervisión	Medio / Alto	Supervisión obligatoria explícita; formación en validación de manifiestos
Sesgo de ecosistema	Tecnologías y patrones del corpus sobrerrepresentados en las salidas	Bajo / Alto	Evaluación con infraestructuras heterogéneas; diversificación del dataset
Sesgo estructural por tamaño de dataset	213 muestras de entrenamiento; posible sobreajuste a los patrones de CloudEval-YAML con capacidad de generalización reducida	Medio / Alto	Ablación de épocas (documentada en el Capítulo 5); evaluación en corpus externo; extensión del dataset en trabajos futuros